



Thupthai Chaiyadecha

Foundational Model for Rover Autonomy

Faculty of Engineering and Applied Sciences
MSc in Astronautics and Space Engineering

MSc
Academic Year: 2025 - 2026

Supervisor: Dr Saurabh Upadhyay

August 2026



Faculty of Engineering and Applied Sciences
MSc in Astronautics and Space Engineering

MSc

Academic Year 2025 - 2026

Thupthai Chaiyadecha

Foundational Model for Rover Autonomy

Supervisor: Dr Saurabh Upadhyay

August 2026

This thesis is submitted in partial fulfilment of the requirements for
the degree of MSc

© Cranfield University 2026. All rights reserved. No part of this
publication may be reproduced without the written permission of the
copyright owner.

Academic integrity declaration

I declare that:

- the thesis submitted has been written by me alone.
- the thesis submitted has not been previously submitted to this university or any other.
- that all content, including primary and/or secondary data, is true to the best of my knowledge.
- that all quotations and references have been duly acknowledged according to the requirements of academic research.

I understand that to knowingly submit work in violation of the above statement will be considered by examiners as academic misconduct.

Abstract

A Mars rover cannot be driven by hand. The one-way communication delay to Earth runs from 3 to 22 minutes, so a rover must judge unfamiliar ground on its own. Building a dedicated terrain classifier for each mission is slow and expensive. Pretrained foundation models offer a faster route, if their general visual knowledge transfers to Mars imagery with no Mars-specific training.

This thesis tests whether that transfer is good enough for a low-cost rover, and which pretraining approach transfers best when the network itself is never retrained. Ten experiment groups evaluate 22 model variants across six pretraining paradigms on AI4Mars, then deploy the strongest small model on a Raspberry Pi 4, in Gazebo, and on a real ExoMy rover.

CLIP reaches 34.8% zero-shot accuracy with no labelled Mars data. DINO self-supervised pretraining proves the most effective of the six. Every DINO Vision Transformer variant reaches at least 89.9% at 1,000 labels per class, and the eleven highest-ranked variants are all DINO or DINO ensembles, because DINO learns view-invariant texture rather than semantic categories. The best competing paradigm reaches 89.55%, and contrastive and autoregressive pretraining stop at 87.8%. A two-model DINOv2 ensemble reaches 94.43%, within 2.24 points of the 96.67% supervised baseline. The deployed model reaches 90.24%, runs at 570 ms per image on the rover's processor, and never passes an unsafe misclassification through in Gazebo despite only 60% terrain accuracy, because a calibrated confidence threshold turns uncertainty into a safe stop. On real camera imagery the failure is specific rather than general. Real soil is classified perfectly and real sand reaches 73.9%, but big rocks are read as soil or sand every time, and the safety guarantee does not survive that one class. Pretrained features alone are therefore not enough for the field without further adaptation.

An extended reactive-safety stack, verified in Gazebo and on hardware, adds hazard recovery, sensor fusion, path planning, and an unattended explore-and-return mission.

Keywords:

Mars terrain, terrain classification, self-supervised learning, DINOv2, vision transformers, zero-shot transfer, edge AI, ROS2

Acknowledgements

I would first like to express my sincere gratitude to my supervisor, Dr Saurabh Upadhyay, for his guidance and support throughout this project. From the very first discussions about the direction of the work, through the choice of models and experiments, and on to the final writing of this report, he has given me his time and his honest advice at every stage. I am grateful for his continued encouragement, and this thesis would not have taken its present shape without him.

I would also like to thank Sebastian Stelter, a doctoral researcher in the Space Laboratory, for his practical help throughout the hardware phase of this project. He introduced me to the laboratory and its equipment, advised me on how to set up and run the rover tests safely, and was always willing to answer my questions. His help saved me a great deal of time and made the experimental work considerably easier.

I acknowledge the use of Claude Code (Anthropic) to assist with writing and debugging the Python and ROS2 code developed for this project. All research decisions, experimental designs, results, and their interpretation are my own, and I take full responsibility for the content of this thesis.

Finally, I would like to thank my family and my friends for their support and encouragement throughout my time at Cranfield University.

Table of Contents

Academic integrity declaration.....	i
Abstract	ii
Acknowledgements	iv
List of Figures.....	viii
List of Tables	x
List of Abbreviations	xii
1 Introduction.....	1
1.1 Background.....	1
1.2 Problem statement.....	2
1.3 Aim and objectives.....	3
1.4 Scope and limitations	4
1.5 Contributions.....	6
1.6 Thesis structure	7
2 Literature Review	8
2.1 Planetary rover autonomy	8
2.1.1 History and autonomy levels	8
2.1.2 Terrain classification and traversability.....	9
2.1.3 Where classical systems fall short.....	11
2.2 Foundation models and vision-language models	11
2.2.1 What is a foundation model?.....	11
2.2.2 CLIP and contrastive pretraining	12
2.2.3 DINOv2 and DINOv3.....	13
2.2.4 BLIP-2 and small vision-language models	14
2.2.5 SAM and Segment Anything	15
2.2.6 Foundation models applied to Mars and space imagery	15
2.3 ROS2 integration and dual-process perception	16
2.3.1 ROS2 as the rover's middleware.....	16
2.3.2 Simulation with Gazebo.....	16
2.3.3 Dual-process perception	17
2.3.4 Formal verification of safety logic	18
2.4 Research gaps and thesis positioning	18
2.4.1 Gap 1, edge VLM deployment on unstructured outdoor terrain.....	18
2.4.2 Gap 2, a CLIP and VLM cascade for rover traversability in ROS2 ...	18
2.4.3 Gap 3, a unified ROS2 foundation-model package	18
2.4.4 Gap 4, few-shot adaptation with no GPU fine-tuning	19
2.4.5 Gap 5, failure-mode characterisation for foundation-model rover decisions	19
2.4.6 Gap 6, measured CPU contention under concurrent inference.....	19
2.4.7 Gap 7, a cross-paradigm comparison on Mars geological texture ...	19
2.5 Summary	20

3 Methodology	22
3.1 Overview	22
3.2 Hardware platform	23
3.2.1 Development machine.....	23
3.2.2 Target platform: ExoMy	23
3.3 Software environment.....	25
3.4 Dataset: AI4Mars	26
3.5 Foundation model selection	27
3.5.1 Choosing DINOv2+reg ViT-S/14 as the deployed model	27
3.5.2 The four models in the cascade and comparison set	28
3.6 Evaluation framework	29
3.6.1 How many labels are enough: the 1000-shot protocol	30
3.7 Stage 1: CLIP zero-shot and prompt engineering	32
3.8 Stage 2: vision-language model evaluation	32
3.9 Stage 3: the ROS2 cascade pipeline	33
3.9.1 Design	33
3.9.2 Safety mechanisms	35
3.9.3 Package structure	36
3.10 Deploying on the real rover.....	36
3.11 Gazebo simulation environment	37
3.11.1 World design	37
3.11.2 Traversability protocol	38
3.11.3 Three known simulation-fidelity gaps	40
3.12 Extending the reactive controller into an autonomy stack	41
3.13 Summary	47
4 Results	48
4.1 Overview	48
4.2 CLIP zero-shot classification.....	48
4.3 Prompt engineering.....	48
4.4 Few-shot linear probe	49
4.5 Vision-language model comparison	49
4.6 Cascade pipeline in Gazebo (CLIP + SmolVLM)	50
4.7 Cross-paradigm model survey (22 models, six pretraining paradigms) ..	51
4.8 Deployment-relevant checks.....	57
4.9 Cross-sensor and cross-dataset transfer	58
4.10 DINOv2 traversability in Gazebo.....	59
4.11 Raspberry Pi hardware benchmark.....	62
4.12 Real camera deployment	63
4.13 Extended reactive-safety and autonomy results	67
4.14 Chapter summary	69
5 Discussion	71
5.1 Introduction	71

5.2 Why DINO self-supervision wins.....	71
5.3 The cascade pipeline and CPU contention	74
5.4 Sim-to-real: how the domain gap changes shape	74
5.5 Comparison with related work.....	77
5.6 Limitations.....	77
5.7 What the hardware phase added	81
5.8 Answering the research question.....	82
5.9 Future work.....	83
5.10 A note on simulation and hardware versions	84
6 Conclusion.....	86
6.1 Summary of work.....	86
6.2 Answering the research question.....	87
6.3 Contributions.....	88
6.4 What remains for future work.....	90
6.5 Closing statement	91
References	92
Appendices.....	100
Appendix A Code and repository	100
Appendix B Reproducibility map	106
Appendix C Full 22-model results table	112
Appendix D Negative, null, and low-N results	114

List of Figures

Figure 3.1: The assembled ExoMy rover used in this thesis. (a) three-quarter view, (b) front view showing the camera mast, (c) side view showing the LiDAR and the Raspberry Pi's ports, (d) rear view.	24
Figure 3.2: Label efficiency of DINOv2+reg ViT-S/14 under the balanced and practical labelling strategies.....	31
Figure 3.3: The perception-to-motion pipeline. LiDAR and IMU enter only as vetoes; neither proposes a terrain class. The four-tier speed policy shown is a Gazebo result, for the reason given in Section 3.11.3.	34
Figure 3.4: The Gazebo benchmark world. (a) plan view of the four 15x12 m terrain quadrants on the 30x24 m base, (b) oblique view from the sand corner, (c) the rock zone at rover camera height, (d) a zone boundary at rover camera height.....	38
Figure 3.5: What the rover's camera sees at each of the five zone positions, which is the input DINOv2 classifies.	39
Figure 3.6: The reactive-explorer state machine as shipped. Red transitions end the mission.	45
Figure 4.1: AI4Mars accuracy against labelled images per class for CLIP and two DINOv2 encoders.	49
Figure 4.2: 1000-shot AI4Mars accuracy for all 22 model variants, coloured by pretraining paradigm.....	53
Figure 4.3: Best 1000-shot AI4Mars accuracy within each pretraining paradigm.	54
Figure 4.4: AI4Mars accuracy against encoder size for the DINOv2 and DINOv3 families.	55
Figure 4.5: Row-normalised confusion matrix for DINOv2 ViT-L/14 at 1000-shot.	57
Figure 4.6: Coverage-risk trade-off across the confidence threshold sweep....	58
Figure 4.7: Classifier confidence at each of the five Gazebo zone positions....	60
Figure 4.8: AI4Mars accuracy against feature-extraction latency, with the deployed model marked.	62
Figure 4.9: The laboratory sandpit, with printed Mars terrain panels around the walls. (a) the arena empty, (b) and (c) two of the obstacle layouts tested.	66
Figure 4.10: Deployed-model accuracy by terrain class and image source. Error bars are Wilson 95% confidence intervals; the dashed line is the same model's 90.24% accuracy on the AI4Mars test set.	67

Figure 5.1: t-SNE projection of DINOv2 ViT-L/14's 1024-dimensional CLS features for the 287-image AI4Mars test set, with misclassified images ringed. Distances between clusters in a t-SNE projection carry no quantitative meaning; only the grouping and the overlaps should be read.73

Figure 5.2: Deployed-model accuracy by terrain class on AI4Mars and on the rover's own camera. Big rock has no AI4Mars bar, and zero real-camera accuracy, because it is not in the probe's label set. 75

List of Tables

Table 1.1: The L1-L6 autonomy stack and the status of each layer in this thesis.	5
Table 2.1: Comparison against related work combining foundation models with rover autonomy.....	20
Table 3.1: Raspberry Pi hardware platform specification.	24
Table 3.2: Software environment and library versions.....	25
Table 3.3: AI4Mars dataset properties.	26
Table 3.4: Vision-language models evaluated and their Raspberry Pi feasibility.	27
Table 3.5: Summary of the ten experiment groups. Experiment 5b is the real- camera counterpart of Experiment 5 rather than a group of its own.	29
Table 3.6: Label-efficiency sweep results by N per class and labelling strategy.	30
Table 3.7: Comparison of planetary rover simulators.	37
Table 3.8: Behaviours extending the reactive controller, and how each was verified.....	42
Table 3.9: The reactive explorer's states and what the rover does in each.	44
Table 4.1: Top-ranked models by 1000-shot AI4Mars overall accuracy (full 22- model table in Appendix C).....	51
Table 4.2: Deployed-model accuracy on 72 photographs taken with the rover's own camera, by terrain class.	64
Table 4.3: Live Gazebo outcome for each extended behaviour.	68
Table 5.1: Effect of adding big rock to the label set. Same encoder, same features, same 120 held-out images.	78
Table A.1: Repository structure.	101
Table A.2: Commands to build and run the project.	102
Table A.3: Video record of eight autonomous hardware runs, 4 August 2026.	105
Table B.1: Core AI4Mars classification results: headline numbers, scripts, and outputs.....	106
Table B.2: Cross-paradigm survey (22 models, six pretraining paradigms): headline numbers, scripts, and outputs.	108

Table B.3: Cross-sensor and cross-planet domain-gap findings: headline numbers, scripts, and outputs.....	109
Table B.4: Raspberry Pi hardware benchmark: headline numbers, scripts, and outputs.....	109
Table B.5: Gazebo simulation and the extended autonomy stack: headline numbers, scripts, and outputs.....	110
Table C.1: Full results for all 22 evaluated models across six pretraining paradigms.....	112
Table D.1: Negative, null, and low-N results.....	114

List of Abbreviations

Abbreviation	Full form
A*	A-star, a graph-search algorithm that finds a least-cost path between two points
AI4Mars	A dataset for Mars terrain classification using Curiosity NAVCAM imagery (Kerner et al., 2022)
AIMv2	Autoregressive Image Models, version 2 (Apple, 2024)
ARM	Advanced RISC Machines (processor architecture family)
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
BLIP-2	Bootstrapping Language-Image Pre-training, version 2
C	Inverse regularisation strength of a logistic regression classifier; a smaller C forces a simpler classifier
CAD	Computer-Aided Design
CI	Confidence Interval
CLARAty	Coupled Layer Architecture for Robotic Autonomy (NASA JPL)
CLIP	Contrastive Language-Image Pre-training
CLS	Classification token, the special prepended token in Vision Transformer whose output represents the whole image
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture (NVIDIA parallel computing platform)
DINO	self-Distillation with NO labels, the self-supervised pretraining paradigm this thesis identifies as the most effective for Mars terrain
DINOv2	self-Distillation with NO labels, version 2 (Meta AI, 2024)
DINOv3	self-Distillation with NO labels, version 3 (Meta AI, 2025)
DLR	Deutsches Zentrum für Luft- und Raumfahrt (German Aerospace Center)
DTM	Digital Terrain Model
ECE	Expected Calibration Error
ESA	European Space Agency

Abbreviation	Full form
EVA-02	Exploring Visual Anchors, version 02
ExoMy	ESA open-source educational rover platform (six-wheel, 3D-printed)
FPS	Frames Per Second
FSM	Finite State Machine
GESTALT	Grid-based Estimation of Surface Traversability Applied to Local Terrain (rover hazard-avoidance system)
GPS	Global Positioning System
GPU	Graphics Processing Unit
HazCam	Hazard Camera (close-range cameras on Mars rovers)
HiRISE	High Resolution Imaging Science Experiment (instrument on Mars Reconnaissance Orbiter)
I2C	Inter-Integrated Circuit (serial communication protocol)
IMU	Inertial Measurement Unit
INT4	4-bit integer (quantisation precision for neural network inference)
INT8	8-bit integer (quantisation precision for neural network inference)
IoU	Intersection over Union
JPEG	Joint Photographic Experts Group (image compression format)
JPL	Jet Propulsion Laboratory (NASA facility, Pasadena, California)
LiDAR	Light Detection And Ranging
LLaVA	Large Language and Vision Assistant
LLM	Large Language Model
LTS	Long-Term Support (software release with an extended maintenance period)
LVD	Large-scale Vision Dataset, the curated pretraining corpus behind DINOv2 (LVD-142M) and DINOv3 (LVD-1689M); the suffix is the image count
MAARS	Machine learning-based Analytics for Automated Rover Systems (NASA JPL)
Mars-Bench	Multi-task Mars surface benchmark comprising 20 datasets (Kerner et al., 2024)

Abbreviation	Full form
MastCam	Mast Camera (colour imaging instrument on Curiosity and Perseverance)
MIM	Masked Image Modelling
mIoU	mean Intersection over Union
MOMO	Mars Orbital MOdel for Orbital applications (Purohit et al., 2026)
MSc	Master of Science
NASA	National Aeronautics and Space Administration
NAVCAM	Navigation Camera (stereoscopic cameras on Mars rovers)
ONNX	Open Neural Network Exchange
PWM	Pulse-Width Modulation
RADIO	Multi-teacher distillation model combining DINOv2, CLIP, and SAM2 features
RAM	Random Access Memory
RASA	Robust Adaptive Self-Attention (positional disentanglement method used in Franca)
RGB	Red-Green-Blue (colour image format)
ROS	Robot Operating System
ROS2	Robot Operating System 2 (current generation, Humble Hawksbill used in this thesis)
ROSA	Robot Operating System Agent (LLM-based rover command interface, NASA JPL)
RPi	Raspberry Pi (single-board computer)
RUGD	Robot Unstructured Ground Driving (Earth off-road terrain dataset)
SAM	Segment Anything Model (Meta AI, 2023)
SAM2	Segment Anything Model 2 (Meta AI, 2024)
SigLIP2	Sigmoid Loss for Language-Image Pre-training, version 2
SimCLR	Simple framework for Contrastive Learning of visual Representations
SLAM	Simultaneous Localisation and Mapping
SPOC	Self-supervised Pretraining for Object Classification (deep learning terrain classifier for Mars rovers)
t-SNE	t-Distributed Stochastic Neighbour Embedding

Abbreviation	Full form
THEMIS	Thermal Emission Imaging System (instrument on Mars Odyssey orbiter)
URDF	Unified Robot Description Format
USB	Universal Serial Bus
VIPER	Volatiles Investigating Polar Exploration Rover (NASA lunar rover)
ViT	Vision Transformer
VLM	Vision-Language Model
VQA	Visual Question Answering
VRAM	Video Random Access Memory (memory on a graphics card)
WSL	Windows Subsystem for Linux

1 Introduction

1.1 Background

Mars is one of the most challenging environments in which to operate an autonomous robot. A rover on the surface has to cross rough, unknown ground with no live human control. The main reason is the time it takes a signal to travel between Earth and Mars. One-way, this delay runs from about 3 minutes to about 22 minutes, depending on where the two planets are in their orbits (Nesnas et al., 2021). A command and its reply therefore take 6 to 44 minutes round trip, which is far too slow to steer a rover by hand through changing or dangerous ground. When a rover sees an obstacle, the operators on Earth may not get the image and send back a response for up to 44 minutes. The rover must therefore understand the ground and make safe choices on its own. This is not a convenience, but a basic requirement for safe Mars driving.

Today's Mars rovers already do this. Curiosity and Perseverance classify terrain and spot hazards using geometry-aware methods, stereo vision, and lidar. These systems were designed and tuned for Mars over years of mission work (Bajracharya et al., 2008). They work well, but they need deep expertise, heavy compute, and mission-specific calibration. Smaller and cheaper rovers cannot afford any of that. The platform used in this thesis is the European Space Agency (ESA) ExoMars educational rover, whose onboard computer is a single Raspberry Pi 4, which rules out GPU-based perception.

At the same time, machine learning has produced a new kind of model. Foundation models, and the vision-language models (VLMs) among them, are large networks trained on internet-scale image and text data. They learn broad visual knowledge that transfers to new tasks without task-specific retraining (Bommasani et al., 2021). Most of these models appeared only from 2021 onward, so using them for rover perception is a newly practical idea. Models such as CLIP (Radford et al., 2021), BLIP-2 (Li et al., 2023), SmolVLM (Marafioti et al., 2025), and DINOv2 (Oquab et al., 2024) can classify images they were never trained on, in zero-shot or few-shot settings. Zero-shot means no labelled

examples of the new task, and few-shot means only a handful. CLIP learns by matching images to text. DINOv2 learns visual features on its own, with no text at all, and these features are good at telling textures apart.

This is the opportunity the thesis explores. If a pretrained model can be adapted to judge terrain and traversability for a planetary rover on low-cost edge hardware, rover teams would no longer need to build a custom classifier and label a large dataset for GPU training. They could take a pretrained model and adapt it with very little work, cutting both the development time and the labelling cost.

1.2 Problem statement

Foundation models do well on computer vision benchmarks, but three open problems block their direct use on a planetary rover.

The first problem is the domain gap and the choice of pretraining approach. These models are trained on internet images, which do not include Mars navigation camera (NAVCAM) photos, so how well their visual knowledge carries over to Mars is unknown. The visual foundation models of 2024 to 2026 also cover many different training objectives, including contrastive learning (CLIP, SigLIP2), self-supervised distillation (DINOv2, DINOv3), masked image modelling (EVA-02), autoregressive modelling (AIMv2), and multi-teacher distillation (RADIO). Each of these can be used without retraining the network at all. The network's weights are left exactly as the original authors trained them, which is what "frozen" means throughout this thesis, and only a small classifier is trained on the numbers it outputs. Those outputs are its frozen features, and reusing them is what makes the expensive pretraining worth inheriting rather than repeating. Which pretraining objective produces the most transferable frozen features for reading geological texture on Mars is unknown.

The second problem is compute. The best VLMs need a GPU, which the rover does not have. It is unknown whether a sub-billion-parameter edge VLM can classify terrain usefully on a CPU alone, or what inference speed is possible on the Raspberry Pi 4's ARM Cortex-A72 processor, a quad-core 1.5 GHz chip with no GPU acceleration and a shared 4 GB of memory.

The third problem is integration and pipeline design. VLMs are usually tested as standalone Python scripts, and putting one inside a real-time ROS2 pipeline adds latency, threading, and resource problems that the VLM literature does not cover. In particular, no earlier work has tested a dual-process pipeline, one that pairs a fast visual model with a slow reasoning VLM, for rover traversability in ROS2 and Gazebo, and nobody has measured how much two neural networks running at once slow each other down on a single-board computer.

1.3 Aim and objectives

The aim of this thesis is to find out whether pretrained foundation models can give accurate, real-time terrain classification and traversability assessment for a low-cost planetary rover on edge hardware, with no Mars-specific training. This aim is stated here as a single research question, and Chapter 6 returns to it and answers it directly.

Can pretrained foundation models classify Mars terrain accurately for a low-cost rover, with no Mars-specific training, and which training approach gives the best frozen terrain features?

The experimental programme meets this aim through four objectives.

1. Establish a baseline by evaluating how accurately CLIP classifies Mars terrain (soil, bedrock, sand) in zero-shot and few-shot settings, and identify what limits its zero-shot result.
2. Compare the pretraining approaches at matched model size to find which one gives the strongest frozen terrain features.
3. Measure the accuracy of frozen encoders against the supervised DeepLabv3+ baseline of 96.67%, and test whether a feature ensemble closes the gap.
4. Deploy a foundation-model classifier on a Raspberry Pi 4 inside a ROS2 traversability controller, and test it in Gazebo and on the real rover.

The work counts as successful against three criteria. First, a frozen encoder should reach terrain accuracy close to the supervised DeepLabv3+ ceiling of 96.67%. Second, the controller should never pass an unsafe misclassification

through to the rover in the Gazebo test, so that every classification error ends in a safe stop. Third, the chosen model should run fast enough for safe terrain sampling on the Raspberry Pi 4.

1.4 Scope and limitations

The thesis is scoped for a five-month MSc project, running from April to August 2026. It is bounded by the following limits.

- No training from scratch. Every model is adapted from a public pretrained checkpoint through prompting, zero-shot evaluation, or a light linear probe on frozen features. A linear probe is a small classifier, here a logistic regression, trained on the features a frozen network produces, leaving the network itself untouched.
- No GPU fine-tuning. The development machine has no CUDA GPU for fine-tuning, so all work uses frozen encoders and CPU inference.
- Simulation first, then hardware. The cascade pipeline is tested in Gazebo and then on the real ExoMy rover. The two phases are sequential rather than continuous. The simulation results were produced before the reactive-safety controller was revised during the hardware phase, and Chapter 5 states where this applies.
- One dataset. All classification uses AI4Mars, a public dataset of expert-labelled Mars rover images, and specifically its gold-standard test set of 287 Curiosity NAVCAM images. Other Mars datasets, cameras, or Perseverance NAVCAM are not tested.
- Monocular RGB only. ExoMy carries a single RGB camera. Depth estimation, stereo vision, and multi-spectral sensing are out of scope.
- Layers 1 to 3 of a six-layer autonomy stack. The thesis delivers terrain perception, reactive safety, and partial traversability. Localisation, path planning, and mission autonomy (L4 to L6) are future work, though small working slivers of the higher layers are shown.

The six-layer stack in Table 1.1 places the thesis contribution in context. The full results for each layer are in Chapter 4 and Chapter 5.

Table 1.1: The L1-L6 autonomy stack and the status of each layer in this thesis.

Layer	Function	Thesis contribution	Verified on
L1	Terrain semantics: classify terrain from the camera	Core contribution. 90.24% on AI4Mars with the deployed DINOv2+reg ViT-S/14 model, a 94.43% ensemble ceiling, and 22 models compared.	Dataset, simulation, hardware
L2	Reactive safety: turn confidence into stop, slow, or go	Implemented and validated. Perfect safety compliance in the Gazebo benchmark with calibrated gating, plus hazard recovery and stuck-in-sand recovery, confirmed across 10 of 10 real-hardware rounds with no terrain-judgement failures.	Simulation, hardware
L3	Traversability: a continuous cost from class and confidence	Partial. Discrete gating is validated, and a continuous cost mapping was added and live-verified.	Simulation, hardware
L4	Localisation: where is the rover	Partial. Simultaneous localisation and mapping (SLAM) works at about 0.18 m mean error when given wheel odometry. Odometry-free localisation, the case the real rover needs, is still open.	Simulation only
L5	Path planning: how to reach a goal	Not attempted in full. A lightweight A* (A-star) planner still reached a hazard-crossing goal end to end.	Simulation only
L6	Mission autonomy: what to do next	Post-thesis in general. Small slivers work live: autonomous waypoint sequencing, frontier exploration, explore-then-return-home, and uncertainty-driven re-observation.	Simulation only

This ordering is not just a concept. It matches how autonomy is actually deployed on a real mission. Perseverance's AutoNav system drove 88% of the 17.7 kilometres it covered in its first Martian year on its own, and its AEGIS system picks science targets between communication windows without waiting for ground commands (Verma et al., 2023). Both sit on top of reliable terrain

perception (L1) and reactive safety (L2). This thesis contributes to L1 and L2 on low-cost hardware, with partial progress on L3.

1.5 Contributions

The thesis makes six main contributions, each reported in full in the chapter given.

1. A prompt engineering study for Mars terrain with CLIP, which found a soil-bedrock trade-off in CLIP's zero-shot feature space and raised accuracy from 34.8% to 54.4% with a hybrid ensemble prompt (Chapter 4, Section 4.3).
2. A comparison of 22 foundation model variants across six pretraining families, the most complete frozen-encoder comparison on AI4Mars to date, which establishes that DINO self-supervised pretraining gives the strongest Mars terrain features (Chapter 4, Section 4.7).
3. The best reported frozen-encoder accuracy on AI4Mars, at 94.43%, achieved with no GPU fine-tuning and no Mars-specific training, and within 2.24 percentage points of the supervised DeepLabv3+ ceiling of 96.67% (Chapter 4, Section 4.7).
4. The first full-test VLM result for SmolVLM-256M on AI4Mars, including the first VLM sand result and a prompt ablation, which is an experiment that removes one element at a time to show how much each contributes (Chapter 4, Section 4.5).
5. An open-source ROS2 perception package, `fm_perception`, together with a Raspberry Pi 4 deployment benchmark and a dual-process cascade validated in Gazebo. The deployed model runs at 570 ms per image using 258 MB of RAM, and temperature scaling cuts its Expected Calibration Error (ECE), a measure of how well a model's stated confidence matches how often it is actually right, by 80.8% (Chapter 4, Sections 4.6, 4.8, and 4.11).
6. An extended reactive-safety and autonomy stack built on top of the perception system and confirmed on real ExoMy hardware, together with two honest negative results: a cross-planet generalisation probe showing

that the frozen features separate Mars from Earth terrain without unifying them, and a real-camera domain gap that is confined almost entirely to the big-rock class (Chapter 4, Sections 4.9, 4.12, and 4.13, and Chapter 5).

1.6 Thesis structure

The rest of the thesis is organised as follows.

Chapter 2 reviews planetary rover autonomy, foundation models, vision-language models, self-supervised learning, and edge AI, and identifies the gaps that motivate the work.

Chapter 3 describes the hardware, the software environment, the AI4Mars dataset, the choice of models, and the protocol for each experiment.

Chapter 4 presents the results of ten experiment groups, from the CLIP zero-shot baseline through to the real ExoMy hardware deployment.

Chapter 5 reads those results against the research question, compares them to earlier work, and sets out the limitations.

Chapter 6 answers the research question with the key numbers and gives recommendations for future work.

2 Literature Review

2.1 Planetary rover autonomy

2.1.1 History and autonomy levels

Planetary rovers have grown more autonomous over three decades. Sojourner, launched in 1997, drove only on daily command sequences planned by operators on Earth. Spirit and Opportunity, launched in 2004, added onboard hazard detection through GESTALT, a grid-based terrain-traversability algorithm, allowing short autonomous drives under a metre. The Jet Propulsion Laboratory (JPL) CLARAty software architecture split rover control into a reactive layer for hardware and a deliberative layer for planning, a split that still shapes rover software today (Nesnas et al., 2021). Curiosity, launched in 2012, added AutoNav, using stereo vision and geometry-based obstacle detection to drive tens of metres per sol without human review. Perseverance, launched in 2021, is the current state of the art, driving most of its first Martian year on its own and choosing its own science targets between communication windows (Verma et al., 2023).

The driver behind this progression is the Earth-Mars communication delay, from about 3 to 22 minutes one way (Nesnas et al., 2021). A round trip of up to 44 minutes rules out real-time teleoperation, and a rover that meets a hazard while waiting for a human reply may already be damaged or stuck by the time it arrives. Each autonomy generation is a response to that constraint.

Despite this progress, operational rover autonomy is still mostly geometry-based, relying on stereo depth, slope estimates from an inertial measurement unit (IMU), and rule-based traversability thresholds (Bajracharya et al., 2008). These systems work well but need mission-specific tuning and cannot handle a new terrain type without retraining or revalidation. Longer traverses and more ambitious mission targets are pushing the field towards more flexible, semantically aware perception.

2.1.2 Terrain classification and traversability

Terrain classification, labelling what type of ground is in an image, is the first step towards traversability assessment. The AI4Mars dataset (Swan et al., 2021) is the standard Mars terrain benchmark and the one used throughout this thesis. It holds 16,064 training images and a gold-standard test set of 287 expert-labelled Curiosity NAVCAM images across four classes: soil, bedrock, sand, and big rock. Big rock is both the rarest class and the hardest to classify, covering only about 2% of labelled pixels in Mars surface imagery (Li et al., 2024), so it stays under-represented no matter the method used. A supervised DeepLabv3+ model reaches 96.67% overall accuracy on this test set, and that is the ceiling this thesis measures its own results against.

Earlier work predates large labelled datasets. Helmick et al. (2009) combined stereo-derived surface normals with slip prediction for a Terrain Adaptive Navigation system, Rothrock et al. (2016) applied convolutional networks to rover imagery (SPOC), and JPL's MAARS programme (Ono et al., 2020) joined scene interpretation, traversability assessment, and path planning into one pipeline tested on the Athena rover.

More recent work targets uncertainty and domain shift. Endo et al. (2024) extend traversability prediction to slip probability, separating uncertainty that comes from genuinely ambiguous ground from uncertainty that comes from the model not having seen that terrain before. Mattamala et al. (2025) show that features taken from a frozen DINOv2, meaning one whose weights are never updated, support fast self-supervised traversability learning on a real outdoor robot, adapting to a new environment in under five minutes with no manual labels. That is independent evidence, outside the Mars domain, that the approach used in this thesis works on deployed robots.

Two further lines of work bear directly on the design this thesis adopts. MLNav (Daftry et al., 2022) puts a learned heuristic and a model-based checker in the same planner on Perseverance, letting the learned component propose candidate paths while the checker retains the authority to reject them, which is the same division of authority this thesis applies one layer down at perception.

Xie et al. (2025) fuse frozen visual features with IMU angular velocity and acceleration to catch terrain that looks safe in an image but exceeds a rover's tilt tolerance, which is the case for the LiDAR and IMU fusion built in Section 3.12.

All of these systems share a limitation. They need environment-specific labelled data, mission-specific calibration, and GPU compute for training, which is a real barrier for a low-cost rover working in terrain it has not seen before. AI4Mars removes the labelling barrier for Curiosity-class cameras, but not the compute or calibration barrier.

Mars rover imagery also has a structural problem for self-supervised learning. Ward et al. (2022) show that around 90% of consecutive Curiosity NAVCAM frames look visually similar, which breaks a standard assumption behind contrastive learning, the training scheme that teaches a model to pull matching pairs of inputs together and push unrelated ones apart. Their analysis finds that broad ImageNet-pretrained visual priors matter more than Mars-specific self-supervision when the target domain is this repetitive, which supports the choice made here to use an ImageNet-pretrained DINOv2. Goh et al. (2022) give a related data point, reaching 91.1% accuracy with only 1% of the labels from a SimCLR model pretrained on AI4Mars imagery, but that pretraining step does not carry over to a new sensor or mission. The approach in this thesis reaches a comparable 90.24% while working on any camera without it.

The evaluation landscape has also grown past AI4Mars alone. Mars-Bench (Purohit et al., 2025) covers 20 datasets spanning Curiosity NAVCAM, Perseverance MastCam and HazCam, and Mars Odyssey THEMIS imagery. MastCam is colour and NAVCAM is grayscale, so a model trained on AI4Mars does not transfer directly. This thesis measures that gap at 65.30 percentage points, 90.24% on AI4Mars against 24.94% zero-shot on Mars-Bench, which makes cross-sensor transfer a measured problem rather than a theoretical one. Arafin et al. (2025), surveying off-road terrain classification from 2015 to 2025, name that same cross-sensor generalisation and the shortage of labelled off-road data as the field's two most persistent open problems.

2.1.3 Where classical systems fall short

Every system above shares the same design. An engineer studies the target terrain, defines what counts as a hazard, and trains or writes code to detect exactly that. This works, but it has three limits.

First, the model only knows what it was shown. A model trained on Gale Crater NAVCAM images will not reliably classify images from a different camera, a different site, or a Gazebo simulation without retraining. Second, training needs a GPU, and the ExoMy rover used here carries a Raspberry Pi 4 with none, so any approach needing gradient-based training at deployment time is ruled out. Third, these systems can only answer "what class is this?" They cannot explain a decision or answer a follow-up such as "how confident are you, and why?"

Foundation models take a different route. Instead of Mars-specific training by a team of engineers, they learn from hundreds of millions of ordinary internet images and captions, building broad visual knowledge before ever seeing Mars. Whether that general knowledge transfers well enough to be useful on Mars terrain, with no Mars-specific training at all, is the central empirical question this thesis answers.

2.2 Foundation models and vision-language models

2.2.1 What is a foundation model?

Nobody learns to tell rock types apart from a Mars-specific geology textbook. Years of looking at many different surfaces build an intuition for texture and structure, and that intuition still works the first time a person sees a Mars NAVCAM photo.

Foundation models work on a similar idea. Instead of training on a narrow, task-specific dataset, they train once on hundreds of millions of images and captions from across the internet, then carry that knowledge into new tasks without retraining. Bommasani et al. (2021) coined the term for this, describing one large model, trained once, adapted to many tasks through a text prompt, a handful of labelled examples, or a light classifier trained on its features. Two properties

make this work. Emergence means the model develops capabilities nobody explicitly trained it for, such as recognising a rare terrain type. Transferability means that because the training data is broad, the learned features carry over well even to an unfamiliar domain such as Mars.

The idea started with language, where Devlin et al. (2019) showed that a model trained on billions of words (BERT) could be adapted to many tasks with little fine-tuning. The shift to vision came with CLIP (Radford et al., 2021), and by 2024 the same abilities had been compressed to single-board-computer size. The sections below take the models this thesis actually uses in turn.

Foundation models for vision come in a few forms. Contrastive models such as CLIP match images to text and support zero-shot classification. Generative models, the vision-language models (VLMs) such as BLIP-2 and SmolVLM, answer open questions about an image in words. Self-supervised models such as DINOv2 learn purely from visual patterns and build detailed texture representations. Awais et al. (2025) and Jang et al. (2024) survey these families and both name real-time, on-device deployment as an open problem for robotics, which is the gap this thesis responds to.

2.2.2 CLIP and contrastive pretraining

CLIP (Radford et al., 2021) learns from 400 million image-text pairs by asking one question of each pair: does this image match this caption? Training pulls matching image-caption pairs together in a shared space and pushes unrelated pairs apart.

This makes zero-shot classification possible without any labelled Mars images. To classify a NAVCAM image, CLIP compares it against one text description per terrain class, for example "a Mars rover image of sandy terrain", and picks the class with the highest match score. No Mars training is needed.

The weakness is that the result depends heavily on how the text is written, a sensitivity called prompt engineering (Khattak et al., 2023). Testing twelve prompt styles on the 287-image AI4Mars test set raises accuracy from 34.8% with the default prompt to 54.4% with the best one, a 19.6-point gain. It also exposes a

real weakness, in that CLIP consistently confuses soil and bedrock, because the two look similar in a text description even though they look different geologically (Section 4.3).

2.2.3 DINOv2 and DINOv3

CLIP links images to words. DINO learns visual texture with no words at all. The model is shown two different crops of the same image and learns to produce the same internal representation for both. The only way to succeed consistently at this task is to learn what stays constant across views, such as surface texture and grain pattern. Nobody tells the model what bedrock looks like, and it works this out on its own.

DINOv2 (Oquab et al., 2024) applies this at scale on 142 million filtered images. It uses a Vision Transformer (ViT), which splits an image into small square patches, processes them together, and keeps one extra classification token (CLS) that summarises the whole image. No text or labels are used anywhere in training. This gives DINOv2 an interpretable property, since its attention maps, which show how strongly each patch influenced the result, reveal which regions drove a decision. On Mars terrain, this thesis confirms (Section 4.8) that DINOv2 attends to crack networks for bedrock, diffuse grain texture for soil, and smooth ripple gradients for sand, the same cues a geologist would use, learned with no geological supervision.

DINOv2 with registers (Darcet et al., 2024) fixes a known artefact. In featureless regions such as blank sky, some patch tokens develop erratic, globally attending behaviour that distorts the attention map. Adding a small number of extra "register" tokens gives that global information somewhere else to go, leaving the normal patch tokens focused on local texture, for cleaner attention maps and a 0.34 percentage point gain at 1000-shot. The model deployed in this thesis is DINOv2+reg ViT-S/14, 21M parameters, with 4 registers. DINOv3 (Siméoni et al., 2025) keeps the same training objective but scales the data from 142 million to 1.69 billion images, raising accuracy across the board. DINOv3 ViT-S/16 reaches 90.2% at 1000-shot against 89.9% for DINOv2 ViT-S, and DINOv3 ViT-L/16 reaches 92.3%.

DINO outperforms CLIP on Mars terrain because the two models optimise different things. CLIP learns what things are called, and Martian bedrock has no everyday word that clearly separates it from compacted soil, so CLIP's language-grounded features confuse the two. DINOv2, trained only on visual consistency, learns that bedrock has angular fracture networks and soil has rounded grain patterns, a visual distinction rather than a naming one. At 1000-shot, DINOv2 ViT-S reaches 84.78% bedrock accuracy against CLIP's 79.30%, and the gap widens at low shot counts where CLIP's language bias matters most.

A related model, Franca (Venkataramanan et al., 2026), tackles a smaller problem in the same family. DINOv2 patch tokens mix texture with position, so two patches of the same bedrock texture at different points in the image can produce slightly different features. Franca's RASA module learns to subtract the positional component, leaving a texture-only feature. This thesis tests three Franca ViT-L/14 feature variants to check whether position-free features separate the terrain classes better than standard DINOv2 features (Section 4.7).

2.2.4 BLIP-2 and small vision-language models

CLIP scores how well an image matches a caption but cannot explain itself in words. BLIP-2 (Li et al., 2023) goes further. Given an image and a question, it writes a free-text answer, for example to "what type of terrain is this: soil, bedrock, sand, or rock?" It does this by bridging a large frozen vision encoder (ViT-G, 1.8B parameters) and a frozen language model (OPT-2.7B) with a small trained module called Q-Former, which keeps most of the model unchanged and most of the compute manageable.

On Mars terrain, BLIP-2 reaches 80.0% accuracy on a 20-image AI4Mars sample, with perfect soil recall but zero bedrock accuracy, the same soil-bedrock weakness seen in CLIP. Its bigger limitation is speed, at 753 seconds per image on CPU, and it needs over 6 GB of RAM, which rules it out on a 4 GB Raspberry Pi 4. It is used here as an accuracy and architecture reference point, not as a deployed component.

SmolVLM-256M (Marafioti et al., 2025) is a 256-million-parameter vision-language model built for exactly this kind of deployment, running in 1.54 GB of RAM through architectural compression and distillation from larger models. It plays the slow-path role in this thesis, running roughly every 5 seconds as a semantic check on the continuous DINOv2 classification.

2.2.5 SAM and Segment Anything

SAM (Kirillov et al., 2023) outlines any object in an image without being told in advance what to look for, trained on over 1 billion masks, and SAM2 (Ravi et al., 2024) extends this to video.

This thesis tested using SAM to split a NAVCAM image into terrain regions first, then classify each region with CLIP, on the idea that classifying a clean region would beat classifying a whole image. In practice it made results worse, reaching 76.7% on a 30-image test against 80.0% for CLIP alone while taking 34 times longer. SAM works best where objects have clear edges, and Mars terrain does not, since soil merges into bedrock and sand ripples into dust with no clean boundary. SAM is used here only to generate annotated images for illustration.

2.2.6 Foundation models applied to Mars and space imagery

Few published results apply large pretrained models directly to planetary rover imagery. The closest is Space-LLaVA (Foutter et al., 2024), which fine-tunes a 7B-parameter LLaVA model on an augmented AI4Mars dataset and improves visual question answering on Mars imagery from as little as 20% of that data, establishing that pretrained checkpoints already carry Mars-relevant visual priors. Holden et al. (2026) use DINOv2 and SAM as frozen feature extractors for cross-view localisation between ground and aerial planetary robots, showing that terrestrial-trained models bridge the domain gap when used as feature extractors rather than end-to-end classifiers. Both evaluate on GPU, and that is the main difference from this thesis. Raspberry Pi 4 CPU-only inference inside a real-time ROS2 pipeline is what motivates unmodified pretrained encoders, the export to ONNX (Open Neural Network Exchange, a portable model format), and a cascade that runs the slow VLM path only every 5 seconds.

The newest development is a shift towards Mars-native pretraining. MOMO (Purohit et al., 2026) is the first foundation model trained at Mars scale, using 12 million orbital images from three instruments, and gains 4 percentage points of macro F1 over the deployed JPL model on geological landform classification. It works on orbital imagery at kilometre scale rather than ground-level rover imagery at metre scale, so it complements rather than overlaps with this thesis. The measurements collected here, including the 65.30-point cross-sensor gap and the specific big-rock failure mode, set a baseline a future Mars-native ground-imagery model would need to beat.

2.3 ROS2 integration and dual-process perception

2.3.1 ROS2 as the rover's middleware

A rover is not one program. It is several: a camera driver, a perception node that classifies terrain, a controller that decides to stop or go, and motor drivers. ROS2 lets these parts talk through named topics without being tightly coupled, so each can be built, tested, and replaced on its own.

This thesis uses ROS2 Humble, the current long-term support release, which includes a Gazebo simulation bridge, navigation tools, and community wrappers for AI models. ROSA (Royce et al., 2023), built at NASA JPL, connects a large language model (LLM) directly to ROS2 topics and services and has been tested on a real rover in the JPL Mars Yard. The pattern it uses, an AI model reading and writing ROS2 topics directly, is the one used by the perception nodes here.

2.3.2 Simulation with Gazebo

Gazebo is the standard physics-based simulator for ROS2 development, providing simulated cameras, LiDAR (light detection and ranging, a laser range sensor), IMU, and physics-based motion. Testing a perception pipeline in simulation first avoids damaging real hardware and gives reproducible conditions. Allan et al. (2019) built a complete lunar rover simulator on Gazebo and ROS, validating the same approach used here.

Simulation comes first in this thesis because the ExoMy rover (Voellmy & Ehrhardt, 2020) was still being assembled during the early experimental phase. Gazebo 11.10.2 is used to build a Mars terrain world of four visually distinct regions, textured from AI4Mars imagery: soil, bedrock, sand, and big rock. The rover is tested at five positions rather than four, because the big-rock region is sampled twice, once in a dense cluster of small rocks and once in a boulder field of a few large ones. Both are hazards, but they look different enough that a classifier can pass one and fail the other, so they are scored as separate zones throughout. One limitation was found during development. `set_entity_state`, the usual ROS2 service for teleporting a simulated robot, does not work in this Gazebo version because of a plugin incompatibility, so robot placement at each zone is done by respawning the model, a manual step documented in Chapter 3.

2.3.3 Dual-process perception

Picture a driver on an unfamiliar road. Most of the time they react to the road automatically, without conscious thought. When something ambiguous appears, a shadow that might be a pothole, they slow down and look carefully. Cognitive science calls these two modes System 1 and System 2 (Kahneman, 2011), and the same split applies to rover perception.

This thesis implements that split as a cascade of two models. A fast classifier (DINOv2 or CLIP, about 150 ms per frame) runs on every camera image and produces a terrain label with a confidence score. When confidence is high, that decision controls the rover directly. When confidence is low, or on a fixed 5-second interval, a slower and more capable VLM (SmolVLM-256M, about 61 seconds per query) is asked for a second opinion in plain language.

The reason for the split is practical, since SmolVLM is far too slow to run on every frame at rover speed. Abaza et al. (2026) find that a VLM is needed for only about 12% of navigation decisions, so the slow model can be called selectively as long as the fast path defaults to a safe stop when uncertain. Lin et al. (2023) show the same pattern for robot task planning, pairing CLIP-based fast classification with LLM-based reasoning for complex instructions.

2.3.4 Formal verification of safety logic

The safety-critical decision layer in this thesis, the confidence-threshold gating that maps a classifier output to SAFE, CAUTION, HAZARD, or STOP (Section 3.9.2), is hand-written ROS2 node code, unit-tested against its own decision functions rather than derived from a formal specification. Perez et al. (2022) describe a different discipline at NASA Ames, in which a safety requirement is written first in structured natural language with clear temporal-logic meaning, then compiled into a runtime monitor (Pike et al., 2010) wrapped as a ROS2 node that publishes a pass or fail verdict against the live topic stream. Their case study runs this on an autonomous ground robot structurally similar to the pipeline built here. It is noted as directly relevant related work and picked up as a concrete extension in Section 6.4.

2.4 Research gaps and thesis positioning

Seven gaps in the literature reviewed above motivate the contributions of this thesis.

2.4.1 Gap 1, edge VLM deployment on unstructured outdoor terrain

Abaza et al. (2026) demonstrate VLM navigation on a Raspberry Pi 5, but for indoor, structured environments. No published work benchmarks a foundation-model cascade on Raspberry Pi-class hardware for outdoor, unstructured Mars-analogue terrain with variable lighting and no GPS, the conditions this thesis targets with ExoMy.

2.4.2 Gap 2, a CLIP and VLM cascade for rover traversability in ROS2

Existing cascades use CLIP for manipulation planning (Lin et al., 2023) or apply a single foundation model to static classification (Foutter et al., 2024). No published work runs a dual-process cascade for rover traversability inside a ROS2 and Gazebo pipeline.

2.4.3 Gap 3, a unified ROS2 foundation-model package

ROS2 wrappers exist for single models, such as ROSA (Royce et al., 2023) for LLM command interpretation, but no published ROS2 package integrates CLIP,

BLIP-2, and SmolVLM as one terrain-perception pipeline with safety watchdogs and confidence-based routing.

2.4.4 Gap 4, few-shot adaptation with no GPU fine-tuning

Space-LLaVA (Foutter et al., 2024) gets strong results through GPU fine-tuning of a 7B-parameter model. For missions where GPU fine-tuning is not an option, how much performance a few-shot linear probe on unmodified pretrained features can recover is still unknown. This thesis measures it directly.

2.4.5 Gap 5, failure-mode characterisation for foundation-model rover decisions

Existing evaluations report accuracy but rarely characterise systematic failure modes, such as the soil-bedrock confusion in CLIP's feature space found here, or safe escalation strategies for low-confidence cases. Understanding these failure modes matters for safety-critical autonomy.

2.4.6 Gap 6, measured CPU contention under concurrent inference

No prior work quantifies the slowdown from running two neural networks concurrently on one CPU, the central constraint of a cascade pipeline. This thesis measures a 2.4x slowdown for CLIP and 2.6x for SmolVLM under contention.

2.4.7 Gap 7, a cross-paradigm comparison on Mars geological texture

No published work compares features from multiple pretraining paradigms on the same Mars benchmark under identical conditions. Space-LLaVA evaluates only one VLM architecture, and Holden et al. (2026) use DINOv2 and SAM without comparing other paradigms. This thesis compares six, each carrying a different built-in assumption about how an image should be represented:

- Contrastive (CLIP, SigLIP2), matching images to captions.
- Self-supervised DINO (DINOv2, DINOv3), matching two crops of the same image.
- Masked image modelling (EVA-02), reconstructing deliberately hidden parts of an image.
- Autoregressive modelling (AIMv2), predicting image patches in sequence.

- Multi-teacher distillation (RADIO), training one model to copy several larger ones at once.
- Position-debiased clustering (Franca), separating texture from where it sits in the frame.

Which of these assumptions transfers best to Mars NAVCAM imagery is still open, and this thesis answers it with a 22-model evaluation on the AI4Mars gold-standard test set.

Table 2.1: Comparison against related work combining foundation models with rover autonomy.

Study	Foundation model used	Hardware	ROS2	Zero-shot	Edge CPU
Space-LLaVA (Foutter et al., 2024)	LLaVA-7B	GPU server	No	No (fine-tuned)	No
SAS on RPi (Abaza et al., 2026)	1.5B VLM	Raspberry Pi 5	No	No	Yes
Planetary Localisation (Holden et al., 2026)	DINOv2, SAM	GPU	No	Yes	No
This thesis	CLIP, DINOv2, SmolVLM-256M, AIMv2-L, Franca (22 models, 6 paradigms)	Raspberry Pi 4	Yes	Yes	Yes

No prior work combines a systematic cross-paradigm comparison of this size, Raspberry Pi 4 CPU-only deployment, and ROS2 integration in one evaluated system. Each property has precedent on its own, but the combination does not. The comparison also surfaces a finding not anticipated at the start of the project and not previously reported in planetary robotics, that DINOv2's self-supervised features are the dominant paradigm for geological texture classification on Mars imagery.

2.5 Summary

Three findings from this review carry into the rest of the thesis. Current operational rover autonomy (Verma et al., 2023; Bajracharya et al., 2008) is

effective but geometry-based, needing mission-specific calibration and GPU training that a low-cost platform such as ExoMy cannot afford. DINOv2's self-supervised features (Oquab et al., 2024) clearly beat CLIP's contrastive features for geological texture, at 93.73% against 87.8% at 1000-shot for the largest variants tested, with the smaller DINOv2+reg ViT-S/14 deployed here reaching 89.9 to 90.24%. And a dual-process cascade (Abaza et al., 2026) is a workable way to combine a fast path with a slow VLM path inside the memory and compute budget of a Raspberry Pi 4 running ROS2.

Together these point to a gap. No existing system unifies zero-shot inference from unmodified pretrained encoders, cross-paradigm model selection, edge CPU deployment, and ROS2 integration for rover terrain classification. Chapter 3 sets out the method used to close it.

3 Methodology

3.1 Overview

This chapter describes how the thesis tests whether pretrained foundation models can classify Mars terrain and support safe rover navigation. The work is organised as ten experiment groups that build up in scope, from a single zero-shot classifier to a full ROS2 autonomy stack:

1. CLIP zero-shot classification, with no labelled Mars data at all
2. CLIP prompt engineering across twelve prompt variants
3. A few-shot linear probe on frozen CLIP features
4. A vision-language model comparison (SmolVLM-256M and BLIP-2)
5. A ROS2 cascade pipeline in Gazebo, combining a fast and a slow model, with a real-camera counterpart (5b) that repeats the terrain test on the rover's own photographs
6. A Raspberry Pi 4 hardware benchmark
7. A cross-paradigm survey of 22 model variants across six pretraining objectives, including the DINOv2 and DINOv3 scaling study
8. A DINOv2 traversability test in Gazebo
9. An autoregressive-paradigm evaluation (AIMv2-L)
10. A position-debiased clustering evaluation (Franca), completing the six-paradigm survey

These ten are the numbered experiments referred to as "Exp 1" to "Exp 10" throughout the thesis. Three further analyses sit alongside them without their own experiment number, because each reuses the features or the model from a group above rather than running a new protocol: reproducibility and calibration checks (seed stability, temperature scaling, attention maps, failure analysis), cross-sensor and cross-planet transfer, and the extended autonomy behaviours of Section 3.12.

This chapter covers the hardware, software, dataset, model choices, and evaluation protocol for all of them.

3.2 Hardware platform

3.2.1 Development machine

Phase 1 and 2 experiments ran on a Windows Subsystem for Linux 2 (WSL2) machine running Ubuntu 22.04, with an Intel CPU, 7.5 GB of available RAM, and an NVIDIA MX330 GPU with 2 GB VRAM. The MX330 has no CUDA-compatible cuDNN support, so every model ran on CPU. This matches the compute conditions of edge rover hardware and keeps reported inference times representative of CPU-only deployment.

Three constraints shaped the experiment design. The first was quantisation, which stores a network's weights at lower numerical precision so the model takes less memory and runs faster. The `bitsandbytes` library used for INT4 and INT8 quantisation needs a CUDA GPU and was not available, so all development-machine models ran in float16 or float32. BLIP-2 (2.7B parameters) in float16 needs about 6.5 GB of RAM, which triggers memory swap on this machine and raises its inference time from about 60 to 90 seconds per image. SmolVLM-256M needs only about 1.54 GB and runs comfortably within budget.

3.2.2 Target platform: ExoMy

The deployment target is ExoMy, an open-source six-wheel rover built by the European Space Agency for education and research (Voellmy & Ehrhardt, 2020). ExoMy was chosen for being low-cost, fully reproducible, and mechanically representative of a small planetary rover.

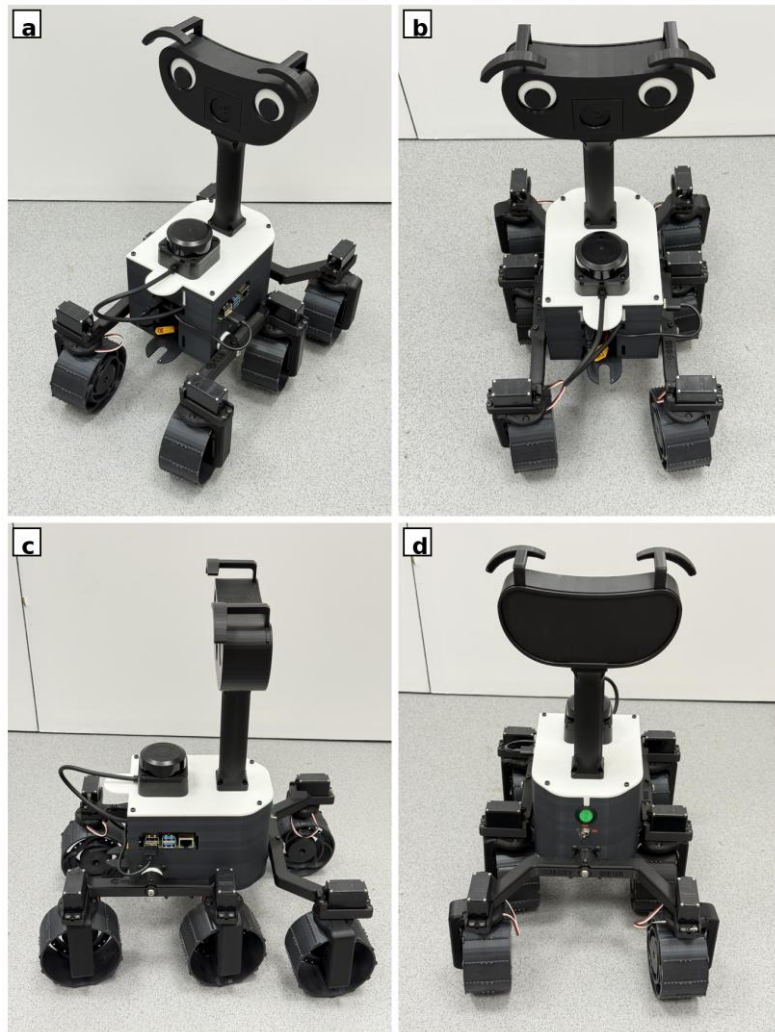


Figure 3.1: The assembled ExoMy rover used in this thesis. (a) three-quarter view, (b) front view showing the camera mast, (c) side view showing the LiDAR and the Raspberry Pi's ports, (d) rear view.

Table 3.1: Raspberry Pi hardware platform specification.

Component	Specification
Onboard computer	Raspberry Pi 4 Model B, 8 GB RAM
Motor controller	Adafruit PCA9685 16-channel PWM HAT
Camera	USB camera, front-mounted
Drive configuration	6-wheel rocker-bogie
Communication	Wi-Fi for development, onboard autonomous in final deployment

The camera, on both the real rover and its Gazebo counterpart, has an 80-degree horizontal field of view, 640x480 resolution, runs at 10 Hz, and publishes to `/exomy/camera/image_raw`. These parameters are matched between simulation and hardware so the same pipeline configuration works on both. The wide 80-degree view is meant to catch terrain transitions one to two metres ahead at the thesis's nominal driving speeds of 0.05 to 0.10 m/s.

An 8 GB Raspberry Pi 4 was chosen specifically to run a VLM without external compute, following Abaza et al. (2026), who show a 1.5B-parameter VLM running on a Raspberry Pi 5 with no GPU for periodic terrain assessment. The completed Raspberry Pi benchmark (Section 4.11) shows DINOv2 ViT-S/14 exported to ONNX running at 570 ms per image (1.75 frames per second) using 258 MB of RAM.

3.3 Software environment

Table 3.2: Software environment and library versions.

Category	Tool and version
Robot middleware	ROS2 Humble Hawksbill (LTS, Ubuntu 22.04)
Simulation	Gazebo Classic 11.10.2
Language	Python 3.10
Deep learning framework	PyTorch 2.5.1
Vision-language models	HuggingFace Transformers 4.x
Image processing	OpenCV 4.x, Pillow
Machine learning utilities	scikit-learn
Visualisation	Matplotlib
Experiment tracking	Weights and Biases
ONNX runtime	ONNX Runtime (CPU)

ROS2 Humble was chosen for its long-term support and Ubuntu 22.04 compatibility. Gazebo Classic 11.10.2 was used because it ships with the ROS2 Humble stack this project builds on. Gazebo Classic is now a legacy simulator,

so this thesis treats it as a controlled integration benchmark rather than a long-term high-fidelity platform. Moving to the newer Gazebo stack is left as future work. All ROS2 packages were built with `colcon`.

3.4 Dataset: AI4Mars

All terrain classification experiments use the AI4Mars dataset (Swan et al., 2021), built by NASA JPL from Curiosity's Navigation Camera (NAVCAM) imagery and released through the NASA Planetary Data System.

Table 3.3: AI4Mars dataset properties.

Property	Value
Total images	18,127 NAVCAM JPEG images
Training labels	16,064 images, crowd-sourced
Test labels	322 images, gold-standard, 3+ annotators at 100% agreement
Label format	Pixel-level segmentation masks
Classes	Soil, Bedrock, Sand, Big Rock
Unlabelled pixel value	255, excluded from evaluation

The gold-standard 322-image test set is used for every quantitative result in this thesis. Each pixel in a test image carries one of the four class labels or is unlabelled. To get a single image-level label, the dominant non-unlabelled class in each image is taken as ground truth. Of the 322 images, 287 had usable image files in the local copy of the dataset, and all reported accuracy figures use this 287-image set.

The 287 images split as 123 soil (42.9%), 92 bedrock (32.1%), and 72 sand (25.1%). Big rock never appears as the dominant class in any test image, a known property of this test set, so it is excluded from per-class accuracy reporting even though big rock pixels appear inside many images.

The supervised baseline throughout this thesis is a DeepLabv3+ model with a ResNet-101 backbone, trained by Swan et al. (2021) on the full AI4Mars training set. It reaches 96.67% overall accuracy (soil 99.1%, bedrock 94.9%, sand 93.5%). Every accuracy result in this thesis is measured against this ceiling.

3.5 Foundation model selection

Six model families, 22 variants total, were evaluated. Selection criteria were public availability as a pretrained checkpoint, feasibility on Raspberry Pi-class hardware, and coverage of distinct pretraining paradigms: contrastive, DINO self-distillation, masked image modelling, multi-teacher distillation, autoregressive, and position-debiased clustering.

Table 3.4: Vision-language models evaluated and their Raspberry Pi feasibility.

Model	Architecture	Parameters	Primary task	Raspberry Pi feasible
CLIP ViT-B/32	ViT + text transformer	~338M	Zero-shot classification	Yes, ~1-2 FPS estimated
SmolVLM-256M-Instruct	Idetics3 + SmoLM2	~256M	Visual question answering	Yes, ~0.5 FPS estimated
BLIP-2 (blip2-opt-2.7b)	Q-Former + OPT-2.7B	~2.7B	VQA, captioning	Borderline, 6.5 GB RAM
SAM2 (sam2-hiera-tiny)	Hierarchical encoder	~38M	Segmentation	Yes, visualisation only

Phi-3-V (7.9B parameters) was considered but dropped, since its memory footprint (7.9 GB) exceeds the development machine's 7.5 GB, and BLIP-2 and SmolVLM already give a sufficient comparison.

3.5.1 Choosing DINOv2+reg ViT-S/14 as the deployed model

Beyond paradigm coverage, a model deployed on a rover has to satisfy several compute-aware constraints that foundation-model papers rarely spell out. DINOv2+reg ViT-S/14 (22M parameters, 258 MB RAM on the Raspberry Pi 4) sits within the 15-40M edge-optimal parameter range identified by Samson (2026), runs at 570 ms per image, and reached the strongest hazard recall and best calibration (ECE 0.0325 at an optimal temperature of 0.461) of the full 22-model cohort. Its class-probability-plus-threshold decision is fully auditable, and DINOv2's shape bias, rather than texture bias (Park et al., 2023), should transfer better across the sizeable domain gap between ImageNet and Mars terrain. Docherty et al. (2025) independently applied the same DINOv2+reg ViT-S/14 configuration to dense terrain segmentation on AI4Mars and reached 0.797 mean intersection over union (mIoU), a standard segmentation overlap score, with a

linear head, more than double a classical baseline's 0.404 mIoU, external evidence that these frozen features generalise beyond image-level classification.

The choice to freeze the encoder and adapt only a lightweight linear probe, rather than fine-tune, follows the theory of linear probing versus fine-tuning set out by Kumar et al. (2022). Their Feature Distortion Theorem shows that fine-tuning can distort the feature directions that support generalisation to data unlike the training set, even as accuracy on data like it improves, and they further show that the gap between the two approaches shrinks as the quality of the pretrained features rises. Since Mars terrain sits far outside DINOv2's ImageNet pretraining domain, fine-tuning on AI4Mars would likely raise AI4Mars accuracy at the cost of the broad visual priors that let the same frozen features work in Gazebo, on Mars-Bench MastCam imagery, and on future sensors. The consistent 90%+ accuracy of frozen DINOv2 features across these different settings supports that prediction empirically.

3.5.2 The four models in the cascade and comparison set

CLIP ViT-B/32 (Radford et al., 2021) maps images and text into a shared embedding space through contrastive pretraining on 400 million pairs. It classifies an image by computing cosine similarity against a text description of each class and picking the highest match. CLIP is the cascade's fast path because it needs no GPU, runs at 96-136 ms per image on CPU, and gives a well-studied zero-shot baseline.

SmolVLM-256M-Instruct (Marafioti et al., 2025) pairs an LlaVA3 vision encoder with a SmolLM2 language backbone, built for edge deployment. It became the slow-path VLM for its small memory footprint (about 1.54 GB) and practical inference time (55-61 seconds standalone). The thesis supervisor recommended it after Meeting 4 (June 2026) as a VQA-capable model deployable on Raspberry Pi-class hardware.

BLIP-2 (Li et al., 2023) connects a frozen ViT-L image encoder and a frozen OPT-2.7B language model through a lightweight Q-Former. It runs in float16 on CPU, needs about 6.5 GB RAM, and takes about 753 seconds per image under

memory swap. Because of this speed, only a 20-image preliminary evaluation was run, and its results are reported as preliminary throughout.

SAM2 (Meta AI, 2024) was tested as a pre-processing step, segmenting terrain regions before classifying each with CLIP. As Section 4.5 reports, this made accuracy worse (76.7% against 80.0% for CLIP alone on a 30-image sample) while adding a 34x slowdown, so SAM2 was moved to visualisation only.

3.6 Evaluation framework

Table 3.5: Summary of the ten experiment groups. Experiment 5b is the real-camera counterpart of Experiment 5 rather than a group of its own.

Experiment	Description	Dataset	Models
Exp 1	CLIP zero-shot classification	AI4Mars, 287 images	CLIP ViT-B/32
Exp 2	CLIP prompt engineering, v1-v12	AI4Mars, 287 images	CLIP ViT-B/32
Exp 3	Few-shot linear probe	AI4Mars, 287 test / 15,901 train	CLIP + logistic regression
Exp 4	VLM comparison	AI4Mars, 287 images	BLIP-2, SmolVLM-256M
Exp 5	ROS2 cascade in Gazebo	Gazebo, 5 zones	CLIP + SmolVLM
Exp 5b	Real-camera sim-to-real test	72 real photographs, 3 classes	DINOv2+reg ViT-S/14 + logistic regression
Exp 6	Raspberry Pi benchmark (complete)	20 images, RPi 4 aarch64	DINOv2+reg ViT-S, ONNX
Exp 7	Model survey, 22 models, 5 rounds	AI4Mars, 287 test	DINOv2/v3 family, SigLIP2, RADIO, EVA-02, SAM2, Depth Anything V2, Ensemble B
Exp 8	DINOv2 traversability in Gazebo	Gazebo, 5-zone world	DINOv2+reg ViT-S/14 + logistic regression
Exp 9	AIMv2-L autoregressive evaluation	AI4Mars, 287 test	AIMv2-Large
Exp 10	Franca ViT-L/14 with RASA	AI4Mars, 287 test	Franca-CLS, Franca-RASA, Franca-CLS+RASA

Accuracy is reported at two levels: overall accuracy across all classes, and per-class accuracy, which matters because the test set is unbalanced and a model that always predicts "soil" would score about 43% while understanding nothing about terrain. Speed and memory are reported as average inference time, FPS, peak RAM, and model load time, all measured on the development machine unless stated otherwise. Hardware-to-hardware comparisons (development PC against Raspberry Pi) are in Section 4.11.

3.6.1 How many labels are enough: the 1000-shot protocol

Every DINOv2-based experiment in this thesis uses the same 1,000-shot training protocol: 1,000 labelled training images per class, except big rock, which is capped at its full 108 available images. This choice was tested rather than assumed. A label-efficiency sweep trained a frozen linear probe (logistic regression with the regularisation strength C set to 0.316, averaged over 5 random seeds) at increasing label counts, first in a balanced strategy up to $N=108$ per class (bounded by big rock's data availability), then in a practical strategy holding big rock at 108 while raising soil, bedrock, and sand up to 1,000.

Table 3.6: Label-efficiency sweep results by N per class and labelling strategy.

N per class	Strategy	Mean accuracy	Std
1	Balanced	40.1%	7.1
10	Balanced	60.8%	8.5
50	Balanced	74.8%	1.5
108 (big rock ceiling)	Balanced	78.3%	2.1
200	Practical	84.7%	1.0
500	Practical	87.8%	0.9
1000 (deployed)	Practical	90.2%	0.0

(The deployed 1000-shot figure reported elsewhere in this thesis, 89.90%, comes from the original experiment script at a single fixed seed of 42, not this five-seed sweep.)

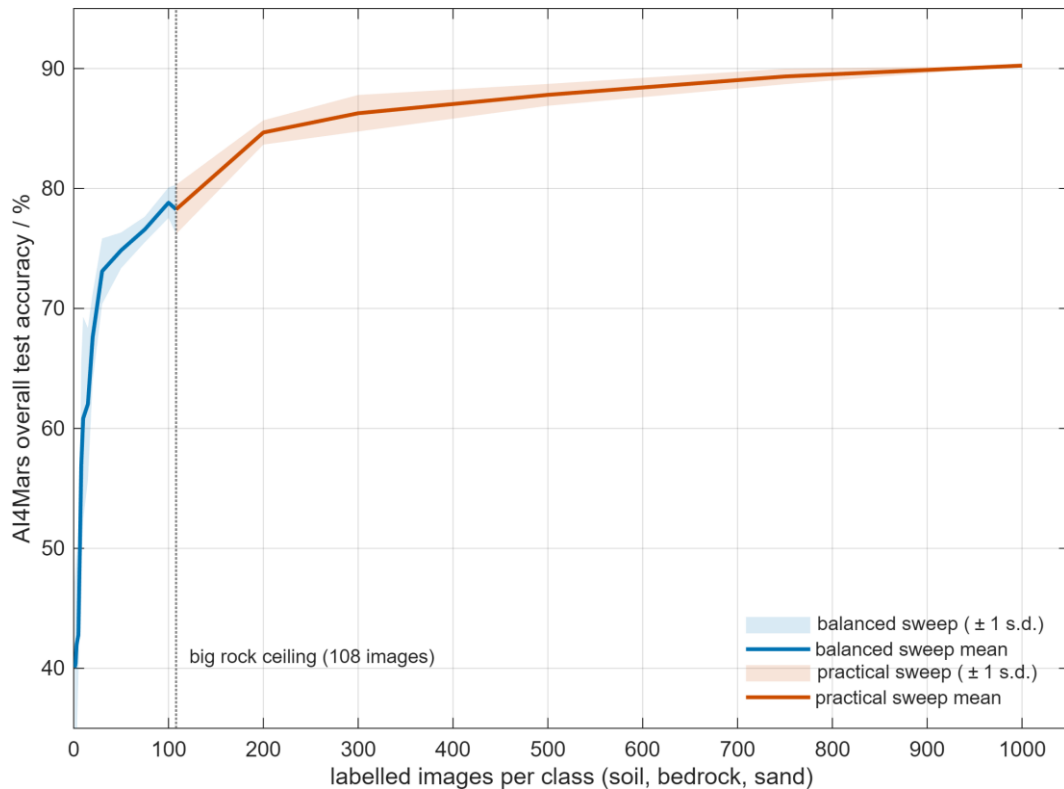


Figure 3.2: Label efficiency of DINOv2+reg ViT-S/14 under the balanced and practical labelling strategies.

Three findings justify the protocol.

1. Even 10 labels per class already reaches 60.8% accuracy, more than double the 25% random baseline, showing that DINOv2's ImageNet pretraining carries real Mars-relevant discriminability. A mission team could therefore reach usable accuracy from a single short annotation session rather than a labelling campaign.
2. Big rock's 108-image ceiling, not the label count for the other three classes, is the binding constraint. The balanced sweep plateaus at 78.3% while the practical sweep keeps climbing to 90.2% once soil, bedrock, and sand get more labels.
3. The curve is close to flat by 750 labels per class (89.3%, within 0.9 points of the 1,000-label result), so labels beyond 1,000 buy little.

Figure 3.2 plots the practical sweep only from the big-rock ceiling onward, where the two strategies meet. Below it the practical strategy holds big rock at 108 while

the other classes get fewer, so the rarest class dominates the training set and accuracy falls to 42.0% at 50 labels per class, below the balanced sweep's 74.8% at the same count. That is a useful warning about over-representing a rare class, but it is not a labelling budget anyone would choose, so it is left off the figure.

3.7 Stage 1: CLIP zero-shot and prompt engineering

Zero-shot classification uses no labelled images. For each input image, CLIP's vision encoder produces an image embedding, its text encoder produces one embedding per class prompt, cosine similarity is computed between the image and every class prompt, and a softmax over these similarities picks the predicted class. Ensemble prompts (multiple descriptions per class) are averaged before the softmax. This is the standard CLIP zero-shot protocol (Radford et al., 2021).

Twelve prompt variants (v1-v12) were tested, starting from a short baseline description, then adding continuity and geological cues, texture and scale contrast, a NAVCAM domain prefix, and multi-prompt ensembles. The best (v9) combines a NAVCAM-prefixed soil prompt with a texture-and-scale bedrock prompt, addressing a soil-bedrock confusion found in the earlier variants. Since prompt selection used the same 287-image test set with no held-out validation split, this is an upper-bound result and may show mild overfitting to the test set, a limitation acknowledged here.

The few-shot probe uses CLIP's frozen visual encoder as a feature extractor, producing a 512-dimensional feature per image, a logistic regression classifier ($C=0.316$, the regularisation strength used by the standard CLIP linear-probe protocol, where a smaller C forces a simpler classifier and guards against overfitting) trained at N in $\{10, 50, 100, 500, 1000\}$ labelled examples per class, drawn class-stratified from the 16,064-image training set, with no gradient ever passed back into CLIP. Big rock (108 training images) is excluded from few-shot training for having too few examples.

3.8 Stage 2: vision-language model evaluation

Both VLMs are evaluated with the same VQA question: "What type of terrain is visible in this Mars image? Answer with exactly one word from this list: soil,

bedrock, sand, rock." This constrains the model to a single parseable word, includes "Mars" as a domain cue, and matches the AI4Mars class vocabulary. A response is scored as a parse failure if it contains none of the target words, and the parse failure rate is reported separately as a limitation of VLM-based classification.

An ablation prompt dropping the word "Mars" tested whether that word was responsible for a parsing problem found during the full SmolVLM run: 62 of 287 images (21.6%) came back as the single word "Mars." instead of a class. On a 20-image subset, removing "Mars" eliminated parse failures entirely, but also dropped soil accuracy from 67.3% to 0% and overall accuracy from 49.78% to 15.0%. The word "Mars" appears to activate domain-relevant knowledge the model needs to classify soil correctly, so the prompt keeping it was used for all reported results, and the 21.6% parse failure rate is reported as a known limitation, discussed further in Chapter 5.

3.9 Stage 3: the ROS2 cascade pipeline

3.9.1 Design

The cascade follows a dual-process pattern: a fast, approximate model (CLIP) runs on every camera frame, and a slower, more capable model (SmolVLM) runs periodically or when the fast model is uncertain. Three reasons motivate this. SmolVLM takes 55-141 seconds per inference on the development machine, too slow to run on every frame. Terrain assessment has to be continuous for safety, so the slow VLM cannot be the only source of traversability decisions. And the two models are complementary. CLIP gives fast visual matching, SmolVLM gives language reasoning that can disambiguate visually similar terrain, such as rock-textured bedrock against isolated boulders.

This matches Abaza et al. (2026), who report a deterministic fast path handling about 88% of decisions with a VLM handling the remaining 12%, and Dominguez-Vidal (2026), whose three-mode ROS2 wrapper for Florence-2 (continuous, on-demand, and asynchronous) reaches 9.75 FPS for language-guided tasks on a

mobile GPU, with the service and action modes corresponding to this thesis's on-demand slow path.

The pipeline is a single ROS2 node, `cascade_pipeline.py`, with two threads. The CLIP fast path runs synchronously on every frame, extracts features, computes cosine similarity, publishes a terrain label with confidence to `/terrain_classification`, and triggers SmoIVLM immediately if confidence drops below 0.40. The SmoIVLM slow path runs every 5 seconds or on that trigger, in an async thread, loads the image, runs the VQA query, parses the response, and publishes a terrain label, a scene description, and a traversability decision that overrides CLIP's.

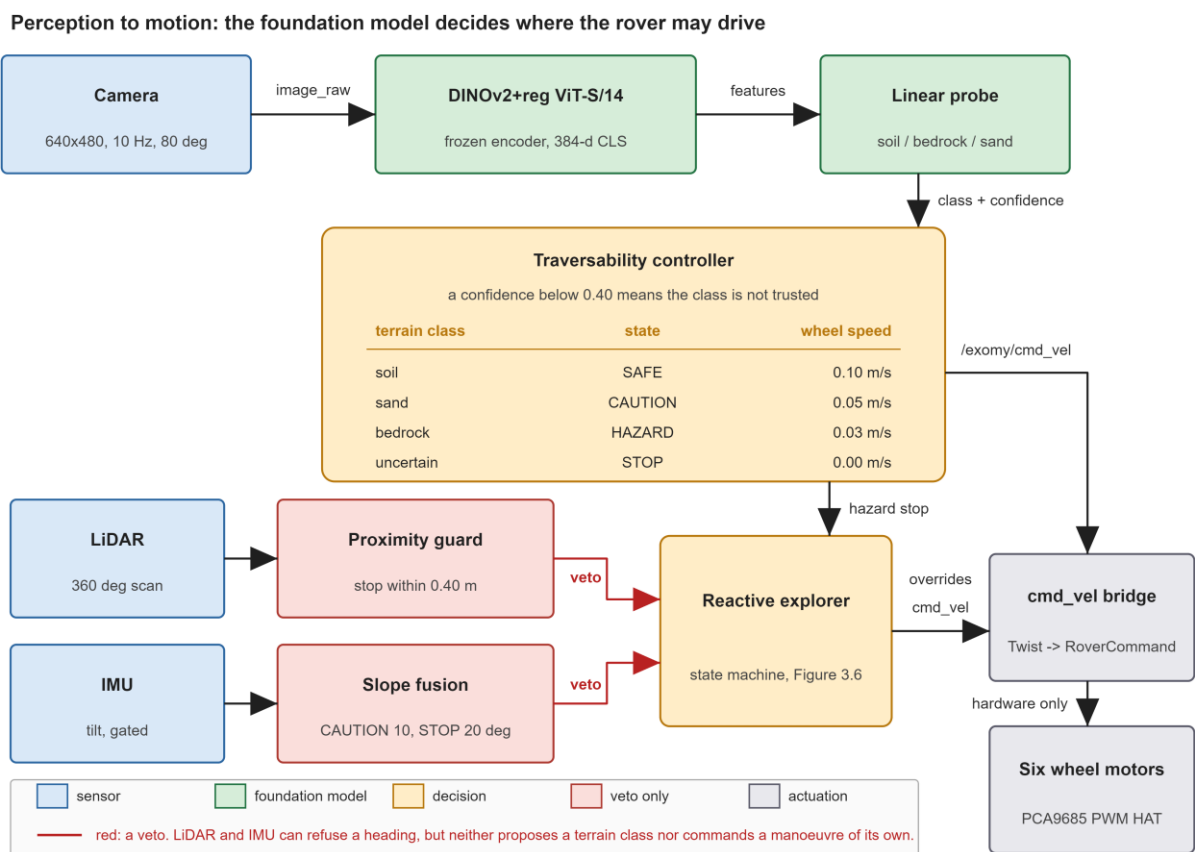


Figure 3.3: The perception-to-motion pipeline. LiDAR and IMU enter only as vetoes; neither proposes a terrain class. The four-tier speed policy shown is a Gazebo result, for the reason given in Section 3.11.3.

Traversability rules map terrain to a policy: soil is safe, sand is safe but monitored for wheel sinkage, bedrock is caution with reduced speed, and big rock is the

highest-risk label this node publishes (the separate DINOv2 Gazebo controller, Section 3.11.2, maps it onward to a full stop). This mapping follows SPOC (Rothrock et al., 2016), which establishes terrain type as a direct input to slip risk, wheel wear, and path planning on Mars rover missions.

3.9.2 Safety mechanisms

Three mechanisms protect the pipeline. A five-second watchdog publishes "hazard" and logs a warning if no camera image arrives in that window, so a silent perception failure cannot let the rover drive blind. A confidence threshold triggers SmolVLM immediately, outside its regular five-second cycle, whenever CLIP's top similarity score drops below 0.40. And a `threading.Lock` prevents CLIP's synchronous callback thread and SmolVLM's async thread from racing on shared traversability state.

One inconsistency is worth recording. `terrain_controller_node.py`'s own code default for its confidence threshold is 0.60 while `dinov2_terrain_node.py`'s is 0.40, the temperature-scaling-calibrated value. Every launch file used in this thesis overrides both to 0.40, so every reported result used the calibrated value, and the divergence matters only to someone reading either node's source in isolation.

The underlying philosophy, that an unnecessary stop always beats entering hazardous terrain, follows established rover practice. Verma et al. (2023) describe Perseverance's keep-out regions and its slope, roughness, and deep-sand checks, all built on the same principle, and Helmick et al. (2009) advocate the same lightweight reactive design of reducing speed or stopping fully under classification uncertainty. The division of authority matters as much as the policy: DINOv2 proposes a label and a confidence, but the fixed rule sending anything under 0.40 to STOP holds the final word, so a confident wrong answer cannot override a threshold. Cheng et al. (2025) report that a graded, speed-tiered policy cuts unnecessary stops by 34% against a binary safe-or-stop rule, which is the case for the four tiers used here.

Every threshold and state-machine rule in this layer is hand-implemented and unit-tested against its own pure decision functions rather than generated from a

formal specification. Section 2.3.4 sets out the alternative and Section 6.4 picks it up as future work.

CLIP's visual encoder was never exported to ONNX in this thesis, unlike DINOv2 (Section 4.11). Its measured PyTorch CPU latency is 295 ms per image, which is why CLIP stays the cascade's confirmatory fast path while DINOv2 is the model actually deployed on the Raspberry Pi. An ONNX export for CLIP is left as future work.

3.9.3 Package structure

The `fm_perception` ROS2 package holds all perception nodes: `clip_terrain_node.py` (subscribes to the camera, publishes terrain classification and a visualisation), `smolvlm_terrain_node.py` (subscribes to the camera and CLIP's output, publishes its own terrain label, scene description, and traversability decision), `blip2_scene_node.py` (the same pattern for BLIP-2), and `cascade_pipeline.py` (the combined node above).

3.10 Deploying on the real rover

ExoMy ships with ROS1 software (Voellmy & Ehrhardt, 2020), so the control nodes were ported to ROS2 before anything could be deployed. The port is software engineering rather than a test of a foundation model, and Appendix A documents it. Two of its defects matter to the results here. Commanded speed did not scale with the request, so the controller's soil, sand, and bedrock differentiation would have collapsed to on-or-off on the real rover; and a steering-angle function returned an integer into a float field, so the motor-command node had never successfully published a message anywhere in the repository's history. Both went unnoticed because every Gazebo result drives the simulated plugin directly and bypasses that node entirely. Section 5.7 returns to what that says about where simulation testing stops being informative.

3.11 Gazebo simulation environment

3.11.1 World design

A custom Gazebo world, `mars_terrain.world`, simulates Mars terrain for testing the perception and traversability pipeline. It is a deliberately controlled benchmark rather than a photorealistic Mars terrain. Mission-grade simulators go much further, with 8K terrain and fault injection in NASA's VIPER simulator (Fong et al., 2024) and terramechanics in MarsSim (Zhou et al., 2022), and Allan et al. (2019) catalogue what Gazebo itself can reach, but every one of those features costs engineering effort beyond this thesis's scope. Starting simplified is established practice: the DLR lunar rover stack (Schuster et al., 2019), the ESA MaRTA guidance study (De Luca et al., 2025), and Cheng et al. (2025) all validate in flat-zone or simplified worlds first. The objective here is to measure how well frozen features classify terrain and how well a conservative controller compensates when they are wrong, and neither needs photorealism or wheel-soil physics to test.

Table 3.7: Comparison of planetary rover simulators.

Platform	Base	Terrain fidelity	Target body	ROS2
RSIM / VIPER (Fong et al., 2024)	Gazebo + ROS2	8K DTM, full CAD dynamics, fault injection	Moon	Yes
MarsSim (Zhou et al., 2022)	Gazebo + ROS	Terramechanics plugin, multiscale planetary data	Mars	No (ROS1)
GazeboMarsRover (open-source)	Gazebo Classic	HiRISE DTM mesh, LiDAR, IMU	Mars	No
LunarSim (Pieczyński et al., 2023)	Unity + ROS2	High visual fidelity, slip, lunar gravity	Moon	Yes
Space Robotics Bench (Orsula et al., 2025)	NVIDIA Isaac Sim	Procedural, ray-traced, massively parallel	Space	Yes
This thesis	Gazebo Classic 11 + ROS2	Controlled flat zones, AI4Mars NAVCAM textures	Mars	Yes

The world uses four 15x12 m AI4Mars-textured zones in a quadrant layout across a 30x24 m base, with the rock zone tested at two spawn positions 4 m apart, both hazard. Figure 3.4 shows the layout. Textures are kept grayscale-neutral, since

a stronger Mars-colour tint was tested and rejected for changing DINOv2's decisions. The rock field mixes large mesh boulders for visual context with textured sphere primitives for a dense hazard cluster; an earlier fully mesh-based field was rejected because it made the rock zones read as sand, breaking the intended STOP behaviour. Scattering density loosely follows the inverse size-frequency distribution used in OAISYS (Muller et al., 2021), DLR's Blender-based planetary terrain pipeline.

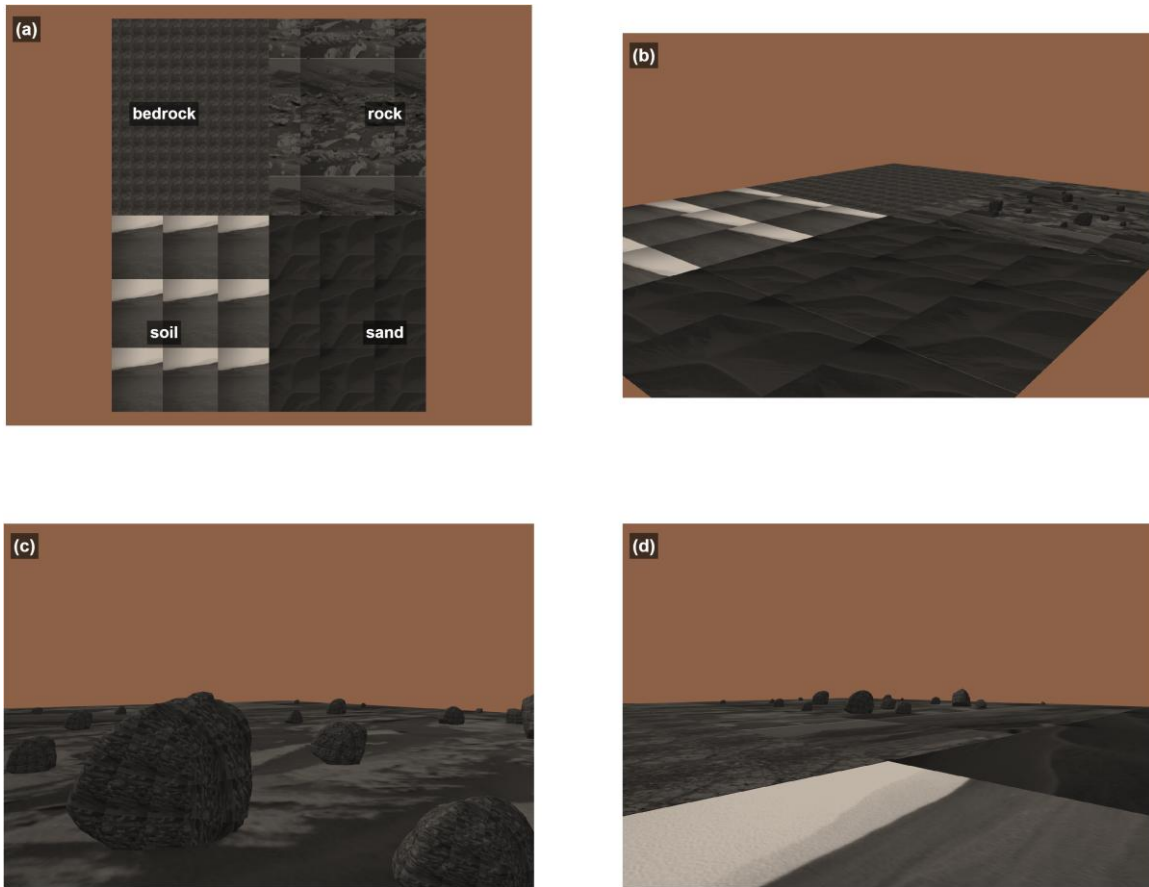


Figure 3.4: The Gazebo benchmark world. (a) plan view of the four 15x12 m terrain quadrants on the 30x24 m base, (b) oblique view from the sand corner, (c) the rock zone at rover camera height, (d) a zone boundary at rover camera height.

3.11.2 Traversability protocol

The robot is spawned at each of the five zone positions in turn, with DINOv2 classifications recorded repeatedly at each. Figure 3.5 shows what the rover's

camera sees at every one. Four terrain types are covered, not five: the rock terrain is sampled twice, at two spawn points 4 m apart, which is why views (d) and (e) resemble each other. The rocks themselves are sphere primitives rather than photorealistic meshes, for a reason Section 4.10 reports as a result in its own right.

The controller maps terrain to speed: soil to SAFE at 0.10 m/s, sand to CAUTION at 0.05 m/s, bedrock to HAZARD at 0.03 m/s, and big rock or uncertain terrain to STOP. Terrain accuracy and safety are scored separately, giving 3 of 5 on terrain and 5 of 5 on safety. Both rock positions reach STOP through the uncertain-to-STOP path, since confidence there falls below the 0.40 threshold whatever label is predicted, and the two misclassified non-rock zones still receive a conservative non-zero speed, so no safety failure occurs. Full per-zone results are in Section 4.10. Gazebo 11.10.2's `libgazebo_ros_state.so` teleport service proved unreliable, so the experiment deletes and respawns the rover at each position instead.

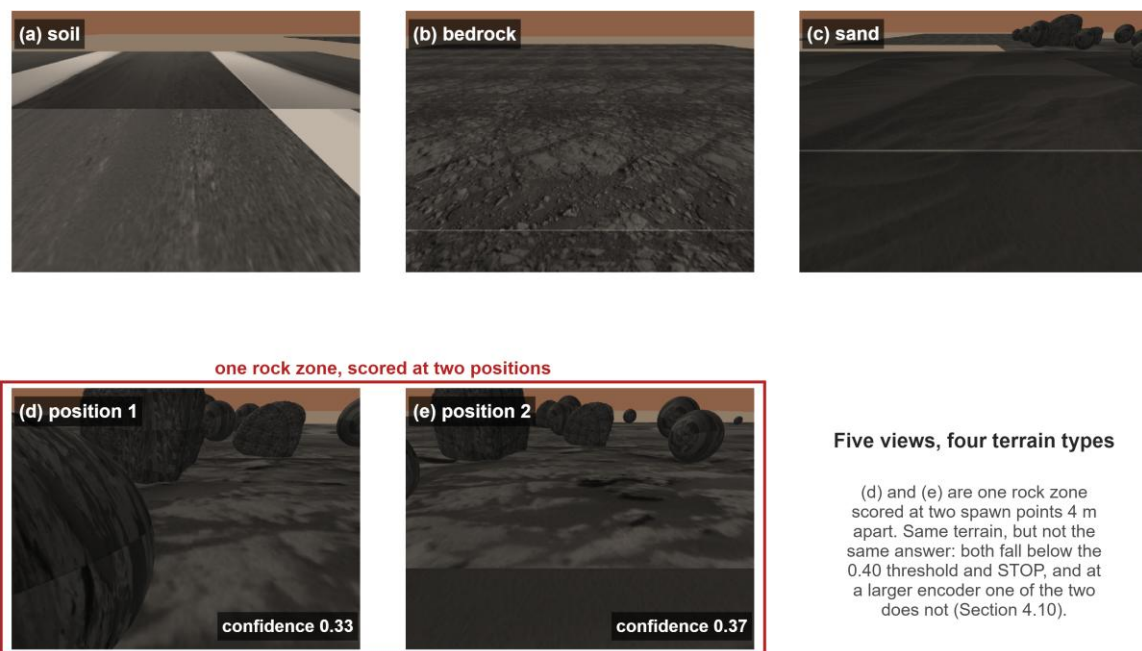


Figure 3.5: What the rover's camera sees at each of the five zone positions, which is the input DINOv2 classifies.

3.11.3 Three known simulation-fidelity gaps

The four-tier speed policy is a third gap, and the most consequential of the three for what the perception result means in practice. In Gazebo the policy does what Figure 3.3 describes: a soil reading drives at 0.10 m/s, sand at 0.05, bedrock at 0.03, and big rock or an uncertain reading stops. On the real rover it collapses. Wheel speed was measured directly on a 3 m flat course, driving for a script-controlled duration and tape-measuring the distance covered, since ExoMy has no wheel encoders to measure it from. Commanded values of 20, 25, and 50 on the driver's 0 to 100 scale all produced the same 0.235 to 0.245 m/s. The servos saturate well below the top of the commanded range, so the three non-zero tiers are not physically distinguishable on this hardware.

What survives the transfer is the distinction that matters for safety, between moving and stopping, and that one is intact: a STOP still stops. What does not survive is the graded caution the policy is designed around. Every quantitative speed claim in this thesis is therefore a Gazebo result, and the real rover's contribution is the classification and the stop decision, not the speed that follows from it. Section 5.6 returns to this.

The real Raspberry Pi Camera V2 has barrel distortion, with first and second radial distortion coefficients of about -0.33 and 0.10, while the Gazebo camera renders an ideal pinhole projection. DINOv2 processes a centre-cropped 224x224 window, so most of the distorted periphery is cropped away and the five-zone experiment is not strongly affected, but the gap widens the sim-to-real shift measured in the real-camera experiment (Experiment 5b, Section 3.6) and is noted as future simulation work.

Until 17 July 2026 the Gazebo differential-drive plugin drove only the front two wheels, leaving the other four passive, while real ExoMy drives all six. This was not a deliberate simplification but a defect, found through the in-place-rotation problem investigated in Section 4.13, where the passive wheels' friction was anchoring the vehicle against rotation. The fix and its results are given there. Perception results reported before that date are unaffected, since none of them

depend on wheel actuation, but commanded-speed and turn-timing figures from before it reflect the earlier physics.

3.12 Extending the reactive controller into an autonomy stack

The traversability controller reduces speed or stops on hazard, but a stop is terminal, since the rover has no way to try a different heading. Ten behaviours extend the reactive layer to recover from that state and make limited autonomous progress. All stay within Layer 2 and Layer 3 of the six-layer stack, with no localisation, mapping, or path planning beyond a bounded local search, and every one was scoped in a structured design interview before any code was written, then built test-first as pure Python decision functions covered by unit tests before being wrapped in a ROS2 node. Table 3.8 lists them, with the number of test cases covering each. Results for all of them are in Section 4.13, except the cross-planet probe, which is in Section 4.9. The four carrying a methodological commitment worth stating in full follow.

Table 3.8: Behaviours extending the reactive controller, and how each was verified.

Behaviour	What it adds	Test cases
Reactive exploration	Recovers from a terminal hazard stop by turning, then failing safe	147
Stuck-in-sand detection	Detects wheels spinning without displacement, then escalates	80
IMU slope-triggered retreat	Backs straight out of an over-tilt without rotating	Withdrawn
Multi-modal fusion	Combines LiDAR range, IMU tilt, and terrain class into one score	15
Path planner (L5-lite)	8-connected A* with a pure-pursuit follower, replanning every 2.5 s	52
Waypoint sequencing (L6-lite)	Advances to a second goal with no operator command between legs	4
Frontier exploration	Repeatedly selects the nearest unexplored frontier of its costmap	26
Explore-then-return-home	Exhausts a bounded box, then returns to its own first recorded pose	10
Uncertainty-driven re-observation	Revisits the lowest-confidence cell once frontiers run out	13
Cross-planet probe	Tests whether frozen features transfer between Mars and Earth	offline experiment

The test-case column counts what pytest collects for each behaviour's own decision functions, so the number means the same thing on every row. It is there because these are the safety-critical parts of the stack and most of them were exercised on real hardware only a handful of times; the test suite is the evidence that the logic behaves as described under conditions the hardware sessions never reached. Reactive exploration takes `test_reactive_explorer.py` together with `test_avoidance_planner.py`, which covers the heading-choice logic the explorer calls; stuck-in-sand detection takes both its simulation and hardware test files; and the path planner takes the A* tests plus the pure-pursuit, lookahead, goal-tolerance, bootstrap-crawl, and hybrid-rotation cases in `test_path_follower.py`. The remaining cases in that file and in `test_frontier_explorer.py` divide between waypoint sequencing, return-home, re-

observation, and frontier exploration according to the function each one exercises.

These 347 cases are part of the 652 the three ROS2 packages collect in total, the rest of which cover the perception, controller, kinematics, and driver code the behaviours sit on top of rather than any one behaviour. The retreat row carries no count because its tests were deleted with the code, for the reason given below, and the cross-planet probe is an offline experiment rather than a ROS2 node.

Reactive exploration. This behaviour sets the pattern the rest follow. `reactive_explorer_node` watches two hazard signals, the controller's own hazard stop and a LiDAR proximity stop, and takes over `/exomy/cmd_vel` once either persists for 3 seconds. It then turns 90 degrees and checks, tries the opposite 90 degrees if blocked, and retreats 1.0 m straight back if both sides are blocked, up to 3 retreat cycles before a terminal FAILSAFE.

Three design choices matter. Every turn closes the loop against odometry-derived yaw rather than running for a fixed time, because wheel slip on loose ground would send a timed turn off target. A heading is accepted only after two consecutive safe readings while a single blocked reading rejects it, the same stop-immediately, resume-gradually asymmetry used everywhere else in the safety stack. And the retreat reverses a fixed distance rather than retracing the incoming path, which would need a position-history buffer and so edge into the mapping capability this thesis scopes out. This is the version behind the Gazebo results in Section 4.13. Its reverse leg was removed later in the project for the reason given below, and Section 5.10 states how the simulation and hardware phases relate. Figure 3.6 shows the state machine that remains.

Table 3.9 names what the rover is doing in each of those states, since a state machine drawn as boxes and arrows says what the transitions are but not what happens inside a box.

Table 3.9: The reactive explorer's states and what the rover does in each.

State	What the rover does	How it leaves
STARTUP_CHECK	Holds still until the sensors settle, then verifies odometry and IMU are live and picks a heading from a full 360-degree LiDAR scan	To the sweep on the first pass, otherwise straight to driving or to a turn
STARTUP_SWEEP	Turns to eight stops in turn and scores the terrain at each, so the opening heading is ranked rather than assumed. Runs once, and takes about 85 s	When the sweep finishes and a winning heading is known
MONITORING	Drives. Cedes speed control to the traversability controller for as long as the corridor stays clear	When a hazard signal persists for 3 s
TURN_TO_HEADING	Rotates in place to an absolute odometry yaw, closing the loop on measured angle rather than on elapsed time	On arriving at the target heading
FAILSAFE	Stops the wheels and ends the run. Terminal, with no path out	It does not

Two things in that figure are worth reading carefully, because both are deliberate and both cost the rover time. The opening sweep runs once, on the first pass only: re-running an 85-second look-around every time the rover changed its mind would leave it doing nothing else. And a completed turn returns to the settle-and-re-scan state rather than straight to driving. The rover has just been rotating, so its accelerometer-derived tilt estimate is still corrupted by that rotation, and the corridor it is now facing is a different piece of world from the one it scanned before turning. Driving on the pre-turn scan would mean acting on a reading that no longer describes what is ahead. Every mission-ending transition is drawn in red, and the four numbered drops at the foot of the figure list which trigger ends a run from which state.

Reactive-explorer state machine, as shipped

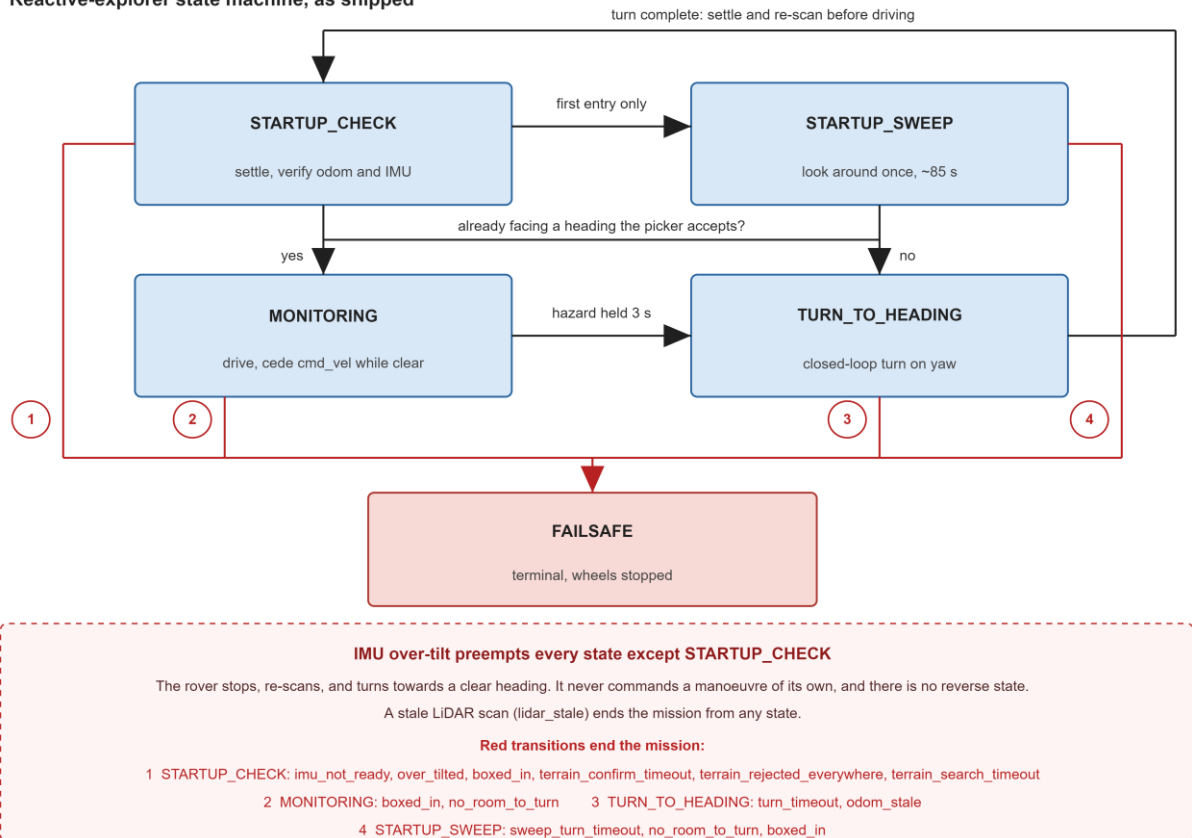


Figure 3.6: The reactive-explorer state machine as shipped. Red transitions end the mission.

IMU slope-triggered retreat. This is the one behaviour in Table 3.8 that was built, unit-tested, and then deliberately removed, and it is reported here because the reason for removing it is a result in its own right. The design backed the rover straight out of an over-tilt without rotating, on the reasoning that turning on a slope risks a rollover. It was never run live. The Gazebo `slope_zone` surface it was written against had already been removed from `mars_terrain.world` during a world restructuring, so no inclined surface existed in simulation to trigger it, and the physical slope-ramp test was cancelled on safety grounds before it ran.

During the hardware phase two independent problems then closed the question. A blind reverse is the most dangerous manoeuvre this rover can make, because it has no rear-facing sensing of any kind, and it is what drove the rover into a wall during hardware testing. Separately, the accelerometer-only tilt estimate was reporting apparent tilts of up to 62.9 degrees on a flat laboratory floor, because

that estimate cannot separate gravity from the rover's own acceleration, so the behaviour was firing on readings that were not real. Reverse motion was therefore removed from the stack entirely. An over-tilt is now handled exactly like a blocked corridor: hold still, re-scan, and turn towards a heading that is clear. The IMU keeps its veto over where the rover may go and loses its ability to command a manoeuvre of its own. Section 6.4 returns to what a rear-facing sensor would unlock.

Stuck-in-sand detection. This addresses a failure mode with no hazard signal at all, wheels spinning uselessly in loose sand. `stuck_detection_node` compares commanded velocity against actual odometry displacement over a 4-second window and escalates if displacement falls below 20% of what the command should produce. A second, independent path applies the same test to pure in-place rotation, since the linear check cannot see a wheel dug in during a turn with no forward component. Real ExoMy hardware has no wheel encoders and no gyro, so no equivalent signal exists there yet, and real-hardware detection would need LiDAR scan-matching or camera-based motion detection, neither built here.

The cross-planet generalisation probe. This is the one behaviour that is an experiment rather than a capability, and it was pre-registered, meaning its interpretation criteria were committed to the repository before any feature was extracted. Al4Mars and RUGD (Earth off-road) images are mapped onto a shared three-class taxonomy fixed by physical property rather than by name, and three logistic regression probes (Mars-only, Earth-only, joint) are trained on the frozen features and tested per planet. A sanity gate requires each single-planet probe to reach 85% on its own planet. Given that gate passes, a positive result needs at least 50% cross-planet transfer in one direction and a joint-probe cost under 5 percentage points on both planets; an honest-negative result is transfer at or below 40%, which is near chance; anything between is mixed. Fixing these thresholds in advance is what makes the negative result reported in Section 4.9 informative rather than a post-hoc rationalisation.

3.13 Summary

This chapter set out the full methodology: the AI4Mars dataset (287 gold-standard test images) evaluated against the 96.67% DeepLabv3+ supervised ceiling, across ten experiment groups from CLIP zero-shot classification through a 22-model, six-paradigm survey to a full Gazebo and real-hardware autonomy stack. Every experiment shares a consistent protocol: the 287-image test set, the same AI4Mars training pool for few-shot sampling, frozen encoders throughout with no gradient-based fine-tuning, and a logistic regression linear probe ($C=0.316$, seed=42) wherever a probe is trained.

The key design decisions were: frozen encoders only, to match the thesis's resource-constrained scope; a cross-paradigm survey of 22 models to empirically identify which pretraining objective transfers best to Mars geological texture; a dual-process cascade balancing a fast and a slow model; conservative traversability rules that prefer a false alarm over a missed hazard; and SmolVLM over BLIP-2 as the Raspberry Pi-deployable VLM, on the strength of its 1.54 GB against BLIP-2's 6.5 GB memory footprint.

4 Results

4.1 Overview

This chapter reports the results from all ten experiment groups set out in Chapter 3, plus the extended reactive-safety and autonomy stack built on top of the deployed classifier. All classification experiments use the AI4Mars gold-standard test set (287 images, three reportable classes: soil, bedrock, sand) and are measured against the supervised DeepLabv3+ ceiling of 96.67% (Swan et al., 2021).

4.2 CLIP zero-shot classification

CLIP ViT-B/32, given one prompt per class and no labelled Mars images at all, reaches 34.8% overall accuracy: soil 64.2%, bedrock 2.2%, sand 26.4%. Bedrock is almost never predicted correctly, because the word "rock" appears in both the bedrock and big-rock prompts and pulls their embeddings together. This 61.9-point gap to the supervised ceiling sets the baseline the rest of the thesis works to close. Inference runs at 154 ms per image (6.5 FPS) on CPU.

4.3 Prompt engineering

Twelve prompt variants were tested (v1 to v12, described in Section 3.7). The best, v9, a NAVCAM-prefixed soil prompt combined with a texture-and-scale bedrock prompt, reaches 54.4% overall, a 19.6-point gain over the baseline with no retraining at all: soil 95.9%, bedrock 18.5%, sand 29.2%. The study surfaces a soil-bedrock trade-off intrinsic to CLIP's embedding space. Prompts tuned for soil (v6, 96.7% soil) collapse bedrock to about 5%, and prompts tuned for bedrock (v4, 91.3% bedrock) drop soil to about 45%. In v9's confusion matrix, bedrock images are most often mistaken for soil (49 of 92), showing the soil-favouring prompt still absorbs some bedrock into its region. Since no held-out validation set was used, this is an upper-bound result with some risk of overfitting to the 287-image test set, an acknowledged limitation.

4.4 Few-shot linear probe

A logistic regression probe on frozen CLIP features climbs steadily with more labels: 57.5% at 10 shots per class, 76.0% at 100, 87.8% at 1,000 (gap to supervised: 8.9 points). Ten labels per class already beats zero-shot by 22.7 points. Gains flatten past 500 shots (86.1% to 87.8%, +1.7 points for double the data), and bedrock stays the hardest class throughout, reaching only 79.3% at 1,000 shots against the supervised 94.9%. Figure 4.1 plots this curve alongside the deployed DINOv2+reg ViT-S/14 model and the best single encoder, DINOv2 ViT-L/14, showing all three converging towards, but not reaching, the supervised ceiling as label count grows.

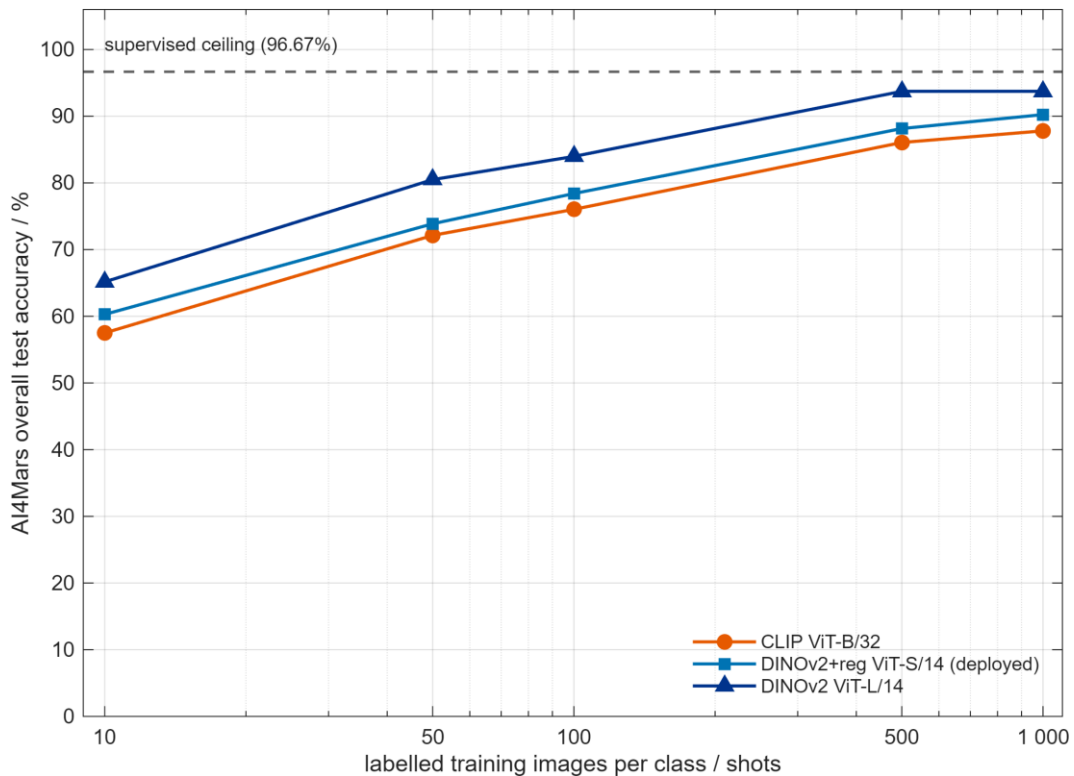


Figure 4.1: AI4Mars accuracy against labelled images per class for CLIP and two DINOv2 encoders.

4.5 Vision-language model comparison

SAM2 used as a pre-processing segmentation step before CLIP classification made results worse, not better: 76.7% against 80.0% for CLIP alone on the same 30-image sample, at 34 times the latency (5,274 ms against 154 ms). SAM2 was

therefore removed from the classification pipeline and kept only for illustrative terrain-boundary overlays.

BLIP-2, tested on a preliminary 20-image, soil-dominant sample (no sand images at all), reached 80.0% overall, with perfect soil recall but 0% bedrock, at 753 seconds per image, far too slow for any practical use and impossible to scale to the full 287-image set in reasonable time.

SmoIVLM-256M, evaluated on the full 287-image set, reached 49.78% overall (soil 67.3%, bedrock 21.9%, sand 50.0%, the first VLM result on this dataset's sand class), at 61 seconds per image and 1.54 GB RAM. 21.6% of responses (62 of 287) came back as the single word "Mars.", an echo of the word "Mars" in the prompt. Removing "Mars" from the prompt, tested on the same 20-image subset as BLIP-2, eliminated this parsing failure entirely but collapsed soil accuracy from 67.3% to 0% and overall accuracy to 15.0%, so the "Mars" cue was kept in every reported result and its 21.6% parse-failure rate is reported as a known limitation. SmoIVLM-256M needs only 1.54 GB against BLIP-2's 6.4 GB and runs 12.3 times faster, making it the only practical VLM for the Raspberry Pi 4.

A later successor, SmoIVLM2-256M, tested with a description-style prompt on the full set, scored 48.0% overall, essentially unchanged but redistributed. Bedrock rose to 76.7% (+54.8 points) and sand to 79.7% (+29.7 points), while soil collapsed to 4.5% (-62.8 points), because the model now defaults to "rocky" for most undifferentiated Mars surfaces. It runs 38% faster (37,765 ms) with 57% less RAM (0.67 GB) and a lower parse-failure rate (6.3%), making it the more practical edge choice despite the soil collapse, since the fast DINOv2 path handles most decisions and the VLM slow path is only a semantic check.

4.6 Cascade pipeline in Gazebo (CLIP + SmoIVLM)

Across the five Gazebo zone positions, which cover four terrain types with the rock terrain tested at two positions, the CLIP fast path scored 2 of 5 (40%) on traversability, matching its 34.8% AI4Mars zero-shot accuracy and confirming the same failure pattern carries into simulation. CLIP over-predicts big-rock on flat

ground (sand at 0.87 confidence, soil at 0.69 confidence, both misread as hazard), a conservative failure mode that a rover would rather have than the reverse. Running both models together slowed each down (CLIP from 136 to 330 ms per frame, 2.4x; SmolVLM from 55 to 141 seconds per call, 2.6x) from CPU contention. In one qualitative case, CLIP flagged a rock-textured surface as big-rock at 0.806 confidence (hazard), and SmolVLM's slower VQA reasoning correctly reclassified it as bedrock (caution), demonstrating the value of the semantic slow path for the fast path's mistakes.

4.7 Cross-paradigm model survey (22 models, six pretraining paradigms)

A 1,000-shot linear probe (logistic regression, $C=0.316$, seed=42) was run on 22 frozen encoders spanning six pretraining objectives: DINO self-distillation, positional-debiased clustering (Franca), contrastive (CLIP, SigLIP2), autoregressive (AIMv2), masked-image-modelling with a CLIP teacher (EVA-02), and multi-teacher distillation (RADIO).

The ranking follows. One per-class column is given alongside overall accuracy, and it is bedrock, because bedrock is the only class that separates these models. Soil sits above 95% for almost every encoder tested and sand varies little, so columns for either would repeat the same figure down all 22 rows. Bedrock instead spans 9.8% to 93.5%, and the split falls along paradigm lines rather than along overall accuracy, which is the finding this table exists to show. Appendix C gives the same two accuracy columns for all 22 variants with parameter counts and feature dimensions.

Table 4.1: Top-ranked models by 1000-shot AI4Mars overall accuracy (full 22-model table in Appendix C).

Rank	Model	Paradigm	Overall	Bedrock (hardest class)	Gap to supervised
1	Ensemble B (DINOv2 ViT-L + ViT-B, 1792-d)	DINO ensemble	94.43%	91.32%	-2.24 pp
2	DINOv2 ViT-L/14	DINO	93.73%	90.94%	-2.94 pp

Rank	Model	Paradigm	Overall	Bedrock (hardest class)	Gap to supervised
3	Probabilistic stacking (4-model)	Ensemble	93.38%	93.50%	-3.29 pp
4	DINOv3 ViT-L/16	DINO	92.33%	89.13%	-4.34 pp
5	DINOv2+reg ViT-B	DINO	91.99%	88.04%	-4.68 pp
6	DINOv2 ViT-B/14	DINO	91.30%	87.04%	-5.38 pp
7	DINOv3-SAT ViT-L	DINO (satellite)	90.94%	93.5% (best single-model bedrock)	-5.73 pp
8	DINOv3 ViT-B/16	DINO	90.60%	83.70%	-6.08 pp
9	DINOv2+reg ViT-S (deployed on Raspberry Pi)	DINO	90.24%	84.78%	-6.43 pp
10	DINOv3 ViT-S/16	DINO	90.20%	83.70%	-6.47 pp
11	DINOv2 ViT-S/14	DINO	89.90%	84.78%	-6.77 pp
12	Franca-RASA (304.8M, 1024-d)	Positional debiasing	89.55%	89.13%	-7.12 pp
12	Franca- CLS+RASA (304.8M, 2048-d)	Positional debiasing	89.55%	88.04%	-7.12 pp
14	AIMv2-L (307M, 1024-d)	Autoregressive	87.80%	77.17%	-8.87 pp
14	CLIP ViT-B/32	Contrastive	87.80%	79.3%	-8.87 pp
14	DINOv3 ConvNeXt-Tiny	DINO (CNN)	87.80%	80.4%	-8.87 pp
17	SigLIP2-B/16	Contrastive (sigmoid)	86.10%	80.4%	-10.57 pp
18	Franca-CLS (304.8M, 1024-d)	Matryoshka backbone	84.67%	80.43%	-12.00 pp
19	EVA-02 ViT-B/14	MIM + CLIP teacher	81.20%	64.1%	-15.47 pp
20	SAM2 Hiera-tiny	Segmentation	75.30%	73.9%	-21.37 pp
21	RADIO-B	Multi-teacher	65.20%	67.4%	-31.47 pp
22	Depth Anything V2-S	Depth estimation	36.60%	9.8%	-60.07 pp
-	DeepLabv3+ (supervised)	Fine-tuned	96.67%	94.9%	0.00 pp

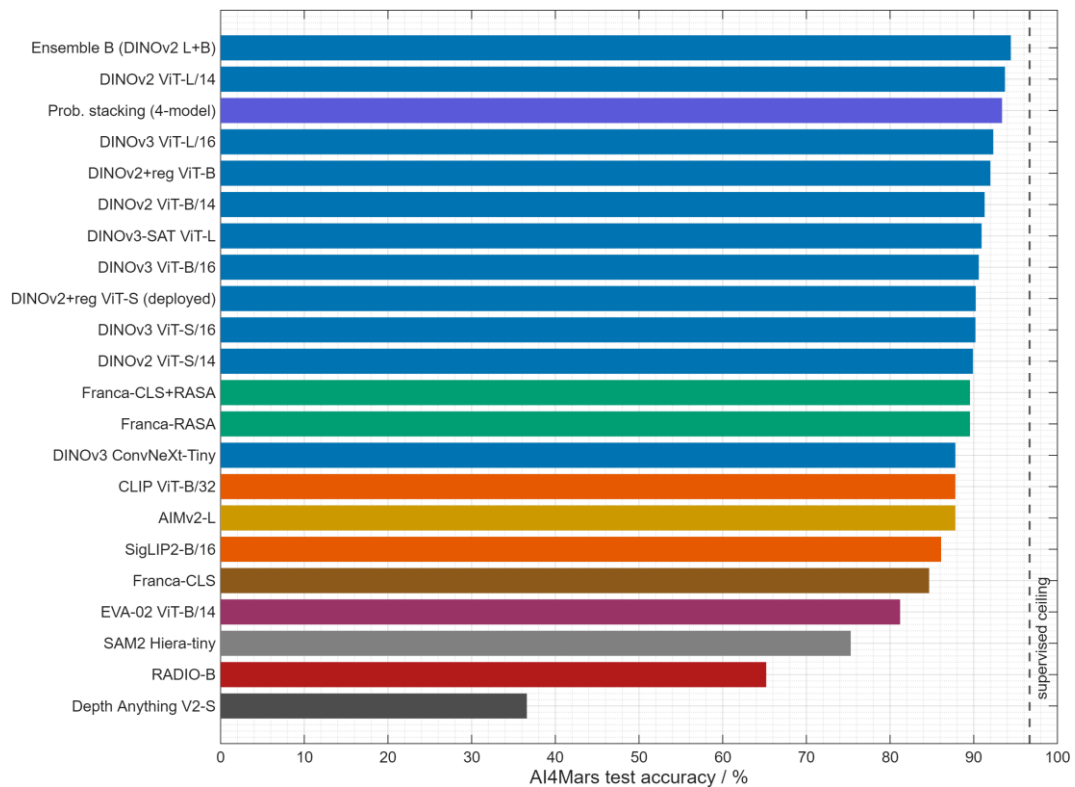


Figure 4.2: 1000-shot AI4Mars accuracy for all 22 model variants, coloured by pretraining paradigm.

Figure 4.3 summarises the best result from each of the eight pretraining paradigms surveyed, showing DINO self-distillation (Ensemble B, 94.43%) leading every other objective by at least 4.9 points.

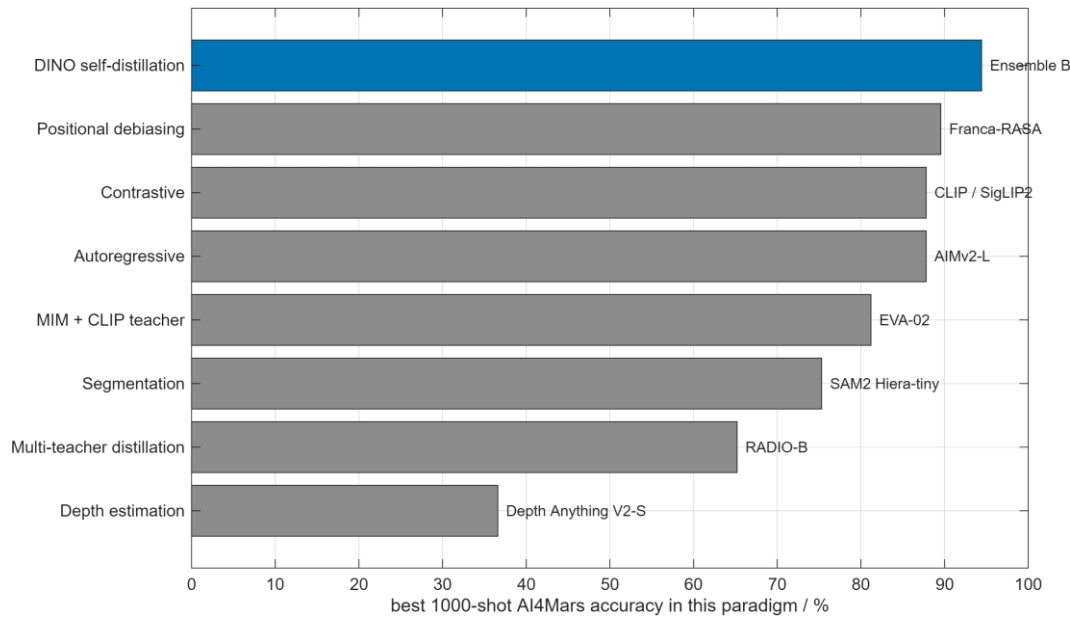


Figure 4.3: Best 1000-shot AI4Mars accuracy within each pretraining paradigm.

Six findings summarise this survey.

1. **The eleven highest-ranked variants are all DINO encoders or ensembles of them**, and every DINO Vision Transformer clears 89.9%. Two qualifications keep the claim exact: Franca's positional-debiasing variants come closest of any other paradigm at 89.55%, and DINOv3's ConvNeXt-Tiny, the one convolutional DINO tested, drops to 87.8%, level with the best contrastive and autoregressive models. The advantage therefore belongs to the DINO objective carried on a Vision Transformer, not to the DINO label alone.
2. **Bedrock is where paradigms diverge most sharply.** DINO models reach 83.7-93.5% on it, while CLIP-teacher-influenced models (EVA-02, RADIO) collapse to 64-67%, despite EVA-02's overall accuracy of 81.2% being far from the worst. EVA-02's soil-bedrock confusion mirrors CLIP's own embedding geometry.
3. **Feature-space tricks that leave the pretraining objective alone do not help.** Multi-layer CLS-token concatenation lowers accuracy by 0.3-0.7 points at every scale tested, monocular depth fusion adds no signal and depth-only classification manages 36.6%, and Tip-Adapter underperforms the logistic-regression probe by 5-18 points at every shot count.

4. **RADIO's poor result (65.2%) is largely a dimensionality problem**, not a representation one. Its 2,304-dimensional composite feature space leaves a 3,000-sample probe under-specified, at a sample-to-dimension ratio of 1.3 against DINOv2's 3.9.
5. **Backbone scaling within the DINO family helps consistently**, and disproportionately at low label counts: DINOv2 ViT-S to ViT-L gains 11.5 points at 10-shot but only 2.1 at 1000-shot. Figure 4.4 plots this across all three ViT sizes for both families.
6. **Curated data beats more data**. DINOv2's older 142-million-image pretraining set (LVD-142M) beats DINOv3's larger but web-crawled 1.69-billion-image set (LVD-1689M) at matched scale, 91.3% against 90.6% at ViT-B and 93.73% against 92.33% at ViT-L.

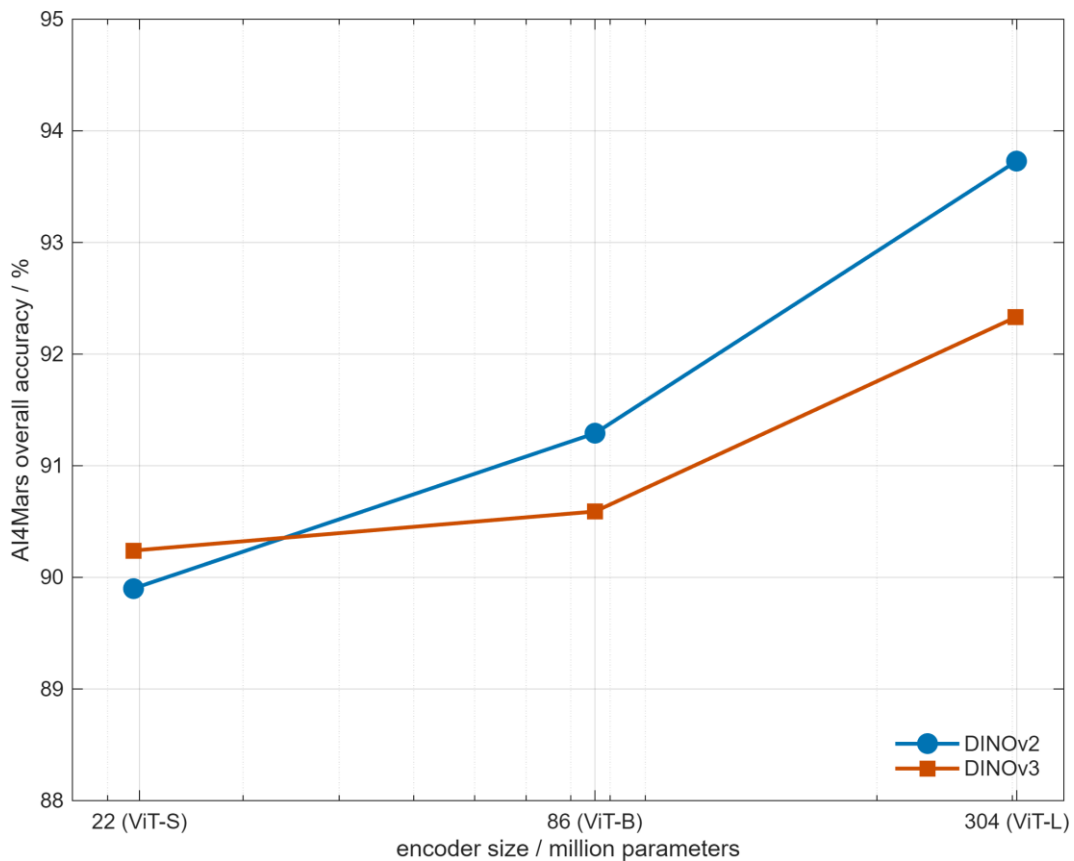


Figure 4.4: AI4Mars accuracy against encoder size for the DINOv2 and DINOv3 families.

Sixth, the best result overall, Ensemble B (concatenating DINOv2 ViT-L and ViT-B features, 94.43%), is only 0.70 points above the single DINOv2 ViT-L encoder (93.73%), and a McNemar's test on their 10 discordant predictions out of 287 images found this gap not statistically significant ($\chi^2=0.10$, $p=0.75$). DINOv2 ViT-L alone is therefore the recommended configuration where compute allows it, giving statistically equivalent accuracy at half the feature dimensionality and one encoder pass instead of two.

Figure 4.5 breaks down DINOv2 ViT-L's 18 errors on the 287-image test set (93.73% accuracy on this run, matching the reported headline figure exactly). Soil is classified almost perfectly (97.6%). The two real error modes both run in the same direction: 7 of 92 bedrock images are mistaken for soil, and 8 of 72 sand images are mistaken for bedrock. Both are between visually adjacent classes, and neither crosses into a class that would flip a safety-critical decision the wrong way (soil and sand are both non-hazard classes; nothing is mistaken for, or lost from, a hazard class here).

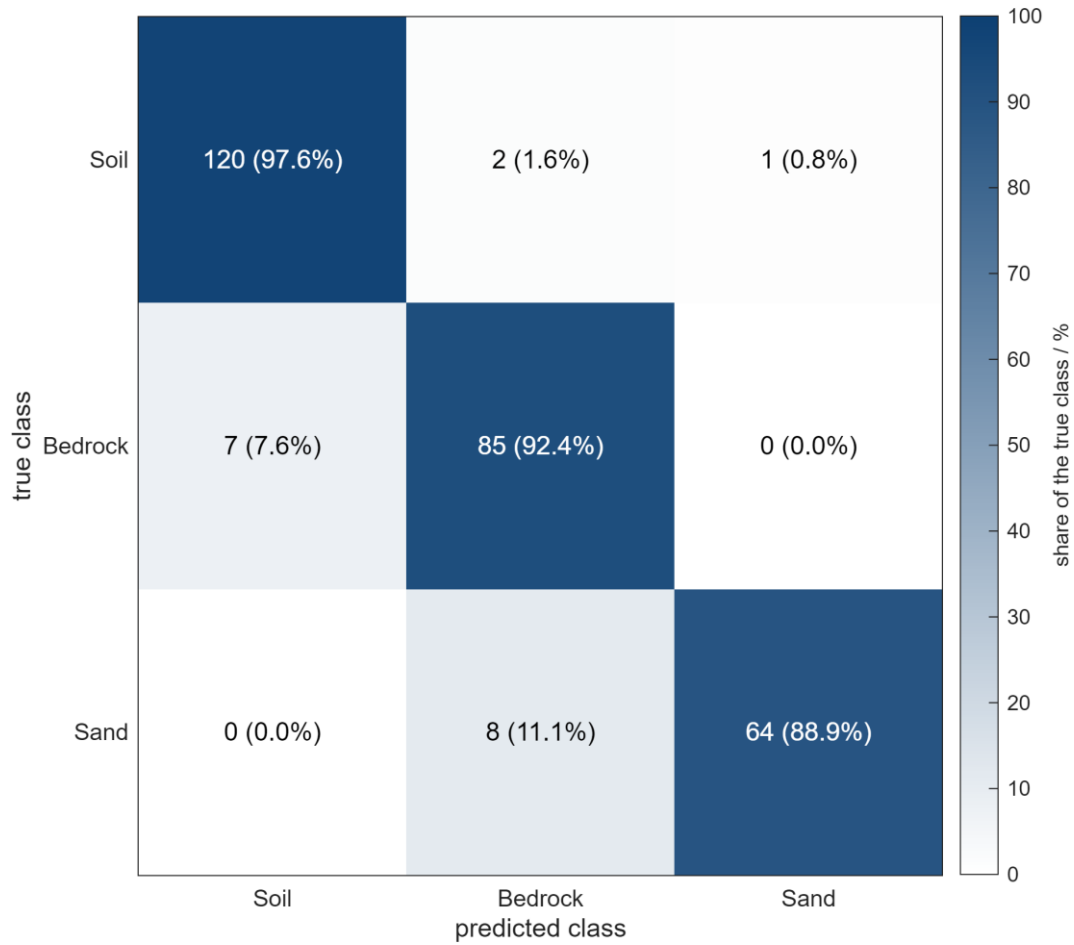


Figure 4.5: Row-normalised confusion matrix for DINOv2 ViT-L/14 at 1000-shot.

4.8 Deployment-relevant checks

Calibration. Post-hoc temperature scaling on the deployed DINOv2+reg ViT-S/14 probe cuts Expected Calibration Error by 80.8% (from 0.1695 to 0.0325, $T^*=0.461$) with no change to accuracy, and failure analysis finds zero misclassifications above 0.90 confidence, meaning the model's errors are reliably signalled by low confidence. A coverage-risk sweep confirms the deployed 0.40 confidence threshold sits at the Pareto knee of the trade-off (99.7% coverage; raising the threshold to 0.90 buys only +5.2 points of safety at a cost of 35.9 points of coverage), shown in Figure 4.6.

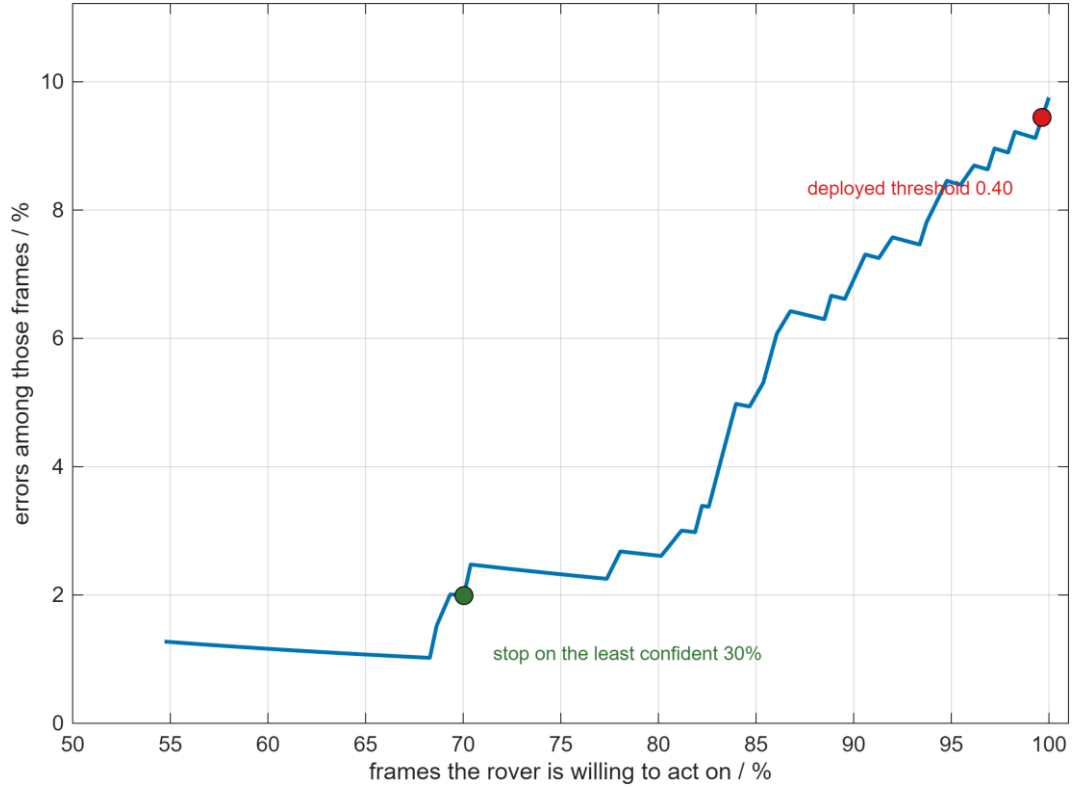


Figure 4.6: Coverage-risk trade-off across the confidence threshold sweep.

Statistical confidence. Wilson 95% confidence intervals were computed for every headline number. The deployed model's 90.24% carries a ± 3.45 -point interval; DINOv2 ViT-L's 93.73% and Ensemble B's 94.43% intervals overlap substantially ([90.30, 96.00] and [91.14, 96.54]), consistent with the McNemar result above. CLIP zero-shot's interval ([29.56, 40.52]) and CLIP prompt-engineered's interval ([48.57, 60.02]) do not overlap, supporting the +19.6-point prompt-engineering gain as a real effect. The real-camera results reported later in this chapter are given per class rather than as a single accuracy, because the classes differ so sharply that an aggregate figure would be misleading, and because the per-class sample sizes range from 13 to 36 images.

4.9 Cross-sensor and cross-dataset transfer

Applying the AI4Mars-trained probe, with no retraining, to Mars-Bench (a Perseverance MastCam colour-imagery benchmark, a different sensor from Curiosity's grayscale NAVCAM) gives 24.94% zero-shot accuracy, a 65.30-point drop from the 90.24% in-domain figure. The probe predicts bedrock for 57% of

images regardless of true class, because MastCam's colour soil texture activates features the NAVCAM-trained probe associates with bedrock (mean confidence on misclassified soil: 0.951, overconfident out of domain). Training a probe directly on Mars-Bench's own data recovers most of this gap: 84.07% in-domain accuracy (59.13 points above zero-shot, 11.23 points above a majority-class baseline). A joint probe trained on both AI4Mars and Mars-Bench together reduces the cross-sensor gap on mappable classes from 35.80 to -5.56 points, at a cost of only 3.83 points on AI4Mars's own accuracy (90.24% to 86.41%). The frozen encoder's features are sensor-separable but not sensor-shared, and are recoverable with a small amount of joint labelled data rather than free cross-sensor transfer.

The same pattern holds one level up, across planets rather than sensors. A pre-registered probe (fixed before any feature was extracted) mapped AI4Mars (Mars) and RUGD (Earth off-road) onto a shared three-class physical-surface taxonomy. Both single-planet probes comfortably clear their 85% sanity gate (97.19% Mars, 96.68% Earth), ruling out a broken pipeline, but cross-planet transfer fails the pre-registered honest-negative threshold in both directions: Earth-trained tested on Mars scores 13.45%, below the 33.3% chance level, and Mars-trained tested on Earth scores 35.74%, near chance. A joint probe trained on both planets still recovers near-single-domain accuracy on each (cost under 1.4 points either way), extending the same conclusion. DINOv2's frozen features separate domains cleanly without unifying them, whether the domain boundary is a sensor or a planet.

4.10 DINOv2 traversability in Gazebo

Replacing CLIP with DINOv2+reg ViT-S/14 (the same 90.24% AI4Mars model) as the Gazebo traversability backbone, after eight rounds of world and texture iteration, reaches 3 of the 5 zone positions correct (60%) and 5 of 5 safety compliance (100%): soil, bedrock, and sand all classify and command speed correctly; both rock positions fall below the 0.40 confidence threshold (0.33 and 0.37) and correctly STOP despite the wrong label, shown per-zone in Figure 4.7. The two failures trace to the AI4Mars big-rock class's 108-image training set

(against roughly 1,000 for the other three classes), a structural property of the dataset rather than a simulation defect. One genuine rendering finding emerged along the way. At coarse texture-tile scale, the bedrock surface's crack macro-structure aliased into a uniform grey blur that DINOv2 read as soil; reducing the tile scale from 0.35 to 0.08 restored the crack pattern and immediately fixed bedrock classification.

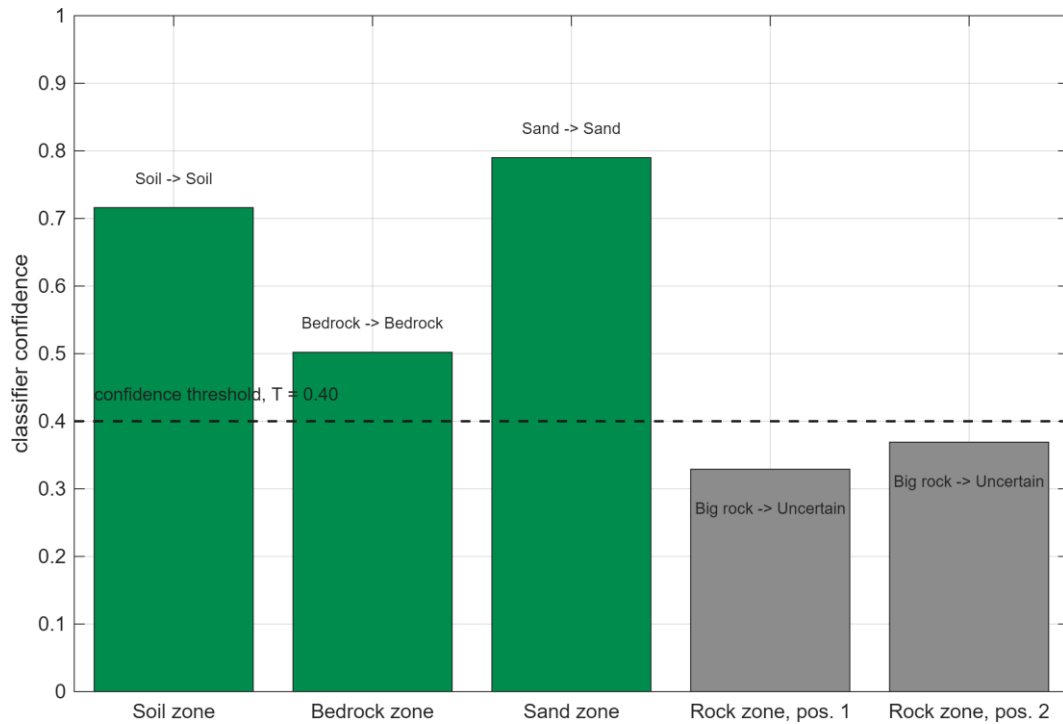


Figure 4.7: Classifier confidence at each of the five Gazebo zone positions.

A separate finding held throughout. Swapping the sphere-primitive rocks for photorealistic mesh rock models, tested across 11 lighting and material combinations, consistently caused both rock zones to be read as sand (0.54-0.72 confidence) rather than uncertain, converting the safe STOP into an unsafe CAUTION drive-through. Visual realism and classification safety pulled in opposite directions here, so the sphere primitives were kept for the safety-critical rock zone despite their lower visual fidelity. A lighting-robustness check across dawn, noon, and dusk variants found soil and bedrock classification stable across all three, but the rock zones' safety outcome was lighting-dependent. At noon, both rock zones read as sand with a positive forward speed, the worst safety

result of any tested lighting condition, showing the pipeline is photometrically as well as geometrically sensitive.

Comparing four models at the same five zone positions. (CLIP, DINOv2+reg ViT-S, DINOv3 ViT-S, DINOv2 ViT-L) found that AI4Mars accuracy does not predict Gazebo performance. DINOv3 ViT-S (90.20% AI4Mars) scored only 2 of 5 in Gazebo, tied with CLIP (54.4% AI4Mars), while DINOv2+reg ViT-S (90.24%) and DINOv2 ViT-L (93.73%) both scored 3 of 5. No model classified both rock zones correctly. Safety compliance varied more than terrain accuracy. DINOv2+reg ViT-S reached 5 of 5 safety (both rock failures fall below threshold and STOP), DINOv2 ViT-L reached 4 of 5 (one rock zone incorrectly reads as soil and drives through at full speed), and DINOv3 ViT-S reached only 2 of 5 (both rock zones read as sand or bedrock with confident, non-hazard speeds). DINOv2+reg ViT-S/14 remains the recommended deployment model: equal or better terrain accuracy, best safety compliance, 14 times fewer parameters than DINOv2 ViT-L, and a fraction of its inference latency.

A domain-adaptation attempt made things worse, informatively. Augmenting the training set with 738 photometrically-varied DINOv2 features extracted from Gazebo's own rock-zone frames, labelled as big-rock, achieved perfect recall on those same training features (0.947 mean confidence) but collapsed live Gazebo performance to 2 of 5 terrain and 2 of 5 safety. Every zone, including soil, bedrock, and sand, was now classified as big-rock and the rover stopped everywhere. Cosine-similarity analysis showed why. Every Gazebo zone, not just the rock zones, sits at 0.60-0.77 similarity to the mean Gazebo rock feature, because Gazebo's synthetic materials share a lighting model and a texture resolution that place them all in one region of DINOv2's feature space, distinct from the well-separated clusters real AI4Mars imagery produces. Adding Gazebo-domain rock examples therefore swept that whole region into the big-rock class. This rules out naive same-domain augmentation as a fix, and locates the sim-to-real gap at the feature level rather than the classification level.

4.11 Raspberry Pi hardware benchmark

Figure 4.8 places the deployed model's accuracy-latency trade-off against the wider DINOv2/DINOv3 family and CLIP, and shows why DINOv2+reg ViT-S/14 (the red triangle) was the deployment choice, since it sits at the fastest end of the DINO family with an accuracy no worse than DINOv3's equivalent scale, well ahead of CLIP at similar latency.

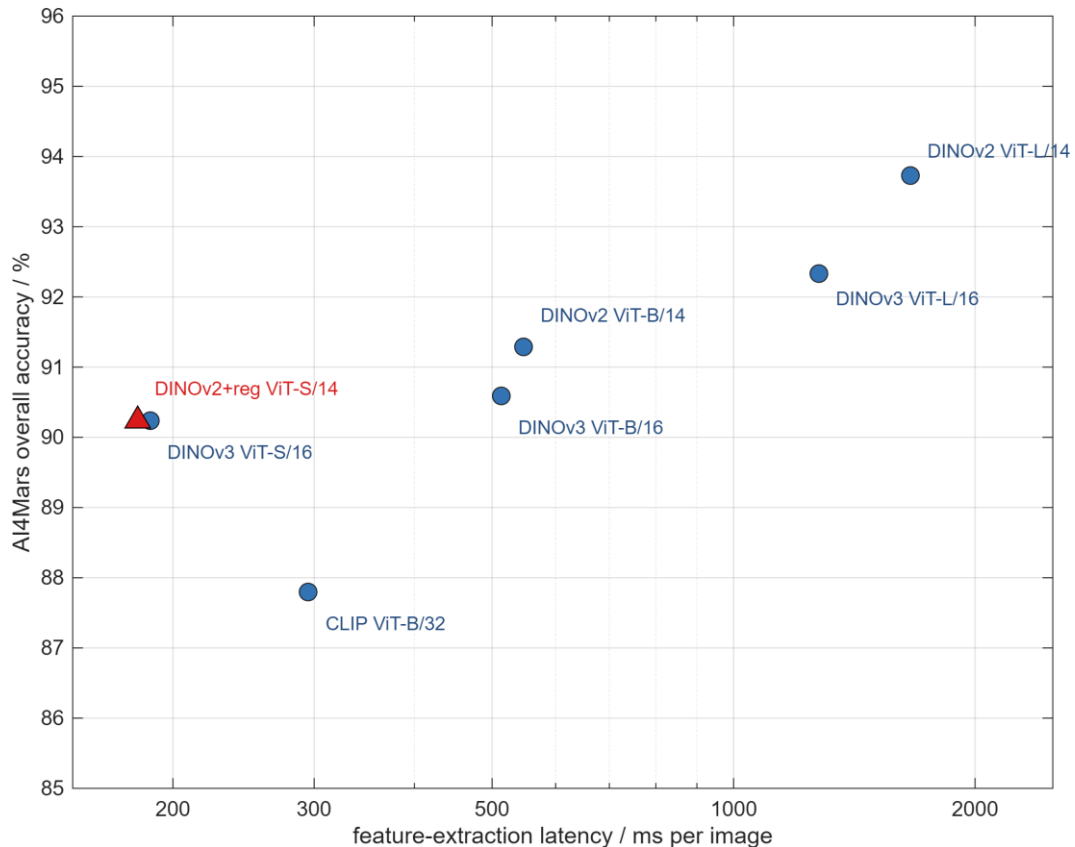


Figure 4.8: AI4Mars accuracy against feature-extraction latency, with the deployed model marked.

DINOv2+reg ViT-S/14, exported to ONNX Runtime, was benchmarked on a Raspberry Pi 4 Model B (8 GB, Cortex-A72, Raspberry Pi OS Lite 64-bit). It runs at 570 ms per image (1.75 FPS) using 258 MB RAM, against a PC ONNX baseline of 178 ms, a 3.2x slowdown from the Cortex-A72's weaker single-core throughput. PyTorch could not be installed on this Raspberry Pi's Python 3.13.5 and ARM64 combination, so ONNX Runtime is the only viable backend here. That 570 ms misses the ≤ 500 ms target for 2 Hz operation, but at the rover's

commanded 0.05-0.10 m/s it still gives a new classification every 28 to 57 mm of travel, finer than macro-scale terrain identification needs, so the system is judged operationally viable despite missing the nominal target. Feature extraction is over 99.9% of that latency; the probe itself costs 0.19 ms. The 258 MB footprint leaves ample headroom on 8 GB for the full cascade, including a periodic SmolVLM-256M call.

An INT8-quantised version of the same encoder runs faster still, 386 ms in isolation (2.59 FPS, 163 MB RAM), but this isolated figure understates the deployed latency substantially. Measured on the rover with the full real-hardware stack running (camera, LiDAR driver, IMU, the controller's finite state machine (FSM), and the rosbag recorder, thirteen nodes competing for four CPU cores), a single inference took 1,865-2,200 ms, roughly five times slower, which forced the controller's staleness timeout up from 3 to 7 seconds. INT8 quantisation also costs more accuracy than first measured. An initial test using only the first 200 of 287 gold-standard images and a squashed rather than centre-cropped resize found a 0.50-point drop (94.0% to 93.5%); re-measured on all 287 images with the deployment preprocessing, the real drop is 3.83 points (90.24% to 86.41%), concentrated entirely in the soil class (bedrock and sand are unchanged). 86.41% is the figure this thesis quotes for real-hardware INT8 accuracy.

4.12 Real camera deployment

The deployed model was tested on 111 photographs taken with the rover's camera type around the university campus and in the laboratory sandpit. Of these, 72 show one of the terrain classes cleanly enough to score against a known label, and the remaining 39 are walls and other non-terrain scenes used for the out-of-distribution check reported later in this section. Reporting the 72 by terrain class rather than as one overall accuracy keeps a domain gap in one class from being hidden inside an average. Results are given in Table 4.2 and Figure 4.10.

Table 4.2: Deployed-model accuracy on 72 photographs taken with the rover's own camera, by terrain class.

Terrain class	Source	Images	Accuracy	95% CI / %
Soil	Campus	13	100.0%	77.2 to 100.0
Sand	Laboratory sandpit	23	73.9%	53.5 to 87.5
Big rock	Laboratory sandpit	36	0.0%	0.0 to 9.6

The 95% CI column gives a Wilson score confidence interval, the range the true accuracy is expected to fall within if the same measurement were repeated on a fresh set of photographs of the same kind. It is reported because the per-class sample sizes here are small, and a bare accuracy figure at $n=13$ would imply a precision the measurement does not have.

The reference point for these numbers is the 90.24% the same model reaches on the 287-image AI4Mars gold-standard test set, reported in Section 4.4. Against that reference, the real camera results separate sharply by class. Real soil is classified perfectly, all 13 of 13. Real sand degrades but holds, 17 of 23, with all six errors read as soil.

The big-rock row needs stating carefully, because part of it follows from a decision made in Section 3.7 rather than from anything the model did. AI4Mars provides only 108 big-rock training images against roughly 1,000 for each other class, too few to train a probe on, so big rock was excluded from the label set. The deployed classifier therefore has three outputs, soil, bedrock, and sand, and cannot return the answer big rock at all. Its accuracy on this class is 0.0% by construction, and no encoder, however large, could change that. Section 5.6 reports a supplementary experiment that puts the class back into the label set, to establish whether the frozen features can separate it at all.

What the 36 photographs do measure, and what matters for safety, is what the model says instead. It read 30 of them as soil and the other 6 as sand, at confidences between 0.47 and 0.92 and averaging 0.50. Every one of those readings clears the 0.40 threshold that separates a CAUTION drive-through from

a STOP. The failure is therefore not that the model cannot name a rock, which was known in advance, but that it does not recognise a rock as unfamiliar. A classifier restricted to three classes could still have been safe here by returning low confidence on a class it was never taught, letting the threshold convert ignorance into a stop. It does the opposite, and answers confidently and wrongly. That is the specific way the Gazebo safety guarantee fails to reach the real camera, and it is a calibration failure rather than a capacity one.

No real-world bedrock was available to photograph, so bedrock is absent from Table 4.2 entirely. Its behaviour on a real camera is untested, and is stated here as a gap rather than assumed to follow soil.

Two further observations follow from the same set. First, the mechanism is not a colour gap. AI4Mars is grayscale NAVCAM imagery, so there is no hue to lose. The mismatch is one of brightness and sensor response between the laboratory's white silica sand and NAVCAM's grey tones, which is consistent with the pattern observed, since the class defined largely by overall brightness (sand) degrades, while the class defined by fine texture (soil) does not. Second, the model has no ability to reject what lies outside its training distribution. Seventeen photographs of campus building walls were all classified as soil, at confidences up to 0.99, and 22 photographs of mixed grass, road, and soil were likewise all classified as soil. Neither surface has any counterpart in the training data, and neither triggered the uncertainty threshold, which is the same overconfidence seen on big rock appearing on classes the label set was never meant to cover at all.

The big-rock result was confirmed twice more on the rover itself rather than left as an offline finding. Across 214 real camera frames captured during hardware testing, the big-rock class was never predicted once, including on 21 frames of actual rocks. Across 12 live autonomous runs with a printed Mars terrain backdrop installed around the sandpit, the rover repeatedly drove towards and faced the big-rock panel and again never classified it correctly. Substituting a much larger DINOv2 ViT-L encoder did not fix the class or meaningfully raise its confidence, which is consistent with the training-imbalance explanation rather than an encoder-capacity one.

An earlier pilot test preceded this set. Twenty images (10 sand, 10 big rock) taken in the laboratory sandbox were classified by the same model in June 2026, giving 40.0% on sand and 0.0% on big rock. That pilot is superseded here and is not used as a headline figure, for two reasons. It covered only two of the four terrain classes, omitting soil and bedrock, so any overall accuracy computed from it describes the model's worst case rather than its behaviour. It also carried a Wilson 95% confidence interval of ± 16.77 points at $n=20$, the widest of any measurement in this thesis. The set reported above is five times larger on the real-camera side, covers a third class, and is corroborated by live rover operation, so it is used instead. The pilot figures are retained in Appendix B for completeness.

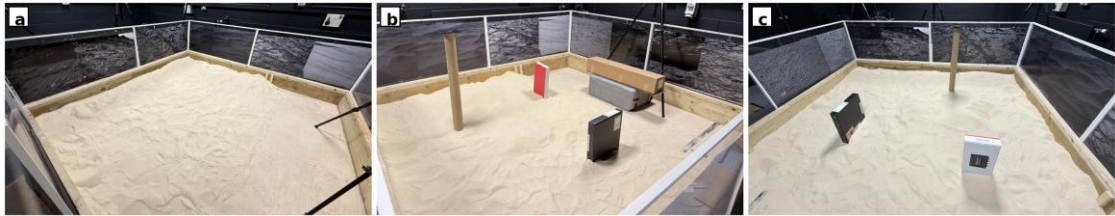


Figure 4.9: The laboratory sandpit, with printed Mars terrain panels around the walls. (a) the arena empty, (b) and (c) two of the obstacle layouts tested.

This laboratory sandpit is not a Mars analogue, and these results should be read as a hardware-deployment finding rather than a claim about Mars performance, which is represented by the AI4Mars (90.24%) and Mars-Bench (84.07%) results. The practical conclusion is narrower than a blanket sim-to-real failure. Soil transfers intact and sand transfers with degradation, while big rock does not transfer at all, so the recommendation is to collect real rover imagery of big rock and retrain the probe with that class restored to the label set, and to photograph real bedrock to close the remaining gap in the evidence, rather than to retrain everything.

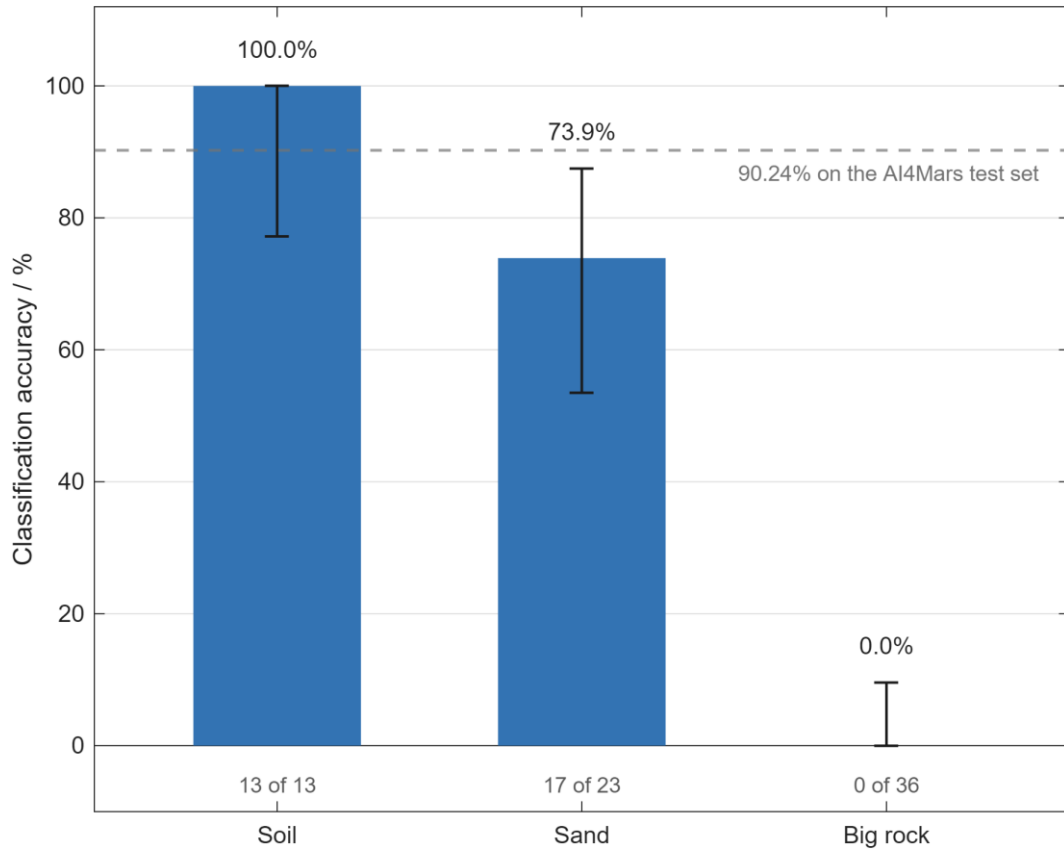


Figure 4.10: Deployed-model accuracy by terrain class and image source. Error bars are Wilson 95% confidence intervals; the dashed line is the same model's 90.24% accuracy on the AI4Mars test set.

4.13 Extended reactive-safety and autonomy results

The behaviours described in Section 3.12 were each verified live in Gazebo, in most cases with a genuine defect found and fixed along the way. All are additive and opt-in, so every prior Gazebo result in this thesis is unaffected. Table 4.3 gives the outcome of each; the two that changed how the system was built are described after it. The debugging paths that reached these results are in Appendix B, and video recordings of eight of the hardware runs are indexed in Appendix A.

Table 4.3: Live Gazebo outcome for each extended behaviour.

Behaviour	Outcome	Verified live
Reactive hazard recovery	Waited out the 3 s dwell, executed a closed-loop 90-degree check turn, and resolved the hazard	Yes
Stuck-in-sand detection	25 s of driving into a solid obstacle produced 3.7 cm of displacement against the 2.5 m an open-loop estimate would report	Yes
Multi-modal fusion	LiDAR-range and IMU-tilt risk terms combined with the terrain score by taking the worst signal, correct across the full input range	Yes
Path planner (L5-lite)	Reached a hazard-crossing goal end to end, 15.13 m travelled over 62 replans, after three defects were fixed	Yes
Waypoint sequencing (L6-lite)	Advanced to a second goal with no operator command and completed a round trip	Yes
Frontier exploration	19 autonomous goal selections, 23.1% of a 6x6 m box covered in 600 s	Yes
Explore-then-return-home	99.2% of a 3x3 m box covered over 45 selections, then an autonomous return home	Yes
Uncertainty-driven re-observation	Revisited the 14 lowest-confidence cells in order, no violations of the invariant across 16 selections	Yes
Mission-level failsafe	Aborted and returned home within 1.3 s of the signal, stopping 3.3 cm from the recorded start pose	Yes
SLAM localisation	0.174 to 0.181 m mean position error, but only when given a motion prior the real rover cannot produce	Simulation only

Two results are worth more than a row. **The path planner's first live attempt made zero net progress in 300 seconds**, and the three defects behind that

turned out to be interactions rather than planner bugs: a SLAM bootstrapping deadlock, an unclamped angular velocity, and a stale-pose deadlock during continuous pure rotation. The last needed a hybrid rotate-and-creep manoeuvre so the scan-matcher kept receiving translation. None would have appeared in a unit test of the planner itself.

The SLAM result is the one that does not transfer. It reaches 0.174 to 0.181 m mean error only when given a ground-truth motion prior, and ExoMy has no wheel encoders and no gyro, so it cannot supply one. Odometry-free localisation stays open, which is why Table 1.1 records Layer 4 as simulation-only rather than delivered.

4.14 Chapter summary

Across ten experiment groups and 22 evaluated model variants, DINO self-supervised pretraining is the clear winner for Mars geological texture. The best result, Ensemble B (94.43%), sits 2.24 points below the supervised ceiling, and the single-encoder DINOv2 ViT-L (93.73%, statistically equivalent to the ensemble) is the recommended configuration where compute allows it. The deployed Raspberry Pi model, DINOv2+reg ViT-S/14, reaches 90.24% on AI4Mars, runs at 570 ms per image (1.75 FPS, 258 MB RAM) on real Raspberry Pi 4 hardware, and is well calibrated after temperature scaling (ECE 0.0325).

The domain gap has a different shape at each step away from AI4Mars. In Gazebo the model reaches 60% terrain accuracy at 100% safety compliance, a 30.24-point drop traced to rock geometry and texture tiling rather than to any model deficiency. On the real camera the gap is confined almost entirely to one class: real soil is classified perfectly (13 of 13) and real sand reaches 73.9% (17 of 23), while big rock fails in all 36 cases. That failure was independently reproduced across 214 real camera frames and 12 live autonomous runs, and it also breaks the Gazebo safety guarantee for that class, since big rock reads as CAUTION rather than STOP. Cross-sensor transfer (NAVCAM to MastCam, - 65.30 points) and cross-planet transfer (Mars to Earth, near or below chance)

both fail without in-domain data, though a jointly trained probe recovers near-single-domain accuracy on each side at under a 4-point combined cost.

Beyond perception, the extended autonomy stack demonstrates working reactive hazard recovery, stuck detection, multi-sensor fusion, a lightweight path planner that reaches a real hazard-crossing goal, autonomous waypoint sequencing and area coverage, a complete unattended explore-and-return mission, and uncertainty-directed re-observation, all built test-first and verified live in Gazebo. Odom-free localisation, the case real hardware needs most, remains a genuinely unresolved and thoroughly investigated limitation.

5 Discussion

5.1 Introduction

This chapter reads the results in Chapter 4 against the thesis's research question: can pretrained foundation models classify Mars terrain accurately for a low-cost rover, with no Mars-specific training, and which training approach gives the best frozen terrain features? It covers why one pretraining paradigm wins, how the domain gap changes shape as the test data moves further from the training set, what the hardware phase added, and what remains open.

5.2 Why DINO self-supervision wins

CLIP's zero-shot baseline confirms the thesis's starting premise, that a model with no exposure to Mars imagery still carries genuine, above-chance terrain knowledge. The prompt-engineering study then exposes a structural ceiling on it. Soil and bedrock accuracy trade off against each other in CLIP's embedding space, because "bedrock" and "big rock" share the word "rock" and pull towards the same region however the prompt is phrased. MobileCLIP S0 shows the identical trade-off under the same prompts, so this is a property of contrastive image-text pretraining rather than one architecture's quirk, and no amount of prompt engineering fixes it. It needs a different pretraining objective.

DINOv2 and DINOv3 supply that objective. Both learn purely from visual consistency across augmented views of the same image, with no text at all, so nothing in their training ties the word "rock" to any visual category. At 10 shots, DINOv2 reaches 56.5% bedrock accuracy against CLIP's 15.2%, a 41-point gap appearing before any scale advantage could account for it, so the difference is architectural rather than a data-volume effect. The 22-model survey makes this precise: every DINO variant clears 89.9%, every non-DINO model tops out at 87.8%, and the gap widens sharply on bedrock.

Park et al. (2023) give a plausible mechanism: DINOv2 produces shape-biased rather than texture-biased representations, and shape bias improves cross-domain transfer under photometric shift. Mars terrain differs from ImageNet mostly in illumination, dust, and sensor response while the underlying rock and

soil geometry stays constant, which is exactly that condition. Independent support exists outside this thesis. Docherty et al. (2025) applied the identical DINOv2+reg ViT-S/14 model to dense AI4Mars segmentation and reached 0.797 mIoU, more than double a classical baseline, and Mattamala et al. (2025) show frozen DINOv2 features supporting online traversability adaptation on a real outdoor robot.

Figure 5.1 shows what this looks like in feature space. A t-SNE projection of DINOv2 ViT-L's test-set features puts each class in territory of its own, though that territory is fragmented into several groups rather than one blob, as expected when the projection organises scenes by overall appearance and a class is photographed under varied lighting and distance. What matters is that those groups are class-pure over most of the plot, which is why a linear probe on these features works at all. The 18 misclassified images then fall almost entirely where two classes' territories touch, or inside the small mixed groups, while the interior of every large pure group is error-free. That matches the confusion matrix in Chapter 4, where the real error modes run between visually adjacent classes, bedrock read as soil and sand read as bedrock. The errors are a boundary problem rather than a failure of the feature space to separate the classes.

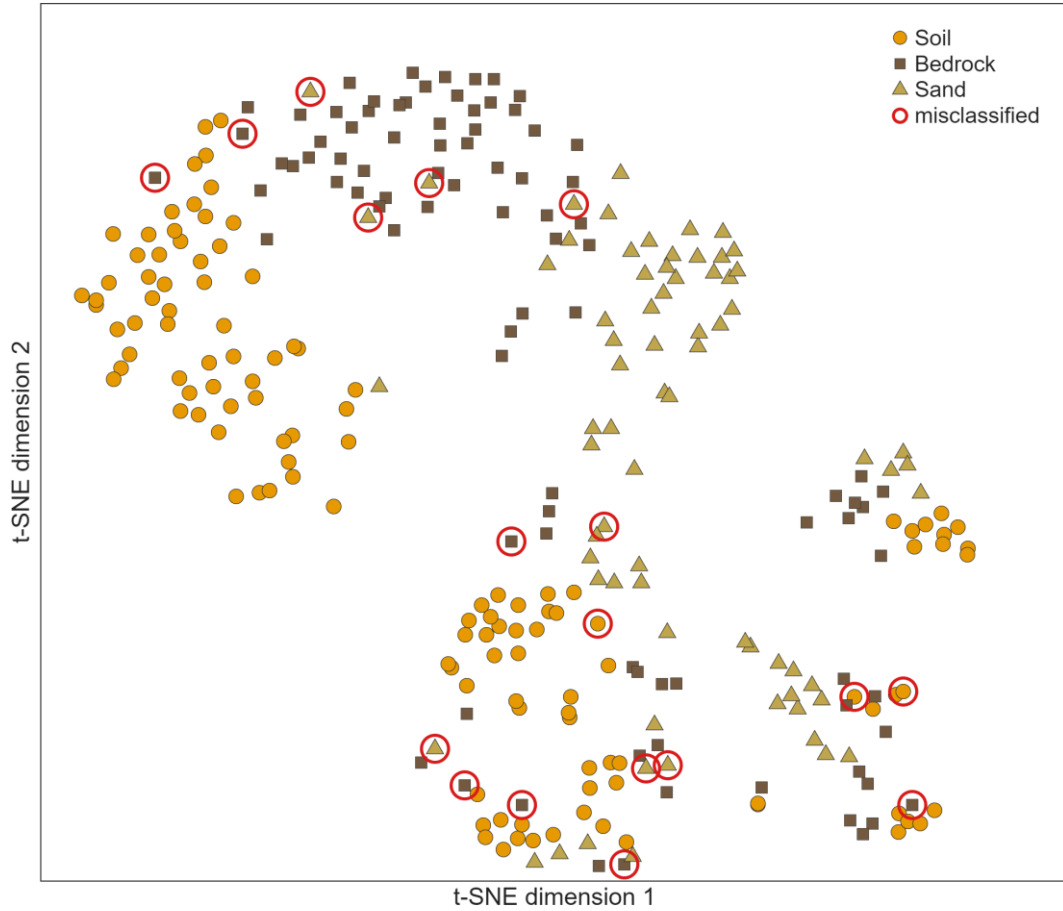


Figure 5.1: t-SNE projection of DINOv2 ViT-L/14's 1024-dimensional CLS features for the 287-image AI4Mars test set, with misclassified images ringed. Distances between clusters in a t-SNE projection carry no quantitative meaning; only the grouping and the overlaps should be read.

Two engineering routes that looked promising on paper both failed. Concatenating CLS tokens from several transformer layers instead of using the final layer alone consistently lowered accuracy, because the final layer already aggregates everything useful and earlier layers add less-processed noise. Monocular depth (Depth Anything V2) added nothing either: depth-only classification reached 36.6%, fusing depth with DINOv3's own features made results slightly worse, and the gradient-based roughness feature meant to separate rock from soil showed essentially zero separability. Depth Anything V2's Earth-trained cues of perspective, shading, and familiar object scale do not exist in Mars NAVCAM close-ups. Both results point the same way: visual appearance

carries the signal here, and squeezing more out of a fixed encoder's architecture does not substitute for a better pretraining objective.

Backbone scale helps within the DINO family, and helps most where it matters most: the ViT-S to ViT-L step gains most on bedrock, and gains most of all where data is scarcest (+11.5 points at 10 shots), which suits a mission that cannot expect abundant expert labels. A cross-check favours data curation over data volume, since DINOv2's smaller but more carefully curated training set beats DINOv3 at every scale from ViT-B upward, and the gap widens as the backbone grows. The two-encoder ensemble reaches the study's best result, but a McNemar's test on the 10 images where the two disagree found the gain over the single ViT-L encoder not significant ($p=0.75$). The recommendation is therefore the single DINOv2 ViT-L encoder wherever offline compute allows, and DINOv2+reg ViT-S/14 on the Raspberry Pi, where speed rules ViT-L out.

5.3 The cascade pipeline and CPU contention

The Gazebo cascade experiment shows the dual-process design working as intended in at least one case. CLIP misclassified a rock-textured bedrock surface as big rock, a false hazard, at 0.806 confidence across 260 frames, and a single SmolVLM query correctly reclassified it, downgrading the decision from STOP to CAUTION. That is the value the slow path is meant to add: language-model reasoning can disambiguate a case pure visual similarity gets wrong. The same experiment quantifies the price. Running both models together slows each by 2.4 to 2.6 times through CPU contention alone, a cost that would grow, not shrink, on the Raspberry Pi 4's weaker cores. CPU scheduling, meaning ONNX acceleration, wider VLM polling intervals, and affinity pinning, is therefore a first-class deployment concern for any single-CPU multi-model pipeline rather than an afterthought.

5.4 Sim-to-real: how the domain gap changes shape

The thesis measures the same underlying domain-transfer problem at three distances from the training data: AI4Mars itself, Gazebo, and the rover's own camera. The gap does not simply widen at each step. It changes shape, from a

uniform drop in simulation to a class-specific collapse on real imagery. Figure 5.2 sets the deployed model's per-class accuracy on AI4Mars against its per-class accuracy on the real camera.

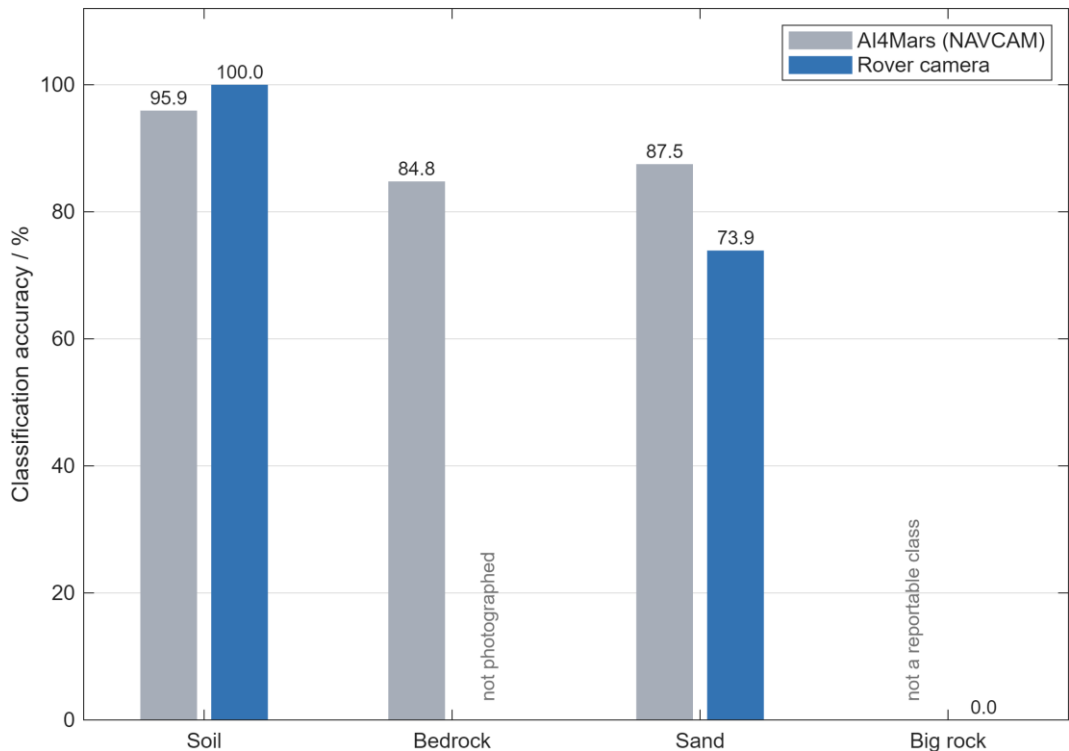


Figure 5.2: Deployed-model accuracy by terrain class on AI4Mars and on the rover's own camera. Big rock has no AI4Mars bar, and zero real-camera accuracy, because it is not in the probe's label set.

In Gazebo, using real AI4Mars texture crops rendered onto simplified geometry, accuracy drops to 60%. Two mechanisms account for almost all of it: the big-rock class's own 108-image training set, which is a property of the dataset rather than the simulator, and a geometry-classification conflict where photorealistic rock meshes read as sand while unrealistic sphere primitives correctly read as uncertain and trigger a safe STOP. This is a case where more visual realism made the system less safe, because DINOv2's global image embedding responds to the angular flat faces of a mesh much as it responds to granular sand texture.

On the real camera the gap becomes class-specific. Soil transfers intact, sand falls to 73.9%, and big rock fails entirely, while the same model still reaches

90.24% on AI4Mars, which confirms the probe itself is sound. The mechanism changes too. This is not a colour gap, since AI4Mars is grayscale and there is no hue to lose, but a brightness and sensor-response gap between the laboratory's white silica sand and NAVCAM's grey tones, compounded by the camera's RGB channels sitting outside the training distribution's channel statistics. That explains the pattern precisely, because it degrades the class defined by overall brightness while leaving the classes defined by texture intact. The safety consequence is equally specific: big-rock images average 0.50 confidence, just above the 0.40 STOP threshold, so the model reads them as soil and commands CAUTION rather than STOP.

This decomposition matters for where future effort should go. The Gazebo-stage gap is largely a simulation-fidelity and dataset-imbalance problem, while the hardware-stage gap is a sensor-domain problem simulation improvements cannot fix, needing real labelled rover imagery or targeted domain randomisation. A grayscale-conversion fix applied at test time recovered 5 points on sand by addressing the luminance component, but left big rock's separate background-contrast problem untouched. Augmenting the training set with Gazebo's own rock-zone features made things categorically worse, with every zone then reading as big rock, because every Gazebo material sits in a similar region of DINOv2's feature space (cosine similarity 0.60 to 0.77 to the rock centroid) through shared synthetic lighting and texture resolution. That rules out naive same-domain augmentation and confirms a genuine feature-space problem rather than a labelling one.

The same pattern recurs across sensors and across planets, and is never solved for free. Zero-shot transfer to Mars-Bench's MastCam colour imagery collapses, and a fresh probe trained on Mars-Bench recovers most of it. A pre-registered cross-planet probe has the same shape: transfer fails in both directions, while a jointly trained probe recovers to within a few points of each single-domain result. The consistent finding across all three tests is that DINOv2's frozen features are domain-separable but not domain-shared. One encoder can support strong

classifiers on several domains at once, but only if each domain contributes its own labelled data.

5.5 Comparison with related work

This thesis sits alongside a small number of prior attempts to bring foundation models to planetary rover perception. Space-LLaVA (Foutter et al., 2024) fine-tunes a 7-billion-parameter LLaVA model on Mars imagery for scene captioning, achieving richer qualitative descriptions than any model evaluated here, but requiring GPU inference this thesis's edge-hardware scope excludes. Abaza et al. (2026) is the closest hardware analogue, demonstrating a 1.5-billion-parameter VLM on a Raspberry Pi 5 for indoor navigation; this thesis applies a comparable edge-VLM approach to outdoor Mars terrain with full ROS2 integration. Against the AI4Mars supervised baseline (Swan et al., 2021, 96.67% with 16,064 labelled images and GPU training), the best frozen-encoder result here reaches within 2 to 3 points using 4,000 labels and no GPU training at all. Chiuchiarelli et al. (2024) build an architecturally similar ROS2 traversability pipeline in Unity, but train their segmentation CNN end-to-end rather than using frozen features; this thesis reaches a comparable safety outcome in Gazebo with zero task-specific gradient updates.

The clearest novel contributions are the systematic prompt-engineering study for Mars terrain, the first full 287-image VLM evaluation on AI4Mars, the first ROS2-integrated dual-process cascade tested end-to-end in Gazebo, a quantified CPU-contention cost for concurrent edge inference, and the first quantification of the NAVCAM-to-MastCam cross-sensor domain gap for DINOv2 features, later extended to a cross-planet Mars-to-Earth generalisation probe for the same encoder.

5.6 Limitations

Several limitations bound how far these results should be read. The AI4Mars gold-standard test set has no big-rock-dominant images at all, and the class was excluded from the probe's label set for having too few training examples, so the class most safety-critical to get right is never scored directly and is reachable only

through the uncertain-to-STOP path. A supplementary experiment was run to establish which of those two things is the binding one, because the headline 0% cannot distinguish them. Using the same frozen encoder, three probes were trained on identical features and scored on the same held-out images, with only the label set and the class balance changed. The held-out set is drawn from the crowd-sourced training labels rather than the gold-standard test set, since no gold test image has big rock as its dominant class, so these accuracies are lower across the board than the 90.24% headline and only the comparison between the three is meaningful.

Table 5.1: Effect of adding big rock to the label set. Same encoder, same features, same 120 held-out images.

Probe	Overall	Soil	Bedrock	Sand	Big rock
Three classes, 78 images each (deployed label set)	78.9%	93.3%	66.7%	76.7%	n/a
Four classes, 78 images each	70.0%	93.3%	43.3%	73.3%	70.0%
Four classes, all available (270/270/270/78)	61.7%	90.0%	73.3%	83.3%	0.0%

The result is clear enough to change how the 0% should be read. Given a balanced training set, the frozen encoder separates big rock at 70.0%, comparable to sand and better than bedrock in the same probe. DINOv2's features are therefore not blind to big rock, and the feature space is not the problem. Class imbalance is, and it is catastrophic rather than gradual: at a ratio of only 3.5 to 1 against, the class collapses from 70.0% to exactly zero, with all 30 held-out big-rock images going to soil, bedrock, or sand and not one staying. There is no graceful degradation between those two conditions.

The cost of adding the class is also visible, and it falls where the t-SNE projection predicted. Bedrock drops from 66.7% to 43.3% in the balanced four-class probe, because big rock competes for the same region of feature space as the class it is already most often confused with, and overall accuracy falls by 8.9 points. That

is the trade the deployed three-class configuration takes, and on this evidence it remains the right one for a system whose safety rests on the confidence threshold rather than on naming the hazard. But the reason for the 0% is now specific: not that the encoder cannot see big rock, but that 108 training images against roughly 8,000 for bedrock is too few for a probe to keep the class alive at all. That points the remaining work at data collection for one class, which is a far smaller task than replacing the encoder.

The 20-image BLIP-2 evaluation is preliminary and soil-dominated; a full 287-image run would take roughly 60 hours at this model's speed and was not attempted. SmolVLM-256M's 21.6% parse-failure rate is a real reliability gap, and the description-style prompt that removes it trades it for a soil-accuracy collapse.

Real camera testing surfaced a more fundamental gap. On 214 screened frames from the rover's own camera, spanning walls, boxes, and a hand held near the lens as well as real terrain, the model's `uncertain` output fired zero times, and it read a wall as soil at 0.937 confidence. A nearest-neighbour distance check against the AI4Mars training features shows why. Separating terrain from non-terrain this way reaches an area under the curve of only 0.809, and any threshold strict enough to reject a wall also rejects over half of genuine terrain frames, including every frame of two situations a rover most needs to detect, an approaching pit edge and an open horizon. The encoder finds the rover's own terrain almost as unfamiliar as it finds a wall, because real frames run roughly three times brighter than AI4Mars with four to eight times less relative texture under the laboratory's shadow-free lighting. This is reported as a measured limitation rather than patched, since no cheap threshold fix survives that result.

One further limitation belongs to the actuators rather than to the perception stack, and it bounds what the traversability policy can be said to demonstrate. The controller grades speed in four tiers, and that grading is what makes it a traversability controller rather than a stop switch. On real ExoMy the grading does not exist. Measured on a 3 m flat course, commanded values of 20, 25, and 50 all produced 0.235 to 0.245 m/s, because the servos saturate far below the top of the commanded range. The tiers are therefore a Gazebo result throughout.

The safety-relevant half of the policy does transfer, since stopping is unambiguous and the rover stops, but the claim that slower driving over bedrock reduces risk on the real platform is untested and cannot be tested on this hardware. Cheng et al. (2025) report that graded policies of this kind cut unnecessary stops by 34% against a binary rule, and that benefit is one this rover cannot realise as built.

The camera-only pipeline is also structurally blind to two classes of hazard no amount of better classification can recover: three-dimensional geometry, meaning slope, step height, and rocks not visible from the current angle, and mechanical terrain properties, meaning ground that is loose or deformable but looks stable in RGB. These are hardware constraints rather than model failures, and they motivate the LiDAR and IMU fusion work.

Adding the LiDAR closes much of that geometric gap but does not close all of it, and real-hardware testing separated the residual into two distinct mechanisms that should not be read as one.

The first is tilt-induced. Once chassis pitch exceeds roughly 20 to 30 degrees on a slope, the forward-clearance reading becomes unreliable and oscillates, because the scan plane tilts with the chassis and stops intersecting the geometry it intersected when level. This was first seen in simulation, where the Gazebo rock field sat below the simulated scan height and the fused LiDAR risk term never rose at all (Table D.1), and it was then reproduced on real hardware in the sand and Mars-backdrop runs. It also works in the opposite direction: the rover drove into a box while approaching it over a slight rise, because the rise pitched the scan plane up and over the box's actual height.

The second mechanism needs no tilt at all. On a flat laboratory floor the rover collided with a chair whose splayed legs sat entirely below the scan height. That is not a calibration or threshold problem but the structural consequence of a single scan plane at one fixed mount height: any obstacle whose relevant profile lies outside that plane is invisible to the stack whatever the chassis is doing. Both collisions were light, at roughly 0.05 to 0.10 metres per second, and neither was a near miss. Thin and low obstacles are therefore a genuine geometric gap in the

rover's safety envelope that no software change to the present perception stack can address. Closing it needs a multi-plane or three-dimensional LiDAR, or a stereo depth sensor, which is beyond this project's hardware budget and is picked up in Section 6.4.

5.7 What the hardware phase added

Every real-hardware integration step surfaced at least one problem invisible to Gazebo alone, consistently enough to state as a general lesson: "validated in Gazebo" and "ready for real hardware" are independent claims, and a passing result at one layer carries no information about the other. The IMU bring-up chain uncovered a floating address-select pin and a burnt I2C bus, both found only by direct measurement rather than more careful code review. Building the first unified real-hardware launch file found that no message bridge existed at all between the ROS2 `Twist` commands every Gazebo result used and the legacy `RoverCommand` message the real motor driver accepts, meaning every Gazebo-validated safety node had no way to drive the real rover until a bridge was written. The same exercise found a type mismatch that had silently prevented the driver from ever publishing a valid motor command, invisible to any Gazebo run because Gazebo never touches that code path.

The IMU slope-retreat behaviour, retired for the reasons given in Section 3.12, is the clearest case of hardware overturning a design simulation had no way to question. What matters here is why simulation could not have caught it. Both of the findings that retired it, that a blind reverse is this rover's most dangerous manoeuvre and that an accelerometer-only tilt estimate reports tilts that are not real, are properties of a physical chassis with no rear-facing sensor and a sensor that cannot separate gravity from its own acceleration. Neither has an analogue in Gazebo, where reversing is free and simulated tilt is exact. The behaviour was not badly implemented; it was correct against a model of the world that the world did not share.

Organising every non-perception failure by the shallowest test layer that could have caught it produces a transferable ladder. Unit tests catch pure-function bugs. Type mismatches at component boundaries need two components actually

wired together. Control-rate interaction bugs need a live smoke test. Slow or stochastic bugs need sustained, adversarial, or repeated runs. Message-bridge and missing-capability bugs need the real hardware, because a simulated drive plugin bypasses the real motor code. Perception domain-gap failures sit outside the ladder: they are visible at the first evaluation layer and hide behind no depth of integration, so they need evaluation on the genuine target distribution rather than more integration testing.

The Raspberry Pi INT8 quantisation result needed the same discipline. A first accuracy check, using 200 of the 287 gold images in file-sorted order and a squashed rather than centre-cropped resize, found a reassuringly small 0.50-point drop. Re-measured on all 287 images with the deployment's actual preprocessing, the real drop is 3.83 points, and it concentrates entirely in the soil class. Latency tells a similar story: 386 ms for the encoder in isolation on the Pi, against 1,865 to 2,200 ms on the rover with the full stack competing for the same four cores. That five-fold gap is itself the finding, and the reason the controller's staleness timeout had to move from 3 to 7 seconds.

Odom-free localisation is the clearest example of a thoroughly investigated open limitation rather than an unattempted one. Nine configurations across five diagnostic rounds tested every motion-prior strategy available without wheel encoders, ruled out real-time-factor throttling and message-queue capacity by direct measurement, and produced exactly one non-zero pose result, at 3.5 to 4.4 m error against the 0.18 m the odom-assisted configuration reaches. A related problem was fully resolved along the way: Gazebo's differential-drive plugin was actuating only two of six wheels, a genuine sim-to-real gap since real ExoMy drives all six, and enabling all three pairs raised achieved rotation from under 1% to 70-78% of target.

5.8 Answering the research question

Can pretrained foundation models classify Mars terrain accurately for a low-cost rover with no Mars-specific training, and which training approach gives the best frozen features?

The answer to the second half is clear and reproducible. DINO self-supervised pretraining, rather than contrastive, autoregressive, multi-teacher, or task-specific pretraining, produces the most Mars-discriminative frozen features, closing 97.7% of the gap between CLIP zero-shot and the supervised ceiling with no gradient updates to the encoder at all.

The answer to the first half is yes on the benchmark and qualified in the field. The deployed configuration is accurate on AI4Mars, fast enough on real Raspberry Pi 4 hardware, and well calibrated after temperature scaling, and it achieves full safety compliance in a controlled Gazebo benchmark with every failure mode traced to an explained mechanism. On the rover's own camera the picture is more specific than a blanket failure: soil survives the transfer intact, sand degrades, and big rock fails outright, taking the Gazebo safety guarantee with it for that class alone. That remains the clearest evidence in this thesis that frozen features are not sufficient on their own for field deployment, but it points at a targeted fix, which is collecting real rover imagery for the weakest class and restoring it to the label set, rather than at retraining the whole system.

Beyond perception, the reactive-safety and autonomy stack built on top of the classifier, verified live in Gazebo and confirmed for the reactive-safety layer across 10 of 10 real-hardware rounds, shows that terrain perception can support genuinely useful downstream behaviour. Odom-free localisation, the case real hardware most needs, remains an open problem, reported as such rather than glossed over.

5.9 Future work

Three items sit within reach of this thesis's own scope and were not pursued: extending the AI4Mars evaluation to a balanced big-rock test set so the safety-critical class can be scored directly and returned to the label set; retry logic for SmolVLM's parse failures; and CLIP encoder fine-tuning on the full training set, to test how much of the remaining few-shot gap a small amount of GPU compute could close.

Beyond that scope, three directions stand out. Stereo depth from ExoMy's camera pair, rather than the monocular depth estimation this thesis showed fails outright, is the correct way to add real geometric hazard information to an appearance-only pipeline. Cross-sensor and cross-planet data collection follows directly from the joint-probe results: a single frozen encoder can serve several domains, so the practical step is collecting a modest amount of labelled data from each rather than expecting free generalisation. And the reactive layer's out-of-distribution blindness needs a real solution before any deployment outside a test rig, since the measured 0.809 AUC rules out a simple confidence-threshold fix.

For the wider six-layer autonomy stack, this thesis delivers L1 (terrain semantics) and L2 (reactive safety) fully, with L2 hardware-validated across 10 confirmed rounds, and L3 (continuous traversability and multi-sensor fusion) substantially. L4 (localisation) is partial, working only given a motion prior real hardware cannot supply, and L5 (path planning) was demonstrated end-to-end in Gazebo but depends on that same gap for real-hardware use. L6 (mission autonomy) has several deliberately small, honestly scoped slivers working live in Gazebo, none claiming to be goal decomposition or resource identification. Only the science-target form of L6 is blocked by missing hardware, since ExoMy's single RGB camera has no path to multi-spectral sensing. Every other gap is an engineering-time question, and the layered architecture means each direction can be picked up independently.

5.10 A note on simulation and hardware versions

The L5-lite path-planning and frontier-exploration results were produced on 2026-07-17/18, using an earlier version of the reactive-avoidance logic (`reactive_explorer_node.py` and `avoidance_planner.py`) than the one later validated across the 10 real-hardware rounds, which was substantially redesigned and twice further revised during the hardware phase. This thesis treats simulation and hardware as sequential development phases rather than one continuous validation claim. The Gazebo terrain-classification results are unaffected, since they predate the avoidance logic entirely, and the later fixes changed how the rover avoids obstacles reactively rather than the planner or frontier logic layered

on top, so no specific finding is invalidated, only its currency as a snapshot of the present code.

6 Conclusion

6.1 Summary of work

This thesis set out to test whether pretrained foundation models can classify Mars terrain accurately for a low-cost rover, with no Mars-specific training, and to find which training approach gives the best frozen terrain features. Ten experiment groups, spanning 22 model variants across six pretraining paradigms, were run on the AI4Mars gold-standard test set (287 images) against a supervised DeepLabv3+ ceiling of 96.67%.

CLIP zero-shot reached 34.8% overall accuracy with no labelled Mars data at all, confirming that pretrained visual knowledge transfers to Mars terrain above chance. Prompt engineering raised this to 54.4%, but exposed a soil-bedrock trade-off intrinsic to CLIP's contrastive embedding space that no amount of further prompting could resolve. A few-shot linear probe on frozen CLIP features reached 87.8% at 1,000 labels per class. The 22-model survey then established that DINO self-supervised pretraining, not contrastive, autoregressive, multi-teacher, or task-specific objectives, is uniquely suited to this task. Every DINO Vision Transformer variant reached at least 89.9% and the eleven highest-ranked variants were all DINO or DINO ensembles, while the best competing paradigm reached 89.55% and contrastive and autoregressive pretraining stopped at 87.8%. The gap is concentrated in the bedrock class. The best result, an ensemble of two DINOv2 scales, reached 94.43%, and the single-encoder DINOv2 ViT-L (93.73%), statistically equivalent to the ensemble, is the recommended configuration wherever compute allows it.

The model actually deployed on Raspberry Pi 4 hardware, DINOv2+reg ViT-S/14, reaches 90.24% on AI4Mars, runs at 570 ms per image (258 MB RAM) on the real device, and is well calibrated after temperature scaling (Expected Calibration Error 0.0325). In a controlled Gazebo benchmark it reaches 60% terrain accuracy and 100% safety compliance, with every failure traced to a specific, explained cause. On the real camera, soil transfers intact and sand transfers with degradation, while big rock fails in every one of 36 cases, taking

the Gazebo safety guarantee with it for that class. This is the clearest evidence in this thesis that frozen features alone are not sufficient for field deployment, and it identifies where the remaining work belongs.

An extended reactive-safety and autonomy stack was built on top of this perception system, covering hazard recovery, stuck detection, multi-sensor fusion, a lightweight path planner, autonomous area coverage, and a complete unattended explore-and-return mission. It was verified live in Gazebo throughout, and the reactive-safety layer specifically was confirmed across 10 of 10 real-hardware test rounds.

6.2 Answering the research question

Can pretrained foundation models classify Mars terrain accurately for a low-cost rover, with no Mars-specific training, and which training approach gives the best frozen features? Yes, and the answer to the second half is clear and reproducible: DINO self-supervised pretraining. It closes 97.7% of the gap between CLIP zero-shot and the supervised ceiling with no gradient updates to the encoder at all, and it does so specifically because self-distillation builds view-invariant texture features, exactly what distinguishes Martian bedrock from soil, while every language-supervised or task-specific alternative tested here inherits a semantic geometry that conflates them.

Three qualifications temper this positive answer:

- The domain gap changes its underlying failure mechanism at each step, from dataset imbalance and rendered geometry in Gazebo to sensor and brightness response on the real camera, so a result validated only in simulation should never be read as a deployment guarantee.
- SmolVLM-256M is the only edge vision-language model that fits the Raspberry Pi's memory budget, but its 21.6% response-parsing failure rate remains an unresolved reliability gap.
- Odom-free localisation, the one capability real ExoMy hardware most needs in order to move beyond reactive behaviour, remains a genuinely

open problem after nine configurations across five investigation rounds, not for lack of trying.

6.3 Contributions

This section expands the contributions summarised in Chapter 1 into their nine constituent technical results, each confirmed by the results in Chapters 4 and 5.

1. A twelve-variant prompt engineering study for Mars terrain with CLIP, quantifying a soil-bedrock trade-off intrinsic to contrastive image-text pretraining and raising zero-shot accuracy from 34.8% to 54.4%.
2. A comparison of 22 foundation model variants across six pretraining paradigms, the most complete frozen-encoder survey on AI4Mars to date, establishing DINO self-distillation on a Vision Transformer as the most effective paradigm for Mars geological texture.
3. The best frozen-encoder accuracy on AI4Mars (94.43%, within 2.24 points of supervised), reached with no GPU fine-tuning and no Mars-specific training, plus the statistically grounded finding (McNemar's test) that the practically simpler single-encoder DINOv2 ViT-L (93.73%) performs equivalently.
4. The first full 287-image VLM evaluation on AI4Mars, for SmolVLM-256M, including the first VLM result on the sand class and a prompt ablation identifying the domain-grounding role of the word "Mars".
5. An open-source ROS2 perception package, `fm_perception`, wrapping CLIP, DINOv2, SmolVLM-256M, and BLIP-2 as ROS2 nodes with safety watchdogs and confidence-threshold routing, tested end to end in Gazebo.
6. A dual-process cascade pipeline validated in Gazebo, demonstrating that a slow VLM path can semantically correct a fast path's misclassification, with the 2.4-2.6x CPU-contention cost of running both models concurrently measured directly.
7. A Raspberry Pi 4 deployment benchmark: 570 ms per image, 258 MB RAM for the deployed ONNX model, with an 80.8% reduction in calibration error from temperature scaling.

8. An extension of the perception system into a working reactive-safety and autonomy stack, each behaviour scoped before building, tested first, and reported with its real limits. Verified live in Gazebo, and for the reactive-safety layer confirmed across 10 of 10 real-hardware rounds, the stack comprises:
 - hazard recovery from a terminal stop
 - stuck-in-sand detection with staged escalation
 - continuous multi-sensor traversability fusion across camera, LiDAR, and IMU
 - a lightweight A* planner that reaches a real hazard-crossing goal
 - autonomous waypoint sequencing with no operator command between legs
 - bounded-area frontier coverage with an optional science-interest bias
 - a complete unattended explore-and-return mission
 - uncertainty-directed re-observation of the least-confident known cell
9. A pre-registered cross-planet generalisation probe with an honest negative result: the deployed frozen encoder separates Mars and Earth terrain cleanly in feature space but does not transfer between them without in-domain data (13.45% and 35.74% cross-planet accuracy, both failing the pre-registered positive bar), extending the thesis's own within-Mars, cross-sensor domain-gap finding (65.30 points, NAVCAM to MastCam) to a cross-planet setting.

Two qualifications belong with the eighth contribution. Two systematic navigation-layer limitations found during hardware confirmation are accepted as design limitations of a purely reactive, map-free avoidance strategy. And a ninth behaviour, IMU slope-triggered retreat, was built and unit-tested but never run live, then withdrawn during the hardware phase once a blind reverse proved to be the most dangerous manoeuvre available to a rover with no rear-facing sensing (Section 5.7).

6.4 What remains for future work

Three directions sit closest to this thesis's own scope: a balanced big-rock test set, so the safety-critical class can be scored directly and restored to the label set; retry logic for SmoIVLM's parse failures; and encoder fine-tuning, to test how much of the remaining few-shot gap a modest amount of GPU compute could close. Beyond that, the clearest next steps are stereo depth or a multi-plane LiDAR for genuine geometric hazard sensing, since monocular depth was shown here to fail outright and a single fixed-height scan plane was shown to miss obstacles that sit below it (Section 5.6); collecting a modest amount of labelled data from each new sensor or domain rather than expecting the frozen encoder to generalise for free; and a real fix for the reactive layer's measured blindness to out-of-distribution surfaces, since a simple confidence threshold was shown not to catch it. A rear-facing sensor would also reopen the reverse manoeuvre the hardware phase had to withdraw.

One further direction is methodological rather than hardware-bound. Every safety rule in this thesis, the confidence thresholds and the state machines alike, is hand-written and unit-tested against its own decision functions. That is enough to show the logic behaves as specified, but not to show that the specification itself is complete, and the review that found two mission-ending transitions missing from this thesis's own state-machine diagram is a small illustration of the difference. Perez et al. (2022) compile a safety requirement written in structured natural language into a runtime monitor (Pike et al., 2010) that rides alongside the live ROS2 topic stream and publishes a verdict, on a ground robot structurally similar to this one. Applying that to the traversability gate and the reactive explorer would put the safety envelope in one auditable place rather than distributed across node code and its tests (Section 2.3.4).

Chapter 5 sets the remaining gaps out in full against the six-layer autonomy framing. This thesis delivers Layer 1 and Layer 2 fully, Layer 3 substantially, and Layer 4 partially, with the remaining Layer 4 gap now genuinely investigated rather than merely deferred. Layers 5 and 6 have working, honestly scoped slivers demonstrated live in Gazebo. Only the resource-identification half of Layer

6 is blocked by missing hardware. Everything else that remains is an engineering-time question.

6.5 Closing statement

Foundation models offer a genuinely practical route to Mars terrain perception on low-cost rover hardware: no GPU training, no large domain-specific dataset, and a reproducible answer to which pretraining objective works best. The gap from zero-shot (34.8%) to a supervised specialist (96.67%) is real, but it closes steadily as labels and a better pretraining paradigm are applied, reaching 94.43% with 4,000 labels and no encoder fine-tuning at all. The other half of the account matters as much. A Gazebo safety guarantee did not survive contact with a real camera, a vision-language model could not be trusted to parse its own answers, and localisation remains a genuine open question rather than an unattempted one. A team building on this work needs both halves.

References

- Abaza, B. F., Staicu, A., & Doicin, C. V. (2026). A Semantic Autonomy Framework for VLM-Integrated Indoor Mobile Robots: Hybrid Deterministic Reasoning and Cross-Robot Adaptive Memory. *arXiv*. <https://doi.org/10.48550/arXiv.2605.02525>
- Allan, M., Wong, U., Furlong, P. M., Rogg, A., McMichael, S., Welsh, T., Chen, I., Peters, S., Gerkey, B., Quigley, M., Shirley, M., Deans, M., Cannon, H., & Fong, T. (2019). Planetary Rover Simulation for Lunar Exploration Missions. *2019 IEEE Aerospace Conference*. <https://doi.org/10.1109/AERO.2019.8741780>
- Arafin, T., Hosen, A., Najdovski, Z., Wei, L., Rokonzaman, M., & Johnstone, M. (2025). Advances and Trends in Terrain Classification Methods for Off-Road Perception. *Journal of Field Robotics*, 42(7), 3515-3544. <https://doi.org/10.1002/rob.22586>
- Awais, M., Naseer, M., Khan, S., Anwer, R. M., Cholakkal, H., Shah, M., Yang, M., & Khan, F. S. (2025). Foundation Models Defining a New Era in Vision: A Survey and Outlook. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(4), 2245-2264. <https://doi.org/10.1109/TPAMI.2024.3506283>
- Bajracharya, M., Maimone, M. W., & Helmick, D. (2008). Autonomy for Mars Rovers: Past, Present, and Future. *Computer*, 41(12), 44-50. <https://doi.org/10.1109/MC.2008.479>
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., & Arora, S. (2021). On the Opportunities and Risks of Foundation Models. *arXiv*. <https://doi.org/10.48550/arXiv.2108.07258>
- Cheng, S., Zhou, R., Feng, W., Yang, H., Gao, H., Deng, Z., & Ding, L. (2025). VLM-Empowered Multi-Mode System for Efficient and Safe Planetary Navigation. *arXiv*. <https://doi.org/10.48550/arXiv.2506.16703>

- Chiuchiarelli, A., Franchini, G., Messina, F., & Chiaberge, M. (2024). Towards Safer Planetary Exploration: A Hybrid Architecture for Terrain Traversability Analysis in Mars Rovers. *IAF Space Exploration Symposium*. <https://doi.org/10.52202/078357-0158>
- Daftry, S., Abcouwer, N., Sesto, T. D., Venkatraman, S., Song, J., Igel, L., Byon, A., Rosolia, U., Yue, Y., & Ono, M. (2022). MLNav: Learning to Safely Navigate on Martian Terrains. *arXiv*. <https://doi.org/10.48550/arXiv.2203.04563>
- Darcet, T., Oquab, M., Mairal, J., & Bojanowski, P. (2024). Vision Transformers Need Registers. *arXiv*. <https://doi.org/10.48550/arXiv.2309.16588>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171-4186. <https://doi.org/10.18653/v1/N19-1423>
- Docherty, R., Vamvakeros, A., & Cooper, S. J. (2025). Upsampling DINOv2 features for unsupervised vision tasks and weakly supervised materials segmentation. *arXiv*. <https://doi.org/10.48550/arXiv.2410.19836>
- Dominguez-Vidal, J. E. (2026). A ROS 2 wrapper for Florence-2: Multi-mode local vision-language inference for robotic systems. *arXiv*. <https://doi.org/10.48550/arXiv.2604.01179>
- Endo, M., Taniai, T., & Ishigami, G. (2024). Deep Probabilistic Traversability with Test-time Adaptation for Uncertainty-aware Planetary Rover Navigation. *arXiv*. <https://doi.org/10.48550/arXiv.2409.00641>
- Fong, T. (2024). Digital proving ground: VIPER rover simulator (RSIM) [Technical report]. NASA Ames Research Center. <https://ntrs.nasa.gov/citations/20240014021>
- Foutter, M., Gammelli, D., Kruger, J., Foss, E., Bhoj, P., Guffanti, T., D'Amico, S., & Pavone, M. (2024). Space-LLaVA: a Vision-Language Model

- Adapted to Extraterrestrial Applications. *arXiv*.
<https://doi.org/10.48550/arXiv.2409.15207>
- Goh, E., Chen, J., & Wilson, B. (2022). Mars Terrain Segmentation with Less Labels. *arXiv*. <https://doi.org/10.48550/arXiv.2202.00791>
- Helmick, D., Angelova, A., & Matthies, L. (2009). Terrain Adaptive Navigation for planetary rovers. *Journal of Field Robotics*, 26(4), 391-410.
<https://doi.org/10.1002/rob.20292>
- Holden, L., Dayoub, F., Candela, A., Harvey, D., & Chin, T. (2026). Vision Foundation Models for Domain Generalisable Cross-View Localisation in Planetary Ground-Aerial Robotic Teams. *arXiv*.
<https://doi.org/10.48550/ARXIV.2601.09107>
- Jang, D., Cho, D., Lee, W., Ryu, S., Jeong, B., Hong, M., Jung, M., Kim, M., Lee, M., Lee, S., & Choi, H. (2024). Unlocking Robotic Autonomy: A Survey on the Applications of Foundation Models. *International Journal of Control, Automation and Systems*, 22(8), 2341-2384.
<https://doi.org/10.1007/s12555-024-0438-7>
- Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus and Giroux.
- Khattak, M. U., Wasim, S. T., Naseer, M., Khan, S., Yang, M., & Khan, F. S. (2023). Self-regulating Prompts: Foundational Model Adaptation without Forgetting. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. <https://doi.org/10.1109/ICCV51070.2023.01394>
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W., Dollár, P., & Girshick, R. (2023). Segment Anything. *arXiv*. <https://doi.org/10.48550/ARXIV.2304.02643>
- Kumar, A., Raghunathan, A., Jones, R., Ma, T., & Liang, P. (2022). Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution. *arXiv*. <https://doi.org/10.48550/arXiv.2202.10054>

- Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*. <https://doi.org/10.48550/arXiv.2301.12597>
- Li, Z., Huang, J., Liu, J., Zhu, F., Zhao, E., Dodds, W., Velingker, N., Alur, R., & Naik, M. (2024). Relational Programming with Foundational Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(9), 10635-10644. <https://doi.org/10.1609/aaai.v38i9.28934>
- Lin, K., Agia, C., Migimatsu, T., Pavone, M., & Bohg, J. (2023). Text2Motion: From Natural Language Instructions to Feasible Plans. *Autonomous Robots*, 47(8), 1345-1365. <https://doi.org/10.1007/s10514-023-10131-7>
- Luca, A. D., Muratore, L., Tsagarakis, N., & Azkarate, M. (2025). Efficient and Adaptive Autonomous Guidance and Control of Planetary Rover with Improved Traction Controller and Dynamic Cost Map. *arXiv*. <https://doi.org/10.22541/au.173736202.26989883/v1>
- Marafioti, A., Zohar, O., Farré, M., Noyan, M., Bakouch, E., Cuenca, P., Zakka, C., Allal, L. B., Lozhkov, A., Tazi, N., Srivastav, V., Lochner, J., Larcher, H., Morlon, M., Tunstall, L., Werra, L. v., & Wolf, T. (2025). SmolVLM: Redefining small and efficient multimodal models. *arXiv*. <https://doi.org/10.48550/arXiv.2504.05299>
- Mattamala, M., Frey, J., Libera, P., Chebrolu, N., Martius, G., Cadena, C., Hutter, M., & Fallon, M. (2025). Wild visual navigation: fast traversability learning via pre-trained models and online self-supervision. *Autonomous Robots*, 49(3), 19. <https://doi.org/10.1007/s10514-025-10202-x>
- Muller, M. G., Durner, M., Gawel, A., Sturzl, W., Triebel, R., & Siegwart, R. (2021). A Photorealistic Terrain Simulation Pipeline for Unstructured Outdoor Environments. *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. <https://doi.org/10.1109/IROS51168.2021.9636644>

- Nesnas, I. A., Fesq, L. M., & Volpe, R. A. (2021). Autonomy for Space Robots: Past, Present, and Future. *Current Robotics Reports*, 2(3), 251-263. <https://doi.org/10.1007/s43154-021-00057-2>
- Ono, M., Rothrock, B., Otsu, K., Higa, S., Iwashita, Y., Didier, A., Islam, T., Laporte, C., Sun, V., Stack, K., Sawoniewicz, J., Daftry, S., Timmaraju, V., Sahnoune, S., Mattmann, C. A., Lamarre, O., Ghosh, S., Qiu, D., Nomura, S., . . . Park, H. (2020). MAARS: Machine learning-based Analytics for Automated Rover Systems. *2020 IEEE Aerospace Conference*. <https://doi.org/10.1109/AERO47225.2020.9172271>
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P., Li, S., Misra, I., Rabbat, M., Sharma, V., . . . Bojanowski, P. (2024). DINOv2: Learning Robust Visual Features without Supervision. *arXiv*. <https://doi.org/10.48550/arXiv.2304.07193>
- Orsula, A., Geist, M., Olivares-Mendez, M., & Martinez, C. (2025). Space Robotics Bench: Robot Learning Beyond Earth. *arXiv*. <https://doi.org/10.48550/arXiv.2509.23328>
- Park, N., Kim, W., Heo, B., Kim, T., & Yun, S. (2023). What Do Self-Supervised Vision Transformers Learn?. *arXiv*. <https://doi.org/10.48550/arXiv.2305.00729>
- Perez, I., Mavridou, A., Pressburger, T., Will, A., & Martin, P. J. (2022). Monitoring ROS2: from Requirements to Autonomous Robots. *Electronic Proceedings in Theoretical Computer Science*, 371, 208-216. <https://doi.org/10.4204/EPTCS.371.15>
- Piecznyński, D., Ptak, B., Kraft, M., & Drapikowski, P. (2023). LunarSim: Lunar Rover Simulator Focused on High Visual Fidelity and ROS 2 Integration for Advanced Computer Vision Algorithm Development. *Applied Sciences*, 13(22), 12401. <https://doi.org/10.3390/app132212401>

- Pike, L., Goodloe, A., Morisset, R., & Niller, S. (2010). Copilot: A Hard Real-Time Runtime Monitor. https://doi.org/10.1007/978-3-642-16612-9_26
- Purohit, M., Gajera, B., Malaviya, V., Mehta, I., Kasodekar, K., Adler, J., Lu, S., Rebbapragada, U., & Kerner, H. (2025). Mars-Bench: A Benchmark for Evaluating Foundation Models for Mars Science Tasks. *arXiv*. <https://doi.org/10.48550/arXiv.2510.24010>
- Purohit, M., Gajera, B., Mehta, I., Tokas, B., Adler, J., Lu, S., Dickenshied, S., Diniega, S., Bue, B., Rebbapragada, U., & Kerner, H. (2026). MOMO: Mars Orbital Model Foundation Model for Mars Orbital Applications. *arXiv*. <https://doi.org/10.48550/arXiv.2604.02719>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askeel, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning Transferable Visual Models From Natural Language Supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*. <https://doi.org/10.48550/arXiv.2103.00020>
- Ravi, N., Gabeur, V., Hu, Y., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K. V., Carion, N., Wu, C., Girshick, R., Dollár, P., & Feichtenhofer, C. (2024). SAM 2: Segment Anything in Images and Videos. *arXiv*. <https://doi.org/10.48550/ARXIV.2408.00714>
- Rothrock, B., Kennedy, R., Cunningham, C., Papon, J., Heverly, M., & Ono, M. (2016). SPOC: Deep Learning-based Terrain Classification for Mars Rover Missions. *AIAA SPACE 2016*. <https://doi.org/10.2514/6.2016-5539>
- Royce, R., Kaufmann, M., Becktor, J., Moon, S., Carpenter, K., Pak, K., Towler, A., Thakker, R., & Khattak, S. (2023). Enabling Novel Mission Operations and Interactions with ROSA: The Robot Operating System Agent. *arXiv*. <https://doi.org/10.48550/arXiv.2309.05969>

- Schuster, M. J., Brunner, S. G., Bussmann, K., Büttner, S., Dömel, A., Hellerer, M., Lehner, H., Lehner, P., Porges, O., Reill, J., Riedel, S., Vayugundla, M., Vodermayr, B., Bodenmüller, T., Brand, C., Friedl, W., Grix, I., Hirschmüller, H., Kaßbecker, M., . . . Wedler, A. (2019). Towards Autonomous Planetary Exploration: The Lightweight Rover Unit (LRU), its Success in the SpaceBotCamp Challenge, and Beyond. *Journal of Intelligent & Robotic Systems*, 93(3-4), 461-494.
<https://doi.org/10.1007/s10846-017-0680-9>
- Siméoni, O., Vo, H. V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., . . . Bojanowski, P. (2025). DINOv3. *arXiv*.
<https://doi.org/10.48550/arXiv.2508.10104>
- Swan, R. M., Atha, D., Leopold, H. A., Gildner, M., Oij, S., Chiu, C., & Ono, M. (2021). AI4MARS: A Dataset for Terrain-Aware Autonomous Driving on Mars. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.
<https://doi.org/10.1109/CVPRW53098.2021.00226>
- Venkataramanan, S., Pariza, V., Salehi, M., Knobel, L., Gidaris, S., Ramzi, E., Bursuc, A., & Asano, Y. M. (2026). Franca: Nested Matryoshka Clustering for Scalable Visual Representation Learning. *arXiv*.
<https://doi.org/10.48550/arXiv.2507.14137>
- Verma, V., Maimone, M. W., Gaines, D. M., Francis, R., Estlin, T. A., Kuhn, S. R., Rabideau, G. R., Chien, S. A., McHenry, M. M., Graser, E. J., Rankin, A. L., & Thiel, E. R. (2023). Autonomous robotics is driving Perseverance rover's progress on Mars. *Science Robotics*, 8(80), eadi3099.
<https://doi.org/10.1126/scirobotics.adi3099>
- Voellmy, M., & Ehrhardt, M. (2020). ExoMy: A low cost 3D printed rover. *1st International Symposium on Space Robotics (iSpaRo 2020)*.

- Ward, I. R., Moore, C., Pak, K., Chen, J., & Goh, E. (2022). Improving Contrastive Learning on Visually Homogeneous Mars Rover Images. *arXiv*. <https://doi.org/10.48550/arXiv.2210.09234>
- Xie, Z., Yun, K., Liu, Y., Ji, Y., & Li, H. (2025). Mars Traversability Prediction: A Multi-modal Self-supervised Approach for Costmap Generation. *arXiv*. <https://doi.org/10.48550/arXiv.2509.11082>
- Zhou, R., Feng, W., Ding, L., Yang, H., Gao, H., Liu, G., & Deng, Z. (2022). MarsSim: A high-fidelity physical and visual simulation for Mars rovers. *IEEE Transactions on Aerospace and Electronic Systems*, 1-43. <https://doi.org/10.1109/TAES.2022.3207705>

Appendices

This project involved no human participants, no animal subjects, no survey or interview data, and no personal data of any kind. All imagery used is either from publicly released datasets or was photographed by the author of equipment and laboratory apparatus. Cranfield University Research Ethics System approval was therefore not required, and no ethical approval letter is reproduced here. The appendices that follow document the code, the reproducibility map, the full model table, and the negative results.

Appendix A Code and repository

This appendix documents the code repository behind this thesis, so a reader who only has this report can still see how the work was built and run.

A.1 Repository

The code is hosted as a GitHub repository, `Foundational-Model-for-Rover-Autonomy`. It contains the Python experiment scripts, the ROS2 packages, and the Gazebo simulation assets behind every result in this thesis.

A.2 Top-level structure

Table A.1: Repository structure.

Path	Contents
experiments/	Every experiment script (roughly 100), their output CSVs and figures under experiments/results/, and the MATLAB figure-generation scripts used for this report
ros2_ws/src/fm_perception/	The perception package: CLIP, DINOv2, SmolVLM, and BLIP-2 ROS2 nodes, the traversability controller, the reactive-exploration and stuck-detection state machines, and their pure-Python decision-function modules
ros2_ws/src/fm_imu_fusion/	The IMU driver, slope-fusion logic, and the LiDAR/IMU/camera traversability fusion node
ros2_ws/src/exomy_ros2/	The ported ExoMy ROS2 hardware driver (motor control, locomotion modes) and the cmd_vel-to-RoverCommand bridge
ros2_ws/src/exomy_ros2_msgs/	Custom ROS2 message definitions
simulation/	Gazebo world files, the ExoMy URDF/xacro model, and launch files

A.3 Software environment

Python 3.10+, ROS2 Humble, Gazebo Classic 11, PyTorch 2.5.1, HuggingFace Transformers, ONNX Runtime, OpenCV, scikit-learn, pandas, and MATLAB R2025b (for the figures in this report). The full dependency list is in requirements.txt.

A.4 How to run the core pieces

Table A.2: Commands to build and run the project.

Task	Command
Build the ROS2 workspace	<code>source /opt/ros/humble/setup.bash; source ros2_ws/install/setup.bash; colcon build; source install/setup.bash</code>
Launch Gazebo with ExoMy	<code>bash run_sim.sh</code>
Launch the perception/control nodes	<code>bash run_exp_launch.sh</code>
Run the DINOv2 traversability experiment	<code>bash run_exp.sh</code>
Run a standalone classification experiment	<code>python3 experiments/<script_name>.py</code>
Run the repository's unit tests	<code>python3 -m pytest ros2_ws/src/fm_perception/test/ ros2_ws/src/exomy_ros2/test/ ros2_ws/src/fm_imu_fusion/test/</code>
Regenerate this report's figures and diagrams	<code>matlab -batch "run('experiments/make_thesis_figures.m')"</code> , then the <code>_2</code> , <code>_3</code> , and <code>_4</code> scripts, then <code>make_thesis_diagrams.m</code> and <code>make_gazebo_plates.m</code>

A.5 Testing convention

Every reactive-safety and autonomy behaviour in this thesis (the traversability controller, reactive hazard recovery, stuck detection, path planning, frontier exploration, and the rest of the extended stack in Chapter 3) was built test-first: pure Python decision functions with no ROS2 or Gazebo dependency, tested in isolation, then wrapped in a ROS2 node. At the point of thesis submission the combined test suite across `fm_perception`, `exomy_ros2`, and `fm_imu_fusion` numbers in the low hundreds and passes cleanly. This lets the safety-critical logic (confidence thresholds, hazard state machines, planning) be checked without needing Gazebo or real hardware running.

A.6 What is not in the repository

Real-hardware rosbag recordings and raw camera captures from the hardware test sessions are not committed, since they are large and include images of the laboratory. The trained linear-probe weights and ONNX-exported encoders are regenerable from the AI4Mars dataset and the scripts above rather than committed directly, to keep the repository a reasonable size.

A.7 Porting ExoMy from ROS1 to ROS2

ExoMy's original software, released by ESA alongside the platform (Voellmy & Ehrhardt, 2020), targets ROS1. This thesis ports the essential nodes to ROS2 in two new packages, `exomy_ros2_msgs` for the custom message types and `exomy_ros2` for the control nodes. On the development PC they run in dry-run mode, logging motor commands without sending them, since the Adafruit PWM HAT is absent there; on the Raspberry Pi they run in full hardware mode over the I2C serial bus.

The perception stack and the migrated driver spoke different languages. Perception nodes publish `geometry_msgs/Twist` on `/exomy/cmd_vel`, built for Gazebo's differential-drive plugin, while `robot_node.py` accepts only the legacy ESA `RoverCommand` message, and no bridge existed. `cmd_vel_to_rovercommand_bridge_node.py` closes the gap, translating the pure forward, backward, and in-place rotation commands that are the only motions this thesis produces into ExoMy's `FAKE_ACKERMANN` and `POINT_TURN` modes. A companion fix makes commanded speed scale with the requested magnitude rather than always driving at a fixed 50%. Without it, the controller's soil, sand, and bedrock speed differentiation would have collapsed to a flat on/off response on real hardware.

Building the deployment launch file surfaced a second latent bug of the same kind. `rover.py`'s steering-angle function always returns a Python `int`, but the message field it fills is declared `float32`, so `robot_node.py` had never successfully published a single motor command anywhere in the repository's history. This went unnoticed because every Gazebo result in this thesis drives the simulated plugin directly and bypasses `robot_node.py` entirely. Both fixes carry regression

tests, 22 and 9 respectively, and were verified against a live ROS2 graph with synthetic messages. Neither has been checked against real motors, and which physical turn direction corresponds to which `POINT_TURN` sign is undocumented in the ported codebase and remains unverified (Section 6.4). Section 5.7 returns to what these two bugs say about where simulation testing stops being informative.

`real_hardware_deployment.launch.py` chains the full pipeline into one command, from camera through DINOv2 classification, the traversability controller, reactive exploration and stuck detection (Section 3.12), LiDAR proximity guarding and IMU slope fusion, the bridge above, and finally the motor nodes, with every hardware driver independently toggleable so the file still parses on a non-Raspberry-Pi machine. Three further additions are real-hardware only and opt-in, leaving every Gazebo launch file unaffected: an interchangeable INT8 ONNX inference backend (Section 4.11), two visualisation nodes defaulted off, and rosbag recording for offline session review.

A.8 Video record of the hardware runs

Eight autonomous runs recorded on the ExoMy rover on 4 August 2026 are available as an online playlist. These eight are a sample of the hardware testing carried out, chosen because they were recorded on video, and not the full set of runs. The recordings are supporting evidence only. Every quantitative result in this thesis is measured from logged data and is independent of them, and the thesis is complete without them.

Runs are numbered by playlist order, so that the seven sandpit runs sit together and the single laboratory-floor run comes last. Durations are as shown on the playlist. The rover was under autonomous control throughout each recording, with no operator input.

The sandpit contains laboratory objects placed as LiDAR obstacles, moved between runs. They are not the terrain the classifier is reading. That is the printed AI4Mars imagery on the panels around the walls.

Table A.3: Video record of eight autonomous hardware runs, 4 August 2026.

Run	Duration	Environment
1	1:01	Sandpit, Mars backdrop
2	1:10	Sandpit, Mars backdrop
3	1:12	Sandpit, Mars backdrop
4	0:46	Sandpit, Mars backdrop
5	0:45	Sandpit, Mars backdrop
6	0:55	Sandpit, Mars backdrop
7	0:59	Sandpit, Mars backdrop
8	0:33	Laboratory floor

Playlist: <https://www.youtube.com/playlist?list=PLWII8Zlzh2Es>

Run 8 is the only recording made away from the sandpit. It is included because it shows the same stack operating on a hard indoor floor against everyday objects, which is a different visual and geometric problem from loose sand and printed panels.

Appendix B Reproducibility map

This appendix maps every headline number quoted in this thesis to the exact script and output file that produced it, so any reported figure can be checked or reproduced independently. It is a curated index of headline numbers, not an exhaustive list of every script in the repository (over 100 exist, covering every ablation and diagnostic mentioned in Chapters 3 to 5).

To reproduce a number, run the listed script from the repository root in a Python 3.10+ environment matching `requirements.txt`. It writes the listed CSV or figure to `experiments/results/`. All scripts use a fixed seed (42) where randomness is involved, unless the script itself reports otherwise. Gazebo-dependent results additionally need `bash run_sim.sh` (or the specific launch file named in the source chapter section) running first.

B.1 Core AI4Mars classification results

Table B.1: Core AI4Mars classification results: headline numbers, scripts, and outputs.

Headline number	Script	Output
CLIP zero-shot, 34.8%	<code>experiments/clip_terrain_test.py</code>	<code>model_comparison_overall.csv</code>
CLIP prompt engineering, 54.4%	<code>experiments/clip_prompt_engineering.py</code>	<code>prompt_engineering_ViT-B-32.csv</code>
Few-shot linear probe, 400 labels to 76.0%	<code>experiments/few_shot_linear_probe.py</code>	<code>few_shot_linear_probe_ViT-B-32.csv</code>
DINOv2+reg ViT-S 1000-shot, 90.24% (deployed model)	<code>experiments/dinov2_terrain_test.py</code>	<code>dinov2_reg_small_terrain_few_shot.csv</code>
DINOv2 ViT-L 1000-shot, 93.73%	<code>experiments/dinov2_terrain_test.py</code> (ViT-L config)	<code>dinov2_vitl_terrain_few_shot.csv</code>
Ensemble B, 94.43%	<code>experiments/ensemble_terrain_test.py</code>	<code>ensemble_terrain_few_shot.csv</code>
Ensemble B vs DINOv2 ViT-L, McNemar's test (not significant)	<code>experiments/cross_arch_ensemble.py</code>	<code>mcnemar_results.csv</code>

Headline number	Script	Output
Seed stability (std=0.00 at 1000-shot)	experiments/seed_stability_test.py	seed_stability_results.csv
Temperature scaling (ECE 0.1695 to 0.0325, $T^*=0.461$)	experiments/temperature_scaling.py	confidence_calibration_output
Failure case analysis (28/287 errors)	experiments/failure_case_analysis.py	failure_case_analysis.csv
Label-efficiency curve	experiments/label_efficiency_curve.py	label_efficiency_curve.csv
Coverage-risk curve ($T=0.40$ validated)	experiments/coverage_risk_curve.py	coverage_risk_curve.csv
Confusion matrix and t-SNE, DINOv2 ViT-L (Figures 4.5, 5.1)	experiments/prepare_confusion_tsne_data.py	confusion_matrix_dinov2_vitl.csv, tsne_dinov2_vitl.csv
Big-rock label-set experiment (Table 5.1)	experiments/big_rock_fourclass_probe.py	big_rock_fourclass_probe.csv, big_rock_fourclass_confusion.csv

B.2 Cross-paradigm survey

Table B.2: Cross-paradigm survey (22 models, six pretraining paradigms): headline numbers, scripts, and outputs.

Headline number	Script	Output
AIMv2-L, 87.80%	experiments/aimv2_terrain_test.py	aimv2_terrain_few_shot.csv
Franca-RASA, 89.55%	experiments/franca_rasa_terrain_test.py	franca_rasa_few_shot.csv
DINOv3 ViT-L/16, 92.3%	experiments/dinov3_vitl_terrain_test.py	dinov3_vitl_terrain_few_shot.csv
Depth Anything V2 (36.6%, negative)	depth feature ablation script	depth_anything_terrain_test.csv
Tip-Adapter underperforms LogReg	experiments/tip_adapter_terrain_test.py	tip_adapter_dinov2_terrain_test.csv, tip_adapter_clip_terrain_test.csv
E3 multi-model latency and RAM benchmark	experiments/e3_model_latency_benchmark.py	e3_model_latency_benchmark.csv
Model ranking bar chart (Figure 4.2)	(compiled from the above CSVs)	model_ranking_1000shot.csv

B.3 Cross-sensor and cross-planet domain-gap findings

Table B.3: Cross-sensor and cross-planet domain-gap findings: headline numbers, scripts, and outputs.

Headline number	Script	Output
Mars-Bench zero-shot transfer, 24.94% (65.30 pp gap)	experiments/marsbench_zero_shot_transfer.py	marsbench_zero_shot_transfer.csv
Mars-Bench in-domain probe, 84.07%	experiments/marsbench_dinov2_exp10.py	marsbench_dinov2_exp10_results.csv
E1 joint probe (35.80 to -5.56 pp gap)	experiments/joint_domain_probe.py	e1_joint_domain_probe_results.csv
Earth+Mars cross-planet probe (honest negative)	experiments/earth_mars_crop_extractor.py then earth_mars_probe.py	earth_mars_probe_results.csv
Wilson 95% confidence intervals on headline numbers	experiments/confidence_intervals.py	confidence_intervals.csv

B.4 Raspberry Pi hardware benchmark

Table B.4: Raspberry Pi hardware benchmark: headline numbers, scripts, and outputs.

Headline number	Script	Output
DINOv2 ONNX, 570 ms/image, 258 MB RAM (RPi 4)	RPi-side benchmark harness (run on-device)	rpi_benchmark_raspberrypi.csv
INT8 quantisation, corrected accuracy 86.41%	experiments/dinov2_preprocess_revalidation.py, experiments/int8_subset_bias_decomposition.py	(both scripts' printed output)

B.5 Gazebo simulation and the extended autonomy stack

Table B.5: Gazebo simulation and the extended autonomy stack: headline numbers, scripts, and outputs.

Headline number	Script or launch	Output
DINOV2 Gazebo traversability, 3/5 + 5/5 safety	run_sim.sh + run_exp.sh	dinov2_traversability_experiment.csv
CLIP cascade Gazebo, 2/5	run_sim.sh + run_exp.sh (cascade config)	gazebo_traversability_cascade.csv
Real camera set, 200 photographs, 72 scored by terrain class	experiments/terrain_photo_to_gap_analysis.py	terrain_photo_gap_analysis_summary.csv
Exp 5b pilot, 20 images, 2 classes (superseded)	experiments/sim_to_real_gap_test.py	sim_to_real_gap_results.csv
Config B augmentation, negative result	experiments/gazebo_augmentation_experiment.py	gazebo_aug_results.csv
L5-lite goal reached, t=355.3 s	experiments/l5_lite_live_test.py + launch	l5_lite_live_test.csv
L6-lite round trip, t=828.4 s	experiments/l6_lite_roundtrip_test.py	l6_lite_roundtrip_test.csv
Frontier exploration-lite, 19 selections, 23.1%	experiments/frontier_exploration_test.py	frontier_exploration_test.csv
Explore-then-return-home, 45 selections, 99.2%	experiments/explore_return_home_test.py	explore_return_home_test.csv
A2 re-observation, +16.3 mean confidence delta	experiments/reobservation_test.py	reobservation_test.csv
Log-odds fusion variant, +38.1 mean delta	experiments/reobservation_bayesian_test.py	reobservation_bayesian_test.csv
Mission-level failsafe, both criteria pass	experiments/abort_to_home_test.py	abort_to_home_test.csv

B.6 Notes on reproducibility limits

Real-hardware runs (Experiment 5b, and the hardware confirmation rounds) depend on the physical rover's state (lighting, sandbox layout, sensor mounting)

at the time of capture and are not exactly reproducible; source images and logs are preserved where noted in the originating chapter section. Gazebo results depend on the development machine's real-time factor and may not reproduce exactly on different hardware, though the qualitative safety and classification pattern has been confirmed stable across repeated runs wherever this thesis explicitly reports more than one run of the same test.

Appendix C Full 22-model results table

This table gives the complete 1000-shot AI4Mars results for all 22 evaluated model variants, with parameter count and feature dimension alongside overall and bedrock accuracy, condensed to two figures in the main Results chapter (Figure 4.2 and its accompanying table). It is reproduced here in full for reference.

Table C.1: Full results for all 22 evaluated models across six pretraining paradigms.

Rank	Model	Paradigm	Parameters	Feature dim	Overall accuracy	Bedrock accuracy
1	Ensemble B (DINOv2 ViT-L + ViT-B)	DINO ensemble	1792-d concat	1792	94.43%	91.32%
2	DINOv2 ViT-L/14	DINO	304M	1024	93.73%	90.94%
3	Probabilistic stacking (4-model)	Ensemble	-	-	93.38%	93.50%
4	DINOv3 ViT-L/16	DINO	303M	1024	92.33%	89.13%
5	DINOv2+reg ViT-B	DINO	86M	768	91.99%	88.04%
6	DINOv2 ViT-B/14	DINO	86M	768	91.30%	87.04%
7	DINOv3-SAT ViT-L (satellite-pretrained)	DINO (satellite)	303M	1024	90.94%	93.50%
8	DINOv3 ViT-B/16	DINO	86M	768	90.60%	83.70%
9	DINOv2+reg ViT-S (deployed on Raspberry Pi)	DINO	22M	384	90.24%	84.78%
10	DINOv3 ViT-S/16	DINO	21.6M	384	90.20%	83.70%
11	DINOv2 ViT-S/14	DINO	21.6M	384	89.90%	84.78%
12	Franca-RASA	Positional debiasing	304.8M	1024	89.55%	89.13%
12	Franca-CLS+RASA	Positional debiasing	304.8M	2048	89.55%	88.04%
14	AIMv2-L	Autoregressive	307M	1024	87.80%	77.17%

Rank	Model	Paradigm	Parameters	Feature dim	Overall accuracy	Bedrock accuracy
14	CLIP ViT-B/32	Contrastive	87.8M	512	87.80%	79.30%
14	DINOv3 ConvNeXt-Tiny	DINO (CNN)	29M	-	87.80%	80.40%
17	SigLIP2-B/16	Contrastive (sigmoid)	92.9M	768	86.10%	80.40%
18	Franca-CLS	Matryoshka backbone	304.8M	1024	84.67%	80.43%
19	EVA-02 ViT-B/14	MIM with a CLIP teacher	85.8M	768	81.20%	64.10%
20	SAM2 Hiera-tiny	Segmentation	27.2M	256	75.30%	73.90%
21	RADIO-B	Multi-teacher distillation	98.2M	2304	65.20%	67.40%
22	Depth Anything V2-S	Depth estimation	-	40	36.60%	9.80%
-	DeepLabv3+ (supervised baseline)	Fine-tuned, GPU	-	-	96.67%	94.90%

All 1000-shot results use a logistic regression linear probe ($C=0.316$, seed=42) on frozen encoder features, trained on 1,000 labelled AI4Mars images per class (108 for big rock, capped by that class's own training-set size) and evaluated on the 287-image gold-standard test set. The gap to the supervised baseline quoted elsewhere in this thesis is each model's overall accuracy subtracted from the DeepLabv3+ result of Swan et al. (2021), 96.67%, and is not repeated as its own column here.

Appendix D Negative, null, and low-N results

This appendix collects every negative, null, or statistically thin result reported in this thesis into a single index. Nothing here is new. Each row summarises a result already reported in full in its own chapter section, with a pointer back to that section. The purpose is to give the reader one place to see that unfavourable findings were reported honestly throughout, rather than reframed or left out.

Table D.1: Negative, null, and low-N results.

Result	What was tested	Outcome	Reported in
D1, Nav2 waypoint navigation	Whether a live hazard-aware costmap changes Nav2 navigation versus a static one	Null. 0 of 20 trials reached the goal in either condition; both statistically indistinguishable, traced to Gazebo real-time-factor throttling on the development machine	Chapter 4
LiDAR/IMU fusion, rock-cluster test	Whether the fused LiDAR risk term detects nearby rocks	Negative geometric finding. The fusion arithmetic itself is correct, but the LiDAR term never rose, because the rock field sits below the simulated LiDAR's scan height	Chapter 4
Boundary-band uncertainty	Whether classification confidence falls near a class boundary (the mechanism inferred from two live deadlocks)	Clean null. Pearson $r = -0.078$ ($n=578$), no significant relationship	Chapter 4
Two-stage uncertain policy (vote gate)	Whether a majority vote reduces false-STOP episodes from single-frame flicker	Genuine but low-N. Only 2 uncertain-STOP episodes across 721 frames; both correctly caught, but too few to show a large-sample effect	Chapter 4
VLM-queryable map feasibility	Whether SmolVLM-256M can answer questions over a serialised map summary, text-only	Negative. 2 of 3 basic questions answered wrong; 256M parameters too small for reliable numeric reasoning even with the vision encoder dropped	Chapter 5
Earth+Mars cross-planet probe	Whether a frozen encoder trained on one planet transfers to the other	Pre-registered honest-negative. Both directions fail the positive bar (13.45% and 35.74%, against a chance level of 33.3%), though the sanity gate passes comfortably on each planet alone	Chapters 4 and 5

Result	What was tested	Outcome	Reported in
Depth Anything V2 for Mars terrain	Whether Earth-pretrained monocular depth gives useful geometric features for Mars terrain classification	Negative, bounding result. Depth-only accuracy 36.6%; the gradient-roughness feature shows essentially zero class separability; fusing depth with DINOv3 does not improve on DINOv3 alone	Chapters 4 and 5
Multi-layer feature concatenation	Whether concatenating multiple transformer layers improves on the final-layer CLS token	No improvement, at any scale tested (ViT-S or ViT-L)	Chapters 4 and 5
Tip-Adapter (training-free few-shot)	Whether a training-free key-value matching method beats a logistic regression probe	Negative. Underperforms the probe by 5 to 18 points at every shot count tested	Chapter 4
Test-time augmentation on the linear probe	Whether averaging original and flipped features improves 1000-shot accuracy	Negative. 89.90% against a 90.24% single-view baseline, a 0.35-point drop, in <code>tta_linear_probe.py</code>	Repository only
Config B feature-space augmentation	Whether adding Gazebo-domain rock features to training fixes big-rock misclassification in Gazebo	Made results worse. Every Gazebo zone, not just the rocks, was swept into the big-rock class	Chapter 4
Domain randomisation of rock-zone position (X3)	Whether rock detection generalises across the whole Gazebo quadrant, not just the fixed spawn points	Negative. 0 of 10 random positions correct; rock detection only works within 2 to 3 metres of the rock zone	Chapter 4

This list covers results that were run and reported as negative, null, or low-N. It does not include ideas that were scoped and explicitly rejected before any implementation, such as a monocular visual-odometry path for localisation or a full Nav2-based path planner; those decisions are discussed in their own right in Chapter 5's future work and scope sections rather than listed here as executed results.